

/THEORY/IN/PRACTICE

The Art of Readable Code

Simple and Practical Techniques for Writing Better Code

O'REILLY®

Dustin Boswell
Trevor Foucher

The Art of Readable Code

"Being aware of how the code you create affects those who look at it later is an important part of developing software. The authors did a great job in taking you through the different aspects of this challenge, explaining the details with instructive examples."

—Michael Hunger, Software Developer, Neo Technology

As programmers, we've all seen source code that's so ugly and buggy it makes our brain ache. Over the last five years, authors Dustin Boswell and Trevor Foucher have analyzed hundreds of examples of "bad code" (much of it their own) to determine why they're bad and how they could be improved. Their conclusion? You need to write code that minimizes the time it would take someone else to understand it—even if that someone else is you.

This book focuses on basic principles and practical techniques you can apply every time you write code. Using easy-to-digest code examples from different languages, each chapter dives into a different aspect of coding, and demonstrates how you can make your code easy to understand.

- 
- Simplify naming, commenting, and formatting with tips that apply to every line of code
 - Refine your program's loops, logic, and variables to reduce complexity and confusion
 - Attack problems at the function level, such as reorganizing blocks of code to do one task at a time
 - Write effective test code that is thorough and concise—as well as readable

Dustin Boswell graduated from CalTech, and worked at Google for five years on web crawling infrastructure and ad programs. He's built several websites and enjoys working on big data and machine learning.

Trevor Foucher has spent the past decade at Microsoft as an engineer, manager, and tech lead on Windows and security products, and at Google working on ad programs and search infrastructure.

US \$34.99 CAN \$36.99
ISBN: 978-0-596-80229-5



Twitter: @oreillymedia
facebook.com/oreilly

O'REILLY®
oreilly.com

代码可读性艺术

达斯汀·波斯韦尔和特雷弗·富歇

O'REILLY®

北京·剑桥·法纳姆·科隆·塞瓦斯托波尔·东京

代码可读性艺术

作者：达斯汀·博斯韦尔和特雷弗·富歇

版权所有 © 2012 达斯汀·博斯韦尔和特雷弗·富歇。保留所有权利。
在美国印刷。

由 O’ Reilly Media, Inc. 出版，地址：1005 Gravenstein Highway North, Sebastopol, CA 95472。

O’ Reilly 的书籍可以购买用于教育、商业或销售推广用途。大多数书名也有在线版本 (<http://my.safaribooksonline.com>)。如需更多信息，请联系我们的公司/机构销售部门：(800) 998-9938 或 corporate@oreilly.com。

编辑：玛丽·特雷塞勒
制作编辑：特蕾莎·埃尔西
文案编辑：南希·沃尔夫·科塔里
校对员：特蕾莎·埃尔西

索引员：Potomac Indexing, LLC
封面设计师：苏珊·汤普森
内部设计师：大卫·富塔托
插图画家：戴夫·奥尔雷德和罗伯特·罗马诺

2011年11月：第一版。

第一版修订历史：

2011-11-01 首次发布

有关发布详情，请参阅 <http://oreilly.com/catalog/errata.csp?isbn=9780596802295>。

Nutshell Handbook、Nutshell Handbook 标志和 O’ Reilly 标志是 O’ Reilly Media, Inc. 的注册商标。可读代码的艺术、乐谱图像和相关商业外观是 O’ Reilly

Media, Inc. 的商标。

制造商和销售商用来区分其产品的许多名称都被声明为
商标。如果这些名称出现在本书中，并且 O’ Reilly Media, Inc. 知道有商标
声明，则这些名称已以大写或首字母大写形式印刷。

虽然在本书的编写过程中已采取一切预防措施，但出版商和作者对错误或遗漏，或因使用本文包含的信息而造成的损失不承担任何责任。

ISBN: 978-0-596-80229-5

[LSI]

1320175254

目录

前言	vii
1 代码应该易于理解	1
是什么让代码“更好”？	2
可读性基本定理	3
越小越好吗？	3
理解所需时间是否与其他目标冲突？	4
困难的部分	4
第一部分表面改进	
2 将信息打包到名称中	7
选择特定的词语	8
避免使用像 tmp 和 retval 这样的通用名称	10
优先使用具体名称而不是抽象名称	13
将额外信息附加到名称	15
名称应该有多长？	18
使用名称格式来传达含义	20
总结	21
3 不会被误解的名字	23
示例：Filter()	24
示例：Clip(text, length)	24
对于（包含性）限制，首选 min 和 max	25
对于包含性范围，首选 first 和 last	26
首选包含/排除范围的开始和结束	26
命名布尔值	27
匹配用户的期望	27
示例：评估多个名称候选者	29
总结	31
4 美学	33
为什么美学很重要？	34
重新排列换行符以保持一致和紧凑	35
使用方法清理不规则性	37
在有帮助时使用列对齐	38
选择一个有意义的顺序，并始终如一地使用它	39
将声明组织成块	40
将代码分解为“段落”	41
个人风格与一致性	42
总结	43

5	知道该注释什么	45
	不该评论什么	47
	记录你的想法	49
	设身处地为读者着想	51
	最后的想法——克服写作障碍	56
	总结	57
6	使评论更精确和紧凑	59
	保持注释简洁	60
	避免使用含义不清的代词	60
	润色粗糙的句子	61
	精确描述函数行为	61
	使用说明边界情况的输入/输出示例	61
	阐明你的代码意图	62
	“命名函数参数”注释	63
	使用信息密集型词语	64
	总结	65
第二部分简化循环和逻辑		
7	使控制流易于阅读	69
	条件语句中参数的顺序	70
	if/else 块的顺序	71
	?: 条件表达式（又名“三元运算符”）	73
	避免 do/while 循环	74
	从函数中提前返回	75
	臭名昭著的 goto	76
	最小化嵌套	77
	你能否追踪执行流程？	79
	总结	80
8	分解大型表达式	83
	解释变量	84
	摘要变量	84
	使用德摩根定律	85
	滥用短路逻辑	86
	示例：与复杂逻辑作斗争	86
	分解巨大的语句	89
	另一种简化表达式的创造性方法	90
	总结	90
9	变量与可读性	93
	消除变量	94
	缩小变量的作用域	97
	倾向于只写一次的变量	103
	最后一个例子	104
	总结	106

第三部分重组你的代码

10	提取无关的子问题	109
	入门示例: <i>findClosestLocation()</i>	110
	纯实用代码	111
	其他通用代码	112
	创建大量通用代码	114
	项目特定功能	115
	简化现有接口	116
	根据您的需求重塑接口	117
	做得太过分	117
	总结	118
11	一次只做一件事	121
	任务可以很小	123
	从对象中提取值	124
	一个更大的例子	128
	总结	130
12	将想法转化为代码	131
	清晰地描述逻辑	132
	了解你的库会有帮助	133
	将此方法应用于更大的问题	134
	总结	137
13	编写更少的代码	139
	不要费心实现那个功能——你不会需要它的	140
	质疑并分解你的需求	140
	保持你的代码库小巧	142
	熟悉你周围的库	143
	示例: 使用 Unix 工具而不是编码	144
	总结	145

第四部分精选主题

14	测试与可读性	149
	使测试易于阅读和维护	150
	这个测试有什么问题?	150
	使这个测试更具可读性	151
	使错误消息可读	154
	选择好的测试输入	156
	命名测试函数	158
	那个测试有什么问题?	159
	测试友好的开发	160
	走得太远	162
	总结	162
15	设计并实现一个“分钟/小时计数器”	165
	问题	166
	定义类接口	166

	尝试 1: 一个幼稚的解决方案	169
	尝试 2: 传送带设计	171
	尝试 3: 基于时间桶的设计	174
	三种解决方案的比较	179
	总结	179
A	进一步阅读	181
	索引	185

前言



我们曾在非常成功的软件公司工作过，与杰出的工程师共事，但我们遇到的代码仍然有很大的改进空间。事实上，我们见过一些非常糟糕的代码，你可能也见过。

但是，当我们看到写得漂亮的代码时，它会令人鼓舞。好的代码可以很快地教会你正在发生的事情。它使用起来很有趣，并且会激励你使自己的代码更好。

本书的目标是帮助你改进你的代码。当我们说“代码”时，我们实际上指的是你在编辑器中盯着看的代码行。我们不是在谈论你项目的整体架构，或者你选择的设计模式。这些当然很重要，但根据我们的经验，我们作为程序员的大部分日常工作都花在“基本”的东西上，比如命名变量、编写循环以及在函数级别上解决问题。其中很大一部分是阅读和编辑已经存在的代码。我们希望你会发现这本书对你的日常编程非常有帮助，以至于你会把它推荐给你的团队中的每个人。

本书是关于什么的

本书是关于如何编写高度可读的代码。本书的关键思想是代码应该易于理解。具体来说，你的目标应该是尽量减少其他人理解你的代码所需的时间。

本书解释了这个想法，并用来自不同语言的许多例子来说明它，包括 C++、Python、JavaScript 和 Java。我们避免了任何高级语言特性，所以即使你不知道所有这些语言，也应该很容易理解。（根据我们的经验，可读性的概念在很大程度上是与语言无关的。）

每一章都会深入探讨编码的不同方面，以及如何使其“易于理解”。
本书分为四个部分：

表面层次的改进

命名、注释和美学——适用于代码库中每一行的简单技巧

简化循环和逻辑

改进程序中的循环、逻辑和变量，使其更易于理解的方法

重新组织你的代码

更高级别的方式来组织大型代码块，并在函数级别上解决问题

精选主题

将“易于理解”应用于测试和更大的数据结构编码示例

如何阅读本书

我们的书旨在成为一本有趣、轻松的读物。我们希望大多数读者在一两周内读完这本书。

这些章节按“难度”排序：基本主题在开头，更高级的主题在结尾。但是，每个章节都是独立的，可以单独阅读。所以，如果你愿意，可以随意跳过。

使用代码示例

本书旨在帮助您完成工作。一般来说，您可以在您的程序和文档中使用本书中的代码。除非您要复制代码的很大一部分，否则您无需联系我们获得许可。例如，编写一个使用本书中几个代码块的程序不需要许可。出售或分发来自 O'Reilly 书籍的示例的 CD-ROM 需要许可。通过引用本书并引用示例代码来回答问题不需要许可。将本书中的大量示例代码纳入您产品的文档中需要许可。

我们感谢但不要求署名。署名通常包括标题、作者、出版商和 ISBN。例如：“可读代码的艺术作者：Dustin Boswell 和 Trevor Foucher。版权所有 2012 Dustin Boswell 和 Trevor Foucher，978-0-596-80229-5。”如果您认为您对代码示例的使用超出了合理使用范

围或上述许可范围，请随时通过 permissions@oreilly.com 与我们联系。

Safari® Books Online



Safari Books Online 是一个按需数字图书馆，可让您轻松搜索超过 7,500 本技术和创意参考书籍和视频，以快速找到您需要的答案。

通过订阅，您可以阅读我们图书馆中的任何页面并观看任何视频。在您的手机和移动设备上阅读书籍。在新书印刷之前访问它们，并获得对手稿的独家访问权限，并在开发中发布作者的反馈。复制并粘贴代码示例，整理您的收藏夹，下载章节，为关键部分添加书签，创建笔记，打印页面，并从大量其他省时功能中受益。

O'Reilly Media 已将本书上传到 Safari Books Online 服务。要完全访问本书以及 O'Reilly 和其他出版商关于类似主题的其他书籍，请在 <http://my.safaribooksonline.com> 免费注册。

如何联系我们

请将有关本书的意见和问题发送给出版商：

O'Reilly Media, Inc.
1005 Gravenstein Highway North
Sebastopol, CA 95472
800-998-9938 (美国或加拿大境内)
707-829-0515 (国际或本地)
707-829-0104 (传真)

我们为本书设立了一个网页，其中列出了勘误表、示例和任何其他信息。您可以通过以下网址访问此页面：

<http://shop.oreilly.com/product/9780596802301.do>

要评论或询问有关本书的技术问题，请发送电子邮件至：

bookquestions@oreilly.com

有关我们的书籍、课程、会议和新闻的更多信息，请访问我们的网站：

<http://www.oreilly.com>。

在Facebook上找到我们：<http://facebook.com/oreilly>

在Twitter上关注我们：<http://twitter.com/oreillymedia>

在YouTube上观看我们：<http://www.youtube.com/oreillymedia>

致谢

我们要感谢我们的同事们，他们贡献了自己的时间来审阅我们的整篇手稿，包括 Alan Davidson、Josh Ehrlich、Rob Konigsberg、Archie Russell、Gabe W. 和 Asaph Zemach。书中如有任何错误，完全是他们的责任（开玩笑）。

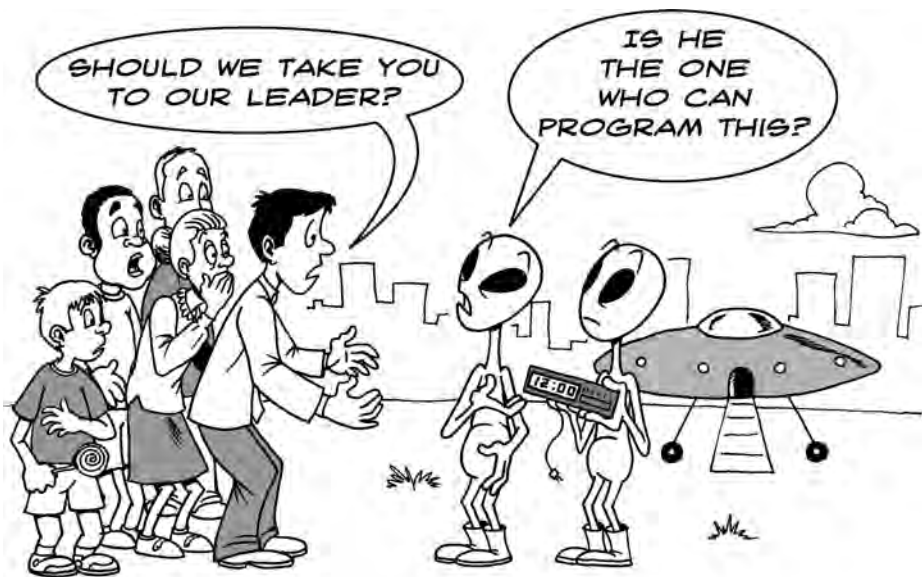
我们感谢许多审阅者对我们书籍的各个草案提出了详细的反馈，包括 Michael Hunger、George Heineman 和 Chuck Hudson。

我们还从 John Blackburn、Tim Dasilva、Dennis Geels、Steve Gerding、Chris Harris、Josh Hyman、Joel Ingram、Erik Mavrinac、Greg Miller、Anatole Paine 和 Nick White 那里获得了许多想法和反馈。感谢众多在线评论员在 O'Reilly 的 OFPS 系统上审阅了我们的草稿。

感谢 O'Reilly 团队的无限耐心和支持，特别是 Mary Treseler（编辑）、Teresa Elsey（制作编辑）、Nancy Kotary（文案编辑）、Rob Romano（插画家）、Jessica Hosman（工具）和 Abby Fox（工具）。还要感谢我们的漫画家 Dave Allred，他让我们的疯狂卡通想法变成了现实。

最后，我们要感谢梅丽莎和苏珊娜，感谢她们一路上的鼓励，以及忍受我们没完没了的编程对话。

代码应该易于理解



在过去的五年里，我们收集了数百个“糟糕代码”的例子（其中很多是我们自己的），并分析了是什么让它们变得糟糕，以及使用了哪些原则/技术来使它变得更好。我们注意到，所有的原则都源于一个单一的主题。

关键思想

代码应该易于理解。

我们认为这是你在决定如何编写代码时可以使用的最重要的指导原则。在本书中，我们将展示如何将这一原则应用于不同的日常编码方面。但在我们开始之前，我们将详细阐述这一原则，并证明它为何如此重要。

是什么让代码“更好”？

大多数程序员（包括作者）都根据直觉和直觉做出编程决策。我们都知道这样的代码：

```
for (Node* node = list->head; node != NULL; node = node->next)
    Print(node->data);
```

比这样的代码更好：

```
Node* node = list->head;
if (node == NULL) return;

while (node->next != NULL) {
    Print(node->data);
    node = node->next;
}
if (node != NULL) Print(node->data);
```

（即使这两个例子表现完全相同）。

但在很多时候，这是一个更艰难的选择。例如，这段代码：

```
return exponent >= 0 ? mantissa * (1 << exponent) : mantissa / (1 << -exponent);
```

比以下代码更好还是更差：

```
if (exponent >= 0) {
    return mantissa * (1 << exponent);
} else {
    return mantissa / (1 << -exponent);
}
```

第一个版本更紧凑，但第二个版本不那么吓人。哪个标准更重要？一般来说，你如何决定以哪种方式编写代码？

可读性基本定理

在研究了许多这样的代码示例后，我们得出一个结论，即存在一个比任何其他指标都更重要的可读性指标。它非常重要，我们称之为“可读性基本定理”。

关键思想

编写代码应尽量减少其他人理解它所需的时间。

这是什么意思？确切地说，如果你找一个你典型的同事，并测量他阅读你的代码并理解它需要多少时间，这个“理解时间”就是你想要最小化的理论指标。

当我们说“理解”时，我们对这个词的要求非常高。要让某人完全理解你的代码，他们应该能够修改它、发现错误，并理解它如何与你代码的其余部分交互。

现在，你可能会想，*谁会关心别人是否能理解它呢？只有我一个人在使用代码！*即使你是一个人的项目，也值得追求这个目标。那个“别人”可能是你六个月后，当你自己的代码对你来说看起来很陌生时。而且你永远不知道——可能会有人加入你的项目，或者你的“一次性代码”可能会被重用于另一个项目。

越小越好吗？

一般来说，解决一个问题，你写的代码越少越好（见第13章，[编写更少的代码](#)）。理解一个2000行的类可能比一个5000行的类花费更少的时间。

但是更少的行并不总是更好！有很多时候，像下面这样的一行表达式：

```
assert((!(bucket = FindBucket(key))) || !bucket->IsOccupied());
```

比写成两行需要更多的时间来理解：

```
bucket = FindBucket(key);  
if (bucket != NULL) assert(!bucket->IsOccupied());
```

类似地，注释可以让你更快地理解代码，即使它“向文件中添加了代码”：

```
// "hash = (65599 * hash) + c" 的快速版本  
hash = (hash << 6) + (hash << 16) - hash + c;
```

因此，即使拥有更少的代码行是一个好目标，但最大限度地缩短理解时间是一个更好的目标。

理解所需时间是否与其他目标冲突？

你可能会想，*其他的约束条件呢，比如让代码高效、架构良好或者易于测试等等？这些难道不会和让代码易于理解的目标相冲突吗？*

我们发现这些其他的目标几乎不会产生干扰。即使是在高度优化的代码领域，仍然有办法使其具有高度的可读性。而且，让你的代码易于理解通常会导致代码架构良好且易于测试。

本书的其余部分讨论了如何在不同的情况下应用“易于阅读”。但请记住，如有疑问，可读性基本定理胜过本书中的任何其他规则或原则。此外，一些程序员有强迫症，需要修复任何没有完美分解的代码。始终重要的是退一步问，*这段代码容易理解吗？*如果是这样，那么继续处理其他代码可能就没问题了。

困难的部分

是的，不断思考一个假想的局外人是否会觉得你的代码容易理解，这需要额外的工作。这样做需要开启你大脑中可能以前在编码时没有开启的部分。

但是，如果你采纳了这个目标（就像我们一样），我们确信你将成为一个更好的程序员，减少错误，对你的工作感到更加自豪，并生成周围每个人都喜欢使用的代码。那么，让我们开始吧！

表面层次的改进

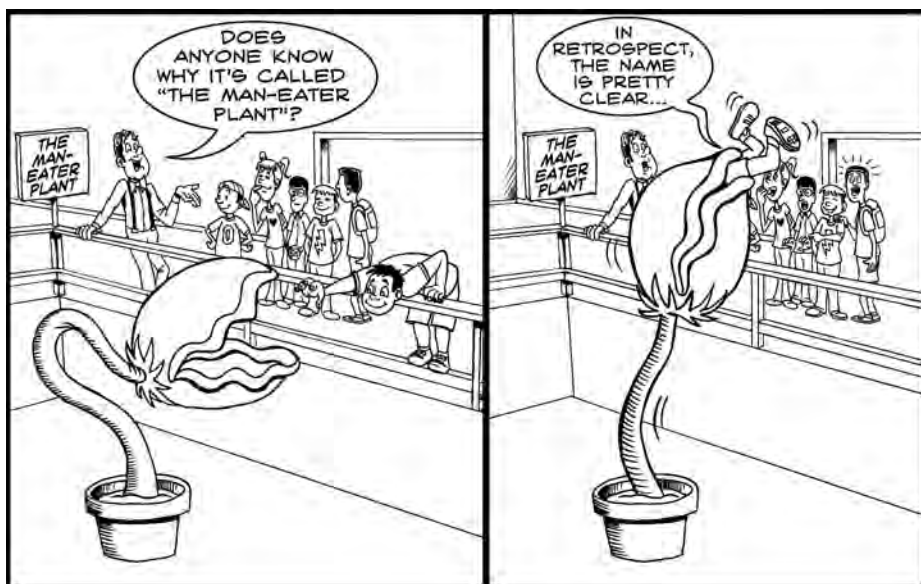
我们从我们认为是“表面层次”的改进开始我们的可读性之旅：选择好的名称，编写好的注释，并整洁地格式化你的代码。这些类型的更改很容易应用。你可以“就地”进行更改，而无需重构你的代码或更改程序的运行方式。你也可以逐步进行更改，而无需大量的时间投入。

这些主题非常重要，因为它们会影响你代码库中的每一行代码。

虽然每个改动看起来都很小，但总的来说，它们可以对代码库进行巨大的改进。如果你的代码有很好的命名、写得很好的注释和清晰的空格使用，你的代码将会更容易阅读。

当然，当涉及到可读性时，表面之下还有很多东西（我们将在本书的后面部分介绍）。但是，这部分内容适用范围非常广，而且只需付出很少的努力，因此值得首先介绍。

将信息打包到名称中



无论您是命名变量、函数还是类，都适用许多相同的原则。我们喜欢将名称视为一个微小的注释。即使没有太多空间，您也可以通过选择一个好名称来传达大量信息。

关键思想

将信息打包到您的名称中。

我们在程序中看到的很多名称都很模糊，比如 `tmp`。即使是像 `size` 或 `get` 这样看似合理的词，也没有包含太多信息。本章将向您展示如何选择能够做到这一点的名称。

本章分为六个具体主题：

- 选择特定的词语
- 避免使用通用名称（或知道何时使用它们）
- 使用具体的名称代替抽象的名称
- 通过使用后缀或前缀，将额外的信息附加到名称上
- 决定名称应该有多长
- 使用名称格式来打包额外的信息

选择特定的词语

“将信息打包到名称中”的一部分是选择非常具体的词语，并避免使用“空洞”的词语。

例如，单词“`get`”非常不具体，如下例所示：

```
def GetPage(url):  
    ...
```

单词“`get`”并没有真正说明什么。这个方法是从本地缓存、数据库还是从互联网获取页面？如果是从互联网获取，一个更具体的名称可能是 `FetchPage()` 或 `DownloadPage()`。

这是一个 `BinaryTree` 类的例子：

```
class BinaryTree {  
    int Size();  
    ...  
};
```

你期望 `Size()` 方法返回什么？树的高度，节点的数量，还是树的内存占用？

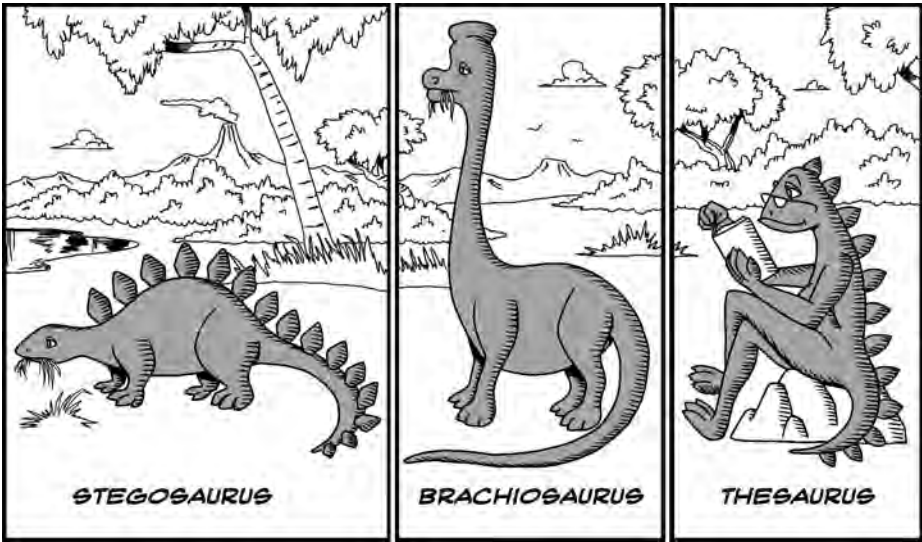
问题在于 `Size()` 并不能传达很多信息。一个更具体的名字应该是 `Height()`、`NumNodes()`，或者 `MemoryBytes()`。

作为另一个例子，假设你有一些 Thread 类：

```
class Thread {  
    void Stop();  
    ...  
};
```

Stop() 这个名字还可以，但取决于它具体做什么，可能会有一个更具体的名字。例如，如果这是一个无法撤销的重量级操作，你可以将其称为 Kill()。或者，如果有一种方法可以 Resume()，你可以将其称为 Pause()。

寻找更多“色彩鲜艳”的词语



不要害怕使用词典或向朋友寻求更好的名字建议。英语是一种丰富的语言，有很多词可以选择。

以下是一些单词的示例，以及可能适用于您情况的更“生动”的版本：

单词	替代词
发送	递送，派遣，宣布，分发，路由
查找	搜索，提取，定位，恢复
开始	启动，创建，开始，打开
制作	创建，设置，构建，生成，组成，添加，新建

不过，别得意忘形。在 PHP 中，有一个函数可以 explode() 一个字符串。这是一个生动的名字，它很好地描绘了将某物分解成碎片的样子，但它有什么不同

来自 `split()`？（这两个函数不同，但很难根据

名称来猜测它们的区别。）

关键思想

清晰和精确比可爱更好。

避免使用像 `tmp` 和 `retval` 这样的通用名称

像 `tmp`、`retval` 和 `foo` 这样的名称通常是意味着“我想不出名字”的逃避方式。不要使用像这样的空名称，选择一个描述实体的值或

目的的名称。

例如，这是一个使用 `retval` 的 JavaScript 函数：

```
var euclidean_norm = function (v) {  
    var retval = 0.0;  
    for (var i = 0; i < v.length; i += 1)  
        retval += v[i] * v[i];  
    return Math.sqrt(retval);  
};
```

当你无法想到更好的返回值名称时，使用 `retval` 很有诱惑力。但是 `retval` 除了“我是一个返回值”（这通常是显而易见的）之外，没有包含太多信息。

一个更好的名称应该描述变量的目的或它包含的值。在这种情况下，该变量正在累加 `v` 的平方和。所以一个更好的名字是 `sum_squares`。这会预先声明变量的目的，并可能有助于发现错误。

例如，想象一下循环内部意外地变成了：

```
retval += v[i];
```

如果名称是 `sum_squares`，这个错误会更明显：

```
sum_squares += v[i]; // 我们正在求和的“平方”在哪里？错误！
```

建议

名称 `retval` 没有包含太多信息。相反，使用一个描述变量值的名称。

然而，在某些情况下，通用名称确实带有含义。让我们来看看什么时候使用它们是有意义的。

tmp

考虑一下交换两个变量的经典案例：

```
if (right < left) {  
    tmp = right;  
    right = left;  
    left = tmp;  
}
```

在这些情况下，名称tmp 非常合适。该变量的唯一目的是临时存储，生命周期只有几行。名称tmp 向读者传达了特定的含义——这个变量没有其他职责。它不会传递给其他函数或被重置或多次重用。

但这里有一个例子，tmp 只是因为懒惰而被使用：

```
String tmp = user.name();  
tmp += " " + user.phone_number();  
tmp += " " + user.email();  
...  
template.set("user_info", tmp);
```

即使这个变量的生命周期很短，但作为临时存储并不是这个变量最重要的事情。相反，像user_info 这样的名称会更具描述性。

在以下情况下，tmp 应该在名称中，但只是作为其中的一部分：

```
tmp_file = tempfile.NamedTemporaryFile()  
...  
SaveData(tmp_file, ...)
```

请注意，我们将变量命名为 tmp_file 而不仅仅是 tmp，因为它是一个文件对象。想象一下如果我们只是把它叫做 tmp：

```
SaveData(tmp, ...)
```

仅看这一行代码，并不清楚 tmp 是一个文件、文件名，还是正在写入的数据。

建议

**名称 tmp 应该只在短暂且临时
是关于该变量的最重要事实使用。**

循环迭代器

像 `i`、`j`、`iter` 和 `it` 这样的名称通常用作索引和循环迭代器。即使这些名称是通用的，它们也被理解为“我是一个迭代器”。（事实上，如果你将这些名称中的一个用于其他目的，那将会令人困惑——所以不要这样做！）

但有时会有比 `i`、`j` 和 `k` 更好的迭代器名称。例如，以下循环查找哪些用户属于哪些俱乐部：

```
for (int i = 0; i < clubs.size(); i++)
    for (int j = 0; j < clubs[i].members.size(); j++)
        for (int k = 0; k < users.size(); k++)
            如果 (clubs[i].members[j] == users[k])
                << "j" << endl; cout << "user[" << j << "] is in club[" << i << "]" << endl;
```

在 `if` 语句中，`members[j]` 和 `users[k]` 使用了错误的索引。像这样的错误很难被发现，因为那行代码单独看起来没问题：

```
if (clubs[i].members[k] == users[j])
```

在这种情况下，使用更精确的名称可能会有所帮助。与其命名循环索引 (`i, j, k`)，另一种选择是 (`club_i, members_i, users_i`) 或者更简洁地 (`ci, mi, ui`)。这种方法将有助于更突出地显示错误：

```
if (clubs[ci].members[ui] == users[mi]) # 错误！首字母不匹配。
```

如果使用正确，索引的第一个字母将与数组的第一个字母匹配：

```
if (clubs[ci].members[mi] == users[ui]) # OK。首字母匹配。
```

关于通用名称的结论

正如你所见，在某些情况下，通用名称是有用的。

建议

如果你要使用像 `tmp`、`it` 或 `retval` 这样的通用名称，请给出一个充分的理由这样做。

很多时候，它们纯粹是因为懒惰而被过度使用。这是可以理解的——当没有更好的想法时，使用像 `foo` 这样的无意义的名字然后继续前进更容易。但是，如果你养成习惯，多花几秒钟想出一个好名字，你会发现你的“命名肌肉”会迅速增强。

优先使用具体名称而不是抽象名称



在命名变量、函数或其他元素时，应具体地描述它，而不是抽象地描述。

例如，假设你有一个名为`ServerCanStart`的内部方法，它测试服务器是否可以监听给定的 TCP/IP 端口。名称`ServerCanStart`有些抽象。一个更具体的名称是`CanListenOnPort()`，这个名字直接描述了该方法将要做什么。

接下来的两个例子更深入地说明了这个概念。

示例：DISALLOW_EVIL_CONSTRUCTORS

这是一个来自 Google 代码库的例子。在 C++ 中，如果你没有为你的类定义复制构造函数或赋值运算符，则会提供一个默认值。虽然方便，但这些方法

很容易导致内存泄漏和其他事故，因为它们在你可能没有意识到的地方“在幕后”执行。

因此，谷歌有一个约定，禁止使用这些“邪恶”的构造函数，使用一个宏：

```
class ClassName {
private:
    DISALLOW_EVIL_CONSTRUCTORS(ClassName);

public:
    ...
};
```

这个宏定义为：

```
#define DISALLOW_EVIL_CONSTRUCTORS(ClassName) \
    ClassName(const ClassName&);\
    void operator=(const ClassName&);
```

通过将此宏放在类的 `private` 部分，这两个方法变为私有，因此它们不能被使用，即使是意外地。

名称 `DISALLOW_EVIL_CONSTRUCTORS` 不是很好。单词“evil”的使用传达了对一个有争议的问题过于强烈的立场。更重要的是，不清楚该宏禁止什么。它禁止 `operator=()` 方法，而这甚至不是一个“构造函数”！

这个名字使用了多年，但最终被更不具挑衅性和更具体的东西所取代：

```
#define DISALLOW_COPY_AND_ASSIGN(ClassName) ...
```

示例：--run_locally

我们的一个程序有一个可选的命令行标志，名为 `--run_locally`。这个标志会导致程序打印额外的调试信息，但运行速度会更慢。该标志通常在本地机器（如笔记本电脑）上进行测试时使用。但是当程序在远程服务器上运行时，性能非常重要，因此不使用该标志。

你可以看到 `--run_locally` 这个名字的由来，但它存在一些问题：

- 团队的一位新成员不知道它的作用。他会在本地运行时使用它（可以想象），但他不知道为什么需要它。
- 偶尔，我们需要在程序远程运行时打印调试信息。将 `--run_locally` 传递给远程运行的程序看起来很奇怪，而且令人困惑。
- 有时我们会在本地运行性能测试，不希望日志记录减慢速度，因此我们不会使用 `--run_locally`。

问题在于 `--run_locally` 是根据它通常使用的环境命名的。相反，像 `--extra_logging` 这样的标志名称会更直接和明确。

但是如果`--run_locally`需要做的不仅仅是额外的日志记录呢？例如，假设它需要设置和使用一个特殊的本地数据库。现在`--run_locally`这个名字似乎更有吸引力，因为它可以同时控制这两个功能。

但是为此目的使用它会选择一个因为它含糊不清且间接的名称，这可能不是一个好主意。更好的解决方案是创建第二个标志，名为`--use_local_database`即使现在你必须使用两个标志，这些标志也更加明确；它们不会试图将两个正交的想法合并为一个，并且它们让你选择只使用一个而不使用另一个。

将额外信息附加到名称



正如我们之前提到的，变量的名称就像一个微小的注释。即使空间不大，你挤进名称的任何额外信息都会在每次看到变量时被看到。

因此，如果关于一个变量有一些非常重要的读者必须知道的事情，那么在名称上附加一个额外的“单词”是值得的。例如，假设你有一个包含十六进制字符串的变量：

```
string id; // 示例: "af84ef845cd8"
```

如果读者需要记住ID的格式，你可能想把它命名为 `hex_id` 代替。

带单位的值

如果你的变量是一个测量值（例如时间量或字节数），将单位编码到变量的名称中会很有帮助。

例如，这里有一些 JavaScript 代码，用于测量网页的加载时间：

```
var start = (new Date()).getTime(); // 页面顶部
...
var elapsed = (new Date()).getTime() - start; // 页面底部
document.writeln("加载时间为: " + elapsed + " 秒");
```

这段代码显然没有问题，但它不起作用，因为 `getTime()` 返回毫秒，而不是秒。

通过在我们的变量后面加上 `_ms`，我们可以使一切更明确：

```
var start_ms = (new Date()).getTime(); // 页面顶部
...
var elapsed_ms = (new Date()).getTime() - start_ms; // 页面底部
document.writeln("加载时间为: " + elapsed_ms / 1000 + " 秒");
```

除了时间，编程中还会出现很多其他单位。下面是一个表格，列出了无单位的函数参数，以及包含单位的更好版本：

函数参数	重命名参数以编码单位
Start(int delay)	delay → delay_secs
CreateCache(int size)	大小 → 大小_MB
ThrottleDownload(float limit)	limit → max_kbps
Rotate(float angle)	angle → degrees_cw

编码其他重要属性

这种将额外信息附加到名称的技术并不局限于带单位的值。当变量存在危险或令人惊讶之处时，您应该随时这样做。

例如，许多安全漏洞源于没有意识到您的程序接收到的某些数据尚未处于安全状态。为此，您可能需要使用如下变量名 `untrustedUr`或 `unsafeMessageBody` 在调用清除不安全输入的函数之后，生成的变量可能是 `trustedUr`或 `safeMessageBody`

下表显示了应该在名称中编码额外信息的其他示例：

情况	变量名	更好的名称
一个密码是以“明文”形式存在的，应该在进一步处理之前进行加密	密码	<code>plaintext_password</code>
用户提供的评论在显示之前需要转义	评论	<code>unescaped_comment</code>
html的字节已转换为 UTF-8	HTML	<code>html_utf8</code>
传入的数据已“URL编码”	数据	<code>数据_urlenc</code>

你不应该在程序中为每个变量使用像 `unescaped_` 或 `_utf8` 这样的属性。如果有人弄错了变量是什么，它们在容易潜入错误的地方最为重要，特别是如果后果很严重，比如安全漏洞。本质上，如果理解这一点至关重要，请将其放入名称中。

这是匈牙利命名法吗？

[匈牙利命名法](#)是微软内部广泛使用的一种命名系统。它将每个变量的“类型”编码到名称的前缀中。以下是一些例子：

名称	含义
<code>pLast</code>	指向某个数据结构中最后一个元素的指针 (p)
<code>pszBuffer</code>	指向以零结尾 (z) 字符串 (s) 缓冲区的指针 (p)
<code>cch</code>	字符 (ch) 的计数 (c)
<code>mpcpx</code>	从指向颜色的指针 (pco) 到指向 x 轴长度的指针 (px) 的映射 (m)

这确实是“将属性附加到名称”的一个例子。但这是一个更正式和严格的系统专注于编码一组特定的属性。

我们在本节中提倡的是一个更广泛、更非正式的系统：识别变量的任何关键属性，并以清晰易懂的方式对其进行编码，如果它们是必需的。你可以称之为“英语符号”。

名称应该有多长？



在选择一个好名字时，存在一个隐含的约束，即名字不应该太长。

没有人喜欢使用这样的标识符：

```
newNavigationControllerWrappingViewControllerForDataSourceOfClass
```

名称越长，就越难记住，并且它在屏幕上占用的空间越多，可能导致额外的行换行。

另一方面，程序员可能会过度使用这个建议，只使用单字（或单字母）名称。那么你应该如何管理这种权衡？你如何决定命名一个变量 `d`, `days`, 或 `days_since_last_update`？

这个决定是一个判断性的决定，其最佳答案取决于该变量的确切使用方式。但这里有一些指导原则可以帮助你做出决定。

对于较小的作用域，较短的名称是可以的

当你去短途旅行时，你通常会比长途旅行携带更少的行李。类似地，具有较小“作用域”（有多少其他代码行可以“看到”这个名称）的标识符不需要携带那么多信息。也就是说，你可以使用较短的名称来摆脱困境，因为所有这些信息（变量的类型、它的初始值、它是如何被销毁的）都很容易看到：

```
if (debug) {  
    map<string,int> m;  
    LookUpNamesNumbers(&m);  
    Print(m);  
}
```

即使 `m` 没有包含任何信息，这也不是问题，因为读者已经拥有理解这段代码所需的所有信息。

然而，假设 `m` 是一个类成员或全局变量，并且你看到了这段代码片段：

```
LookUpNamesNumbers(&m);  
Print(m);
```

这段代码的可读性要差得多，因为不清楚 `m` 的类型或用途是什么。

因此，如果一个标识符具有很大的作用域，那么名称需要携带足够的信息来使其清晰。

输入长名称——不再是问题

有很多好的理由来避免长名称，但“它们更难输入”不再是其中之一。我们见过的每个编程文本编辑器都内置了“单词补全”功能。

令人惊讶的是，大多数程序员没有意识到这个功能。如果你还没有在你的编辑器上尝试过这个功能，请现在放下这本书并尝试一下：

1. 输入名称的前几个字符。
2. 触发单词补全命令（见下文）。
3. 如果完成的单词不正确，请继续触发命令，直到出现正确的名称。

它非常准确。它适用于任何类型的文件，任何语言。它适用于任何令牌，即使您正在输入注释。

编辑器	命令
Vi	Ctrl-p
Emacs	Meta/（按ESC，然后/）
Eclipse	Alt-/
IntelliJ IDEA	Alt-/
TextMate	ESC

首字母缩略词和缩写

程序员有时会使用首字母缩略词和缩写来保持名称简短——例如，将一个类命名为 `BEManager` 而不是 `BackEndManager`。这种缩减是否值得潜在的混淆？

根据我们的经验，特定于项目的缩写通常是一个坏主意。对于那些刚接触项目的人来说，它们显得神秘而令人生畏。随着时间的推移，它们甚至开始让作者感到神秘和令人生畏。

所以我们的经验法则是：新的团队成员会理解这个名字的含义吗？如果是这样，那么可能没问题。

例如，程序员通常使用 `eval` 代替 `evaluation`，`doc` 代替 `document`，`str` 代替 `string`。因此，一个新的团队成员看到 `FormatStr()` 可能会理解它的含义。但是，他或她可能不理解 `BEManager` 是什么。

扔掉不必要的词语

有时候，名字中的某些词可以被删除，而不会丢失任何信息。例如，与其使用 `ConvertToString()`，不如使用 `ToString()` 这样名字更短，而且不会丢失任何实际信息。类似地，与其使用 `DoServeLoop()`，不如使用 `ServeLoop()` 这样同样清晰。

使用名称格式来传达含义

你使用下划线、破折号和大小写的方式也可以在名称中包含更多信息。例如，下面是一些 C++ 代码，它遵循了 Google 开源项目使用的格式约定：

```
static const int kMaxOpenFiles = 100;

class LogReader {
public:
    void OpenFile(string local_file);

private:
    in_offset_
    DISALLOW_COPY_AND_ASSIGN(LogReader);
};
```

为不同的实体设置不同的格式就像一种语法高亮显示——它能帮助你更轻松地阅读代码。

这个例子中的大多数格式都很常见——类名使用 `CamelCase`，变量名使用 `lower_separated`，但其他一些约定可能会让你感到惊讶。

例如，常量值的形式是 `kConstantName` 而不是 `CONSTANT_NAME`。这种风格的好处是可以很容易地与 `#define` 宏区分开来，按照惯例，`#define` 宏是 `MACRO_NAME`。

类成员变量类似于普通变量，但必须以下划线结尾，例如 `offset_`。起初，这种约定可能看起来很奇怪，但能够立即区分

从其他变量中获取成员非常方便。例如，如果您正在浏览一个大型方法的代码，并且看到以下行：

```
stats.clear();
```

你通常可能会想，stats 属于这个类吗？这段代码是否正在改变类的内部状态？如果使用了 member_ 约定，你可以很快得出结论，不，stats 必须是一个局部变量。否则它会被命名为 stats_。

其他格式约定

根据您的项目或语言的上下文，可能还有其他格式约定可用于使名称包含更多信息。

例如，在 *JavaScript: The Good Parts* (道格拉斯·克洛克福德 (Douglas Crockford)，O'Reilly, 2008) 中，作者建议“构造函数”（旨在用 new 调用的函数）应该大写，而普通函数应该以小写字母开头：

```
var x = new DatePicker(); // DatePicker() 是一个“构造函数”  
var y = pageHeight(); // pageHeight() 是一个普通函数
```

这里是另一个 JavaScript 示例：当调用 jQuery 库函数（其名称为单个字符 \$）时，一个有用的约定是在 jQuery 结果前加上 \$：

```
var $all_images = $("img"); // $all_images 是一个 jQuery 对象  
var height = 250; // height 不是
```

在整个代码中，很明显 \$all_image 是一个 jQuery 结果对象。

这是一个最后的例子，这次是关于 HTML/CSS 的：当给一个 HTML 标签一个 id 或 class 属性时，下划线和破折号都是可以在值中使用的有效字符。一种可能的约定是使用下划线来分隔 ID 中的单词，使用破折号来分隔

类中的单词：

```
<div id="middle_column" class="main-content"> ...
```

是否决定使用像这样的约定取决于你和你的团队。但是无论你使用哪种系统，都要在你的项目中保持一致。

总结

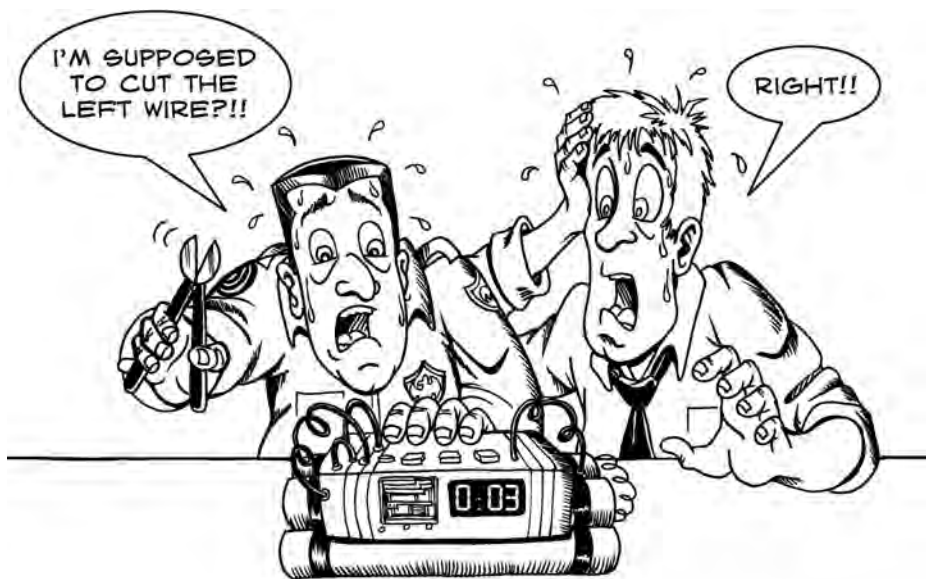
本章的唯一主题是：将信息打包到你的名字中。通过这个，我们的意思是读者可以仅从阅读名称中提取大量信息。

以下是我们涵盖的一些具体技巧：

- 使用特定的词——例如，代替 Get，像 Fetch 或 Download 这样的词可能更好，取决于上下文。
- 避免使用通用名称，如 tmp 和 retval，除非有特定的理由使用它们。

- 使用具体的名称，更详细地描述事物——名称 `ServerCanStart()` 与 `CanListenOnPort()` 相比，显得模糊。
- 将重要的细节附加到变量名上——例如，将 `_ms` 附加到值以毫秒为单位的变量，或者将 `raw_` 前置到需要转义的未处理变量。
- 对于较大的作用域，使用较长的名称——不要对跨越多个屏幕的变量使用隐晦的单字母或双字母名称；对于仅跨越几行的变量，较短的名称更好。
- 以有意义的方式使用大小写、下划线等——例如，您可以将 “_” 附加到类成员，以将其与局部变量区分开。

不会被误解的名称



在前一章中，我们介绍了如何将大量信息放入您的名称中。在本章中，我们将重点关注一个不同的主题：注意可能被误解的名称。

关键思想

通过问自己“其他人可能会从这个名称中解释出什么其他含义？”来积极审查您的名称。

在这里真正尝试发挥创造力，积极寻找“错误的解释”。这一步将帮助你发现那些含义模糊的名称，以便你可以更改它们。

在本章的示例中，我们将“大声思考”，讨论我们看到的每个名称的误解，然后选择更好的名称。

示例：Filter()

假设您正在编写代码来操作一组数据库结果：

```
results = Database.all_objects.filter("year <= 2011")
```

results 现在包含什么？

- 年份 <= 2011 的对象？
- 年份 不是 <= 2011 的对象？

问题在于 filter 是一个模棱两可的词。目前尚不清楚它指的是“挑选出来”还是“摆脱掉”。最好避免使用名称 filter，因为它很容易被误解。

如果您想要“挑选出来”，一个更好的名称是 select()。如果您想要“摆脱掉”，一个更好的名称是 exclude()。

示例：Clip(text, length)

假设您有一个函数可以剪切段落的内容：

```
# 截断文本的末尾，并附加 “...”
def Clip(text, length):
    ...
```

你可以想象 Clip() 的行为方式有两种：

- 它从末尾移除 length
- 它截断到最大 length

第二种方式（截断）最有可能，但你永远无法确定。与其让你的读者心存疑虑，不如将函数命名为 Truncate(text, length)。

然而，参数名 `length` 也应受到指责。如果它是 `max_length` 那会更清楚。

但我们还没有完成。名称 `max_length` 仍然留下多种解释：

- 字节数
- 字符数
- • 单词数

正如你在上一章中看到的，这是一个单位应该附加到名称上的例子。在这种情况下，我们指的是“字符数”，所以应该用 `max_chars` 代替 `max_length`。

对于（包含性）限制，首选 `min` 和 `max`

假设你的购物车应用程序需要阻止人们一次购买超过 10 件商品：

```
CART_TOO_BIG_LIMIT = 10
```

```
如果 shopping_cart.num_items() >= CART_TOO_BIG_LIMIT:  
    Error("购物车中商品过多。")
```

这段代码有一个经典的差一错误。我们可以很容易地通过将 `>=` 改为 `>` 来修复它：

```
如果 shopping_cart.num_items() > CART_TOO_BIG_LIMIT:
```

（或者通过将 `CART_TOO_BIG_LIMIT` 重新定义为 11）。但根本问题在于 `CART_TOO_BIG_LIMIT` 是一个含义模糊的名称——不清楚你指的是“最多”还是“最多且包括”。

建议

命名限制最清晰的方法是在被

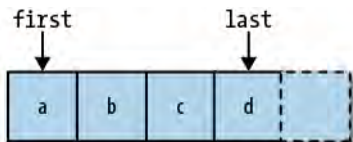
```
限制时使用 MAX_ 或 MIN_。
```

在这种情况下，名称应该是 `MAX_ITEMS_IN_CART` 新代码简单明了：

```
MAX_ITEMS_IN_CART = 10
```

```
如果 shopping_cart.num_items() > MAX_ITEMS_IN_CART:  
    Error("购物车中商品过多。")
```

对于包含性范围，首选 first 和 last



这里是另一个例子，你无法判断它是“直到”还是“直到并包括”：

```
print integer_range(start=2, stop=4)
# 这会打印 [2,3] 还是 [2,3,4]（或其他什么）？
```

虽然 `start` 是一个合理的参数名称，但 `stop` 在这里可以有多种解释。

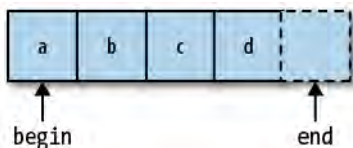
对于像这样的 *inclusive* 范围（范围应包括两个端点），一个好的选择是 `first/last`。例如：

```
set.PrintKeys(first="Bart", last="Maggie")
```

与 `stop` 不同，单词 `last` 显然是包含性的。

除了 `first/last`，名称 `min/max` 也适用于包含性范围，假设它们在上下文中“听起来正确”。

首选包含/排除范围的开始和结束



在实践中，使用包含/排除范围通常更方便。例如，如果你想打印 10 月 16 日发生的所有事件，写起来更容易：

```
PrintEventsInRange("10月16日凌晨12:00", "10月17日凌晨12:00")
```

比写作更容易：

```
PrintEventsInRange("10月16日凌晨12:00", "10月16日下午11:59:59.9999")
```

那么，这些参数的好名称是什么呢？嗯，对于命名包含/排除范围的典型编程惯例是开始/结束。

但是“结束”这个词有点含糊。例如，在句子“我在书的结尾”中，“结尾”是包含性的。不幸的是，英语没有一个简洁的词来表示“刚好超过最后一个值”。

因为开始/结束是如此地道的（至少，它在C++的标准库中以这种方式使用，以及大多数需要以这种方式“切片”数组的地方），所以它是最好的选择。

命名布尔值

在为一个布尔变量或一个返回布尔值的函数选择名称时，请确保清楚真和假真正意味着什么。

这是一个危险的例子：

```
bool read_password = true;
```

根据你如何理解它（没有双关语的意思），有两种非常不同的解释：

- 我们 *需要* 读取密码
- 密码已经 *被* 读取

在这种情况下，最好避免使用“读取”这个词，而是命名为 `need_password` 或者 `user_is_authenticated` 代替。

一般来说，添加像 `is`, `has`, `can`, 或者 `should` 这样的词可以使布尔值更清晰。

例如，一个名为 `SpaceLeft()` 的函数听起来像是会返回一个数字。如果它意味着返回一个布尔值，一个更好的名字应该是 `HasSpaceLeft()`。

最后，最好避免在名称中使用 *negated* 术语。例如，代替：

```
bool disable_ssl = false;
```

说以下内容会更容易阅读（也更简洁）：

```
bool use_ssl = true;
```

匹配用户的期望

有些名称具有误导性，因为用户对该名称的含义有先入为主的观念，即使你的意思是其他意思。在这些情况下，最好“屈服”并更改名称，使其不具有误导性。

示例：get*()

许多程序员习惯于这样一种约定，即以 `get` 开头的方法是“轻量级访问器”，它们只是返回一个内部成员。违背这一约定可能会误导这些用户。

这是一个Java中的例子，说明了 不应该做什么：

```
public class StatisticsCollector {
    public void addSample(double x) { ... }

    public double getMean() {
        // 遍历所有样本并返回总数 / 样本数
    }
    ...
}
```

在这种情况下，getMean()的实现是迭代过去的的数据并动态计算平均值。如果数据量很大，这一步可能会非常昂贵！但是一个毫无戒心的程序员可能会随意调用getMean() 假设这是一个廉价的调用。

相反，该方法应该重命名为类似computeMean()的名称，这听起来更像是一个昂贵的操作。（或者，应该重新实现它，使其确实成为一个轻量级的操作。）

示例：list::size()

这是一个来自C++标准库的例子。以下代码是一个非常难以找到的错误的原因，该错误导致我们的一台服务器速度慢到爬行：

```
void ShrinkList(list<Node>& list, int max_size) {
    while (list.size() > max_size) {
        FreeNode(list.back());
        list.pop_back();
    }
}
```

这个“bug”是作者不知道list.size()是一个 $O(n)$ 操作——它通过链接列表节点逐个计数，而不是仅仅返回一个预先计算好的计数，这使得 ShrinkList() 成为一个 $O(n^2)$ 操作。

从技术上讲，这段代码是“正确的”，而且实际上通过了我们所有的单元测试。但是当 ShrinkList() 在一个包含一百万个元素的列表上调用时，它花了一个多小时才完成！

也许你在想，“那是调用者的错——他或她应该更仔细地阅读文档。”这是真的，但在这种情况下，list.size()不是一个常数时间操作是令人惊讶的。C++ 中的所有其他容器都有一个常数时间 size() 方法。

如果 size()被命名为 countSize()或 countElements() 同样的错误就不太可能发生。

C++ 标准库的编写者可能想将该方法命名为 size()，以匹配

vector 和 map 等所有其他容器。但正因为如此，程序员很容易误认为

它是一个快速操作，就像其他容器一样。值得庆幸的是，最新的 C++ 标准

现在强制 size() 为 $O(1)$ 。

谁是巫师？

不久前，其中一位作者正在安装 OpenBSD 操作系统。在磁盘格式化步骤中，出现了一个复杂的菜单，要求输入磁盘参数。其中一个选项是进入“Wizard 模式”。他很高兴找到这个用户友好的选项并选择了它。令他沮丧的是，它将安装程序放入一个低级提示符中，等待手动磁盘格式化命令，并且没有明确的方法可以退出它。显然，“wizard”意味着 你是巫师！

示例：评估多个名称候选者

在决定一个好名字时，你可能会有多个候选名字需要考虑。
在确定最终选择之前，在脑海中辩论每个名字的优点是常见的。
以下示例说明了这个评论过程。

高流量网站经常使用“实验”来测试对网站的更改是否能改善业务。这是一个控制一些实验的配置文件的例子：

```
experiment_id: 100
description: "将字体大小增加到14pt"
traffic_fraction: 5%
...
```

每个实验由大约15个属性/值对定义。不幸的是，当定义另一个非常相似的实验时，你必须复制和粘贴大部分行：

```
experiment_id: 101
description: "将字体大小增加到 13pt"
[其他行与experiment_id 100相同]
```

假设我们想通过引入一种方法让一个实验重用另一个实验的属性来解决这个问题。（这是“原型继承”模式。）最终结果是，你将输入类似这样的内容：

```
experiment_id: 101
the_other_experiment_id_I_want_to_reuse: 100
[根据需要更改任何属性]
```

问题是：the_other_experiment_id_I_want_to_reuse 真正应该命名为什么？

这里有四个名字可以考虑：

1. 模板
2. 重用
3. 复制
4. 继承

这些名字对我们来说都有意义，因为是我们向配置语言添加了这个新功能。但是我们必须想象一下，这个名字对于那些偶然看到代码并且不了解此功能的人来说会是什么感觉。因此，让我们分析每个名称，考虑一下人们可能会误解它的方式。

1. 让我们想象一下使用名称模板：

```
experiment_id: 101
模板: 100
...
```

模板有几个问题。首先，不清楚它是在说“我是一个模板”还是“我正在使用另一个模板”。其次，“模板”通常是 *抽象的* 必须在它是 *具体的* 之前“填充”。有人可能会认为模板化的实验不是一个“真正的”实验。总的来说，模板在这种情况下太模糊了。

2. 那么重用 怎么样：

```
experiment_id: 101
重用: 100
...
```

重用是一个不错的词，但正如所写的，有人可能会认为它在说，“这个实验最多可以重用 100 次。”将名称更改为 `reuse_id` 会有所帮助。但是一个困惑的读者可能会认为 `reuse_id: 100` 意味着“我用于重用的 id 是 100”。

3. 让我们考虑一下 复制。

```
experiment_id: 101
复制: 100
...
```

复制是一个好词。但就其本身而言，复制: 100 似乎可能在说“复制这个实验 100 次”或“这是某物的第 100 个副本”。为了明确这个术语指的是另一个实验，我们可以将名称更改为 `copy_experiment`。到目前为止，这可能是最好的名称。

4. 但现在让我们考虑一下 继承：

```
experiment_id: 101
继承: 100
...
```

“继承”这个词对大多数程序员来说都很熟悉，并且人们理解在继承之后会进行进一步的修改。通过类继承，你可以获得另一个类的所有方法和成员，然后修改它们或添加更多。即使在现实生活中，当你从亲戚那里继承财产时，人们也理解你可能会出售它们或自己拥有其他东西。

但是，再次明确一下，我们是从另一个实验中继承。我们可以将名称改进为 `inherit_from` 甚至 `inherit_from_experiment_id`。

总的来说，`copy_experimen`和 `inherit_from_experiment_`是最好的名称，因为它们最清楚地描述了正在发生的事情，并且最不容易被误解。

总结

最好的名称是那些不会被误解的名称——阅读你代码的人会按照你的意图理解它，而不会有其他方式。不幸的是，很多英语单词在编程方面都是模棱两可的，比如 `filter`、`length`和 `limit`。

在确定名称之前，扮演一下魔鬼代言人，想象一下你的名称可能会如何被误解。最好的名称能够抵抗误解。

当定义一个值的上限或下限时，`max_`和 `min_`是很好的前缀。对于包含范围，`first`和 `last`很好。对于包含/排除范围，`begin`和 `end` 最好，因为它们是最惯用的。

在命名布尔值时，使用像 `is` 和 `has` 这样的词来明确它是一个布尔值。避免使用否定词（例如，`disable_ssl`）。

注意用户对某些词的期望。例如，用户可能期望 `get()` 或 `size()` 是轻量级方法。

第四章

美学



杂志的布局需要经过深思熟虑——段落的长度、栏的宽度、文章的顺序以及封面内容。一本好的杂志应该让人可以轻松地从一页跳到另一页，也可以轻松地从头到尾阅读。

好的源代码也应该同样“赏心悦目”。在本章中，我们将展示如何通过合理地使用空格、对齐和排序来使你的代码更易于阅读。

具体来说，我们使用以下三个原则：

- 使用一致的布局，让读者可以习惯这些模式。
- 使相似的代码看起来相似。
- 将相关的代码行分组到块中。

美学与设计

在本章中，我们只关注你可以对代码进行的简单的“美学”改进。这些类型的更改很容易进行，并且通常可以大大提高可读性。有时，对你的代码进行更大的重构（例如拆分出新的函数或类）可以提供更大的帮助。我们的观点是，良好的美学和良好的设计是独立的概念；理想情况下，你应该同时追求两者。

为什么美学很重要？



想象一下，如果你必须使用这个类：

```
class StatsKeeper {
public:
// 用于跟踪一系列双精度值的类
    void Add(double d); // 以及用于快速统计它们的方法
private: int count; /* 到目前为止有多少
*/ public:
    double Average();

private: double minimum;
list<double>
    past_items
    ;double maximum;
};
```

相比之下，如果你有这个更简洁的版本，你理解起来会花费更长的时间：

```
// 用于跟踪一系列双精度值的类
// 以及用于快速统计它们的方法。
class StatsKeeper {
public:
    void Add(double d);
    double Average();

private:
    list<double> past_items;
    int count; // 到目前为止的数量

    double minimum;
    double maximum;
};
```

很明显，使用美观的代码更容易。如果你仔细想想，你大部分的编程时间都花在看代码上！你越能快速浏览你的代码，每个人就越容易使用它。

重新排列换行符以保持一致和紧凑

假设你正在编写 Java 代码来评估你的程序在各种网络连接速度下的行为。你有一个 `TcpConnectionSimulator`，它在构造函数中接受四个参数：

1. 连接速度 (Kbps)
2. 平均延迟 (ms)
3. 延迟的“抖动” (ms)
4. 数据包丢失（百分比）

你的代码需要三个不同的 TcpConnectionSimulator 实例：

```
public class PerformanceTester {
    public static final TcpConnectionSimulator wifi = new TcpConnectionSimulator(
        500, /* Kbps */
        80, /* 毫秒延迟 */
        200, /* 抖动 */
        1 /* 数据包丢失 % */);

    public static final TcpConnectionSimulator t3_fiber =
        new TcpConnectionSimulator(
            45000, /* Kbps */
            10, /* 毫秒延迟 */
            0, /* 抖动 */
            0 /* 数据包丢失 % */);

    public static final TcpConnectionSimulator cell = new TcpConnectionSimulator(
        100, /* Kbps */
        400, /* 毫秒延迟 */
        250, /* 抖动 */
        5 /* 丢包率 % */);
}
```

这个示例代码需要很多额外的换行符才能适应 80 个字符的限制（这是您公司的编码标准）。不幸的是，这使得 t3_fiber 的定义看起来与其相邻的定义不同。这段代码的“轮廓”很奇怪，它无缘无故地引起了对 t3_fiber 的注意。这不符合“相似的代码应该看起来相似”的原则。

为了使代码看起来更一致，我们可以引入额外的换行符（并在我们进行操作时对齐注释）：

```
public class PerformanceTester {
    public static final TcpConnectionSimulator wifi =
        new TcpConnectionSimulator(
            500, /* Kbps */
            80, /* 毫秒延迟 */
            200, /* 抖动 */
            1 /* 数据包丢失 % */);

    public static final TcpConnectionSimulator t3_fiber =
        new TcpConnectionSimulator(
            45000, /* Kbps */
            10, /* 毫秒延迟 */
            0, /* 抖动 */
            0 /* 数据包丢失 % */);

    public static final TcpConnectionSimulator cell =
        new TcpConnectionSimulator(
            100, /* Kbps */
            400, /* 毫秒延迟 */
            250, /* 抖动 */
            5 /* 数据包丢失 % */);
}
```


这段代码有一个很好的、一致的模式，并且更容易浏览。但不幸的是，它使用了大量的垂直空间。它还将每个注释复制了三次。

这是一个更紧凑的编写类的方法：

```
public class PerformanceTester {
    // TcpConnectionSimulator(吞吐量, 延迟, 抖动, 丢包率)
    //           [Kbps] [ms] [ms] [百分比]

    public static final TcpConnectionSimulator wifi =
        新的 TcpConnectionSimulator(500, 80, 200, 1);

    public static final TcpConnectionSimulator t3_fiber =
        新的 TcpConnectionSimulator(45000, 10, 0, 0);

    public static final TcpConnectionSimulator cell =
        新的 TcpConnectionSimulator(100, 400, 250, 5);
}
```

我们已将注释移至顶部，然后将所有参数放在一行上。现在，即使注释不在每个数字旁边，“数据”也排列在一个更紧凑的表格中。

使用方法清理不规则性

假设你有一个人事数据库，它提供了以下功能：

```
// 将类似 "Doug Adams" 的 partial_name 转换为 "Mr. Douglas Adams"。
// 如果不可能，'error' 会被填入解释。
字符串 ExpandFullName(数据库连接 dc, 字符串 partial_name, 字符串* error);
```

并且这个函数通过一系列的例子进行了测试：

```
DatabaseConnection database_connection;
字符串错误;
assert(ExpandFullName(database_connection, "Doug Adams", &error)
    == "Mr. Douglas Adams");
assert(error == "");
assert(ExpandFullName(database_connection, " Jake Brown ", &error)
    == "Mr. Jacob Brown III");
assert(error == "");
assert(ExpandFullName(database_connection, "No Such Guy", &error) == "");
assert(error == "no match found");
assert(ExpandFullName(database_connection, "John", &error) == "");
assert(error == "more than one result");
```

这段代码在美学上并不令人满意。有些行太长，以至于会换行到下一行。这段代码的轮廓很丑陋，而且没有一致的模式。

但这这是一个重新排列换行符也无济于事的情况。更大的问题

是存在大量重复的字符串，例如“assert(ExpandFullName(database_connection...,”和“error”，这些字符串妨碍了代码的编写。为了真正改进这段代码，我们需要一个辅助方法，以便代码看起来像这样：

```

CheckFullName("Doug Adams", "Mr. Douglas Adams", "");
CheckFullName(" Jake Brown ", "Mr. Jake Brown III", "");
CheckFullName("No Such Guy", "", "no match found");
CheckFullName("John", "", "more than one result");

```

现在更清楚的是，有四个测试正在进行，每个测试都有不同的参数。即使所有的“脏活”都在 `CheckFullName` 内部，该函数也不算太糟糕：

```

void CheckFullName(string partial_name,
                  string expected_full_name,
                  string expected_error) {
    // database_connection 现在是一个类成员
    string error;
    string full_name = ExpandFullName(database_connection, partial_name, &error);
    assert(error == expected_error);
    assert(full_name == expected_full_name);
}

```

即使我们的目标只是让代码在美学上更令人愉悦，但这种改变还有许多其他附带的好处：

- 它消除了之前的大量重复代码，使代码更加紧凑。
- 每个测试用例的重要部分（名称和错误字符串）现在都单独存在，一目了然。
在此之前，这些字符串与诸如 `database_connection` 和 `error` 之类的标记交错在一起，这使得很难一眼“吸收”代码。
- 现在添加新测试应该容易得多。

这个故事的寓意是，让代码“看起来漂亮”往往不仅仅带来表面上的改进，它可能有助于你更好地组织代码。

在有帮助时使用列对齐

笔直的边缘和列使读者更容易浏览文本。

有时你可以引入“列对齐”来使代码更易于阅读。例如，在上一节中，你可以将参数隔开并对齐到

```

CheckFullName():

    CheckFullName("Doug Adams", "Mr. Douglas Adams", "");
    CheckFullName(" Jake Brown ", "Mr. Jake Brown III", "");
    CheckFullName("No Such Guy", "", "no match found");
    CheckFullName("John", "", "more than one result");

```

在这段代码中，更容易区分 `CheckFullName` 的第二个和第三个参数。

这是一个简单的例子，包含一大组变量定义：

```

# 将POST参数提取到局部变量
details = request.POST.get('details')
location = request.POST.get('location')
phone   = request.POST.get('phone')

```

```
email = request.POST.get('email')
url = request.POST.get('url')
```

正如你可能已经注意到的，第三个定义中有一个错字（equest而不是 request）。当所有东西都排列得如此整齐时，这样的错误会更加明显。

在 wget 代码库中，可用的命令行选项（超过 100 个）按如下方式列出：

```
commands[] = {
    ...
    { "timeout",      NULL,      cmd_spec_timeout },
    { "timestamping", &opt.timestamping, cmd_boolean },
    { "tries",        &opt.ntry,   cmd_number_inf },
    { "useproxy",     &opt.use_proxy, cmd_boolean },
    { "useragent",    NULL,      cmd_spec_useragent },
    ...
};
```

这种方法使得列表非常容易浏览，并且可以从一列跳到下一列。

你应该使用列对齐吗？

列边缘提供了“视觉扶手”，使其更容易扫描。这是一个很好的“使相似代码看起来相似”的例子。

但有些程序员不喜欢它。一个原因是设置和维护对齐需要更多的工作。另一个原因是它在进行更改时会产生更大的“差异”——单行更改可能会导致其他五行更改（主要只是空格）。

我们的建议是尝试一下。根据我们的经验，它并不像程序员担心的那样需要大量工作。如果确实如此，你可以简单地停止。

选择一个有意义的顺序，并始终如一地使用它

在许多情况下，代码的顺序不会影响正确性。例如，这些五个变量定义可以用任何顺序编写：

```
details = request.POST.get('details')
location = request.POST.get('location')
phone   = request.POST.get('phone')
email   = request.POST.get('email')
url     = request.POST.get('url')
```

在这种情况下，将它们按照某种有意义的顺序排列，而不是随机排列，会很有帮助。以下是一些建议：

- 将变量的顺序与相应的 <input> 字段在

HTML 表单上的顺序相匹配。

- 按照从“最重要”到“最不重要”的顺序排列。
- 按照字母顺序排列。

无论顺序如何，您都应该在整个代码中使用相同的顺序。如果在以后更改顺序，将会令人困惑：

```
if details: rec.details = details
if phone: rec.phone = phone # 嘿, 'location'去哪儿了?
if email: rec.email = email
if url: rec.url = url
if location: rec.location = location # 为什么'location'现在在这里?
```

将声明组织成块

大脑自然地以群体和层级的形式思考，因此你可以通过以这种方式组织代码来帮助读者快速理解你的代码。

例如，这是一个前端服务器的 C++ 类，包含其所有方法声明：

```
class FrontendServer {
public:
    FrontendServer();
    void ViewProfile(HttpRequest* request);
    void OpenDatabase(string location, string user);
    void SaveProfile(HttpRequest* request);
    string ExtractQueryParam(HttpRequest* request, string param);
    void ReplyOK(HttpRequest* request, string html);
    void FindFriends(HttpRequest* request);
    void ReplyNotFound(HttpRequest* request, string error);
    void CloseDatabase(string location);
    ~FrontendServer();
};
```

这段代码并不糟糕，但布局肯定无助于读者理解所有这些方法。与其在一个巨大的块中列出所有方法，不如将它们按逻辑组织成组，像这样：

```
class FrontendServer {
public:
    FrontendServer();
    ~FrontendServer();

    // 处理程序
    void ViewProfile(HttpRequest* request);
    void SaveProfile(HttpRequest* request);
    void FindFriends(HttpRequest* request);

    // 请求/回复实用程序
    string ExtractQueryParam(HttpRequest* request, string param);
    void ReplyOK(HttpRequest* request, string html);
    void ReplyNotFound(HttpRequest* request, string error);
};
```

```

// 数据库助手
void OpenDatabase(string location, string user);
void CloseDatabase(string location);
};

```

这个版本更容易理解。它也更容易阅读，即使有更多的代码行。原因是您可以快速找出四个高级部分，然后在必要时阅读每个部分的详细信息。

将代码分解为“段落”

书面文本被分成段落有以下几个原因：

- 这是一种将相似的想法组合在一起，并将它们与其他想法区分开来的方法。
- 它提供了一个视觉“垫脚石”——没有它，很容易在页面上迷失方向。
- 它方便了从一个段落到另一个段落的导航。

代码应该出于同样的原因被分成“段落”。例如，没有人喜欢阅读像这样的巨大代码块：

```

# 导入用户的电子邮件联系人，并将他们与我们系统中的用户进行匹配。
# 然后显示一个他/她还不是朋友的那些用户的列表。
def suggest_new_friends(user, email_password):
    friends = user.friends()
    friend_emails = set(f.email for f in friends)
    contacts = import_contacts(user.email, email_password)
    contact_emails = set(c.email for c in contacts)
    non_friend_emails = contact_emails - friend_emails
    suggested_friends = User.objects.select(email__in=non_friend_emails)
    display['user'] = user
    display['friends'] = friends
    display['suggested_friends'] = suggested_friends
    return render("suggested_friends.html", display)

```

可能不明显，但此函数经历了许多不同的步骤。因此，将这些代码行分成段落特别有用：

```

def suggest_new_friends(user, email_password):
    # 获取用户的朋友的电子邮件地址。
    friends = user.friends()
    friend_emails = set(f.email for f in friends)

    # 导入此用户电子邮件帐户中的所有电子邮件地址。
    contacts = import_contacts(user.email, email_password)
    contact_emails = set(c.email for c in contacts)

    # 查找他们还不是朋友的匹配用户。
    non_friend_emails = contact_emails - friend_emails
    suggested_friends = User.objects.select(email__in=non_friend_emails)

```

```
# 在页面上显示这些列表.  
    display['user'] = user  
display['friends'] = friends  
    display['suggested_friends'] = suggested_friends  
  
    return render("suggested_friends.html", display)
```

请注意，我们还在每个段落中添加了摘要注释，这也帮助读者浏览代码。（参见第5章，了解评论的内容。）

与书面文本一样，可能有多种方法可以分解代码，程序员可能更喜欢更长或更短的段落。

个人风格与一致性

有些审美选择可以归结为个人风格。例如，类定义的左大括号应该放在哪里：

```
class Logger {  
    ...  
};
```

或者

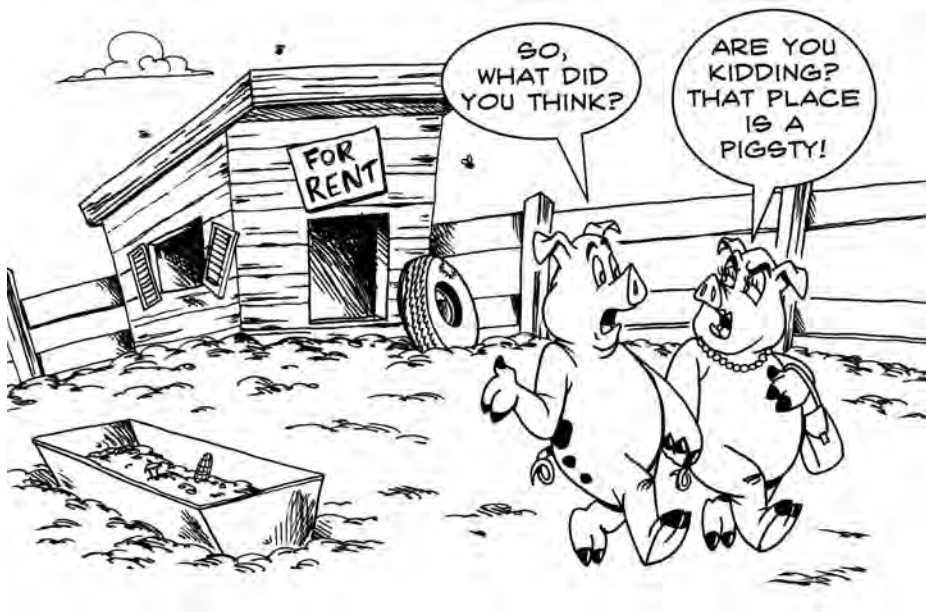
```
class Logger  
{  
    ...  
};
```

如果选择其中一种风格而不是另一种风格，它不会实质性地影响代码库的可读性。但是，如果这两种风格在代码中混合使用，它确实会影响可读性。

我们参与过许多项目，在这些项目中，我们觉得团队使用的是“错误”的风格，但我们遵循了项目约定，因为我们知道一致性远比其他更重要。

关键思想

一致的风格比“正确”的风格更重要。



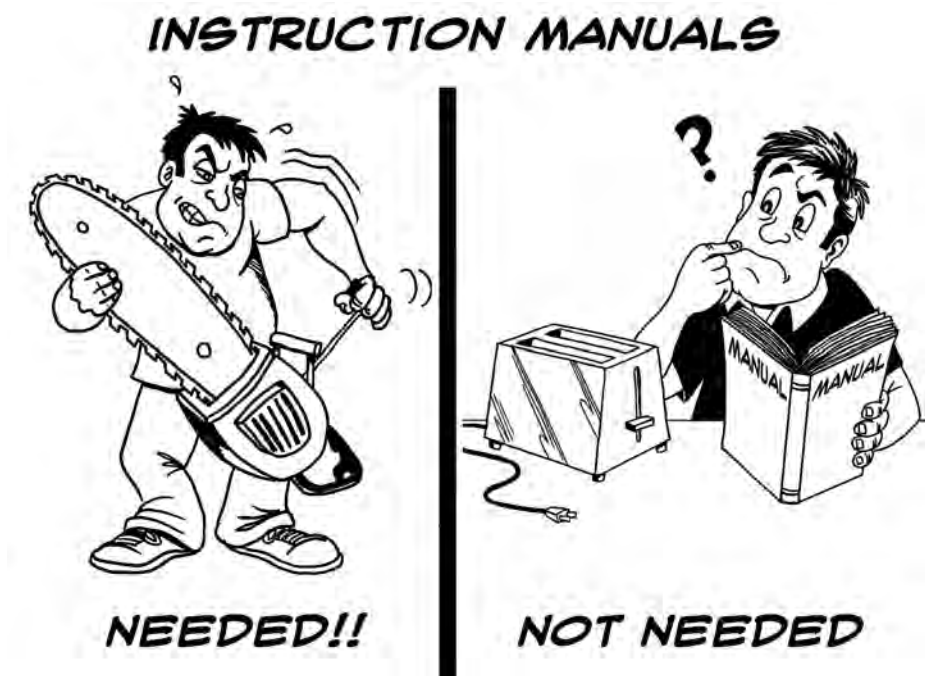
总结

每个人都喜欢阅读在美学上令人愉悦的代码。通过以一致的、有意义的方式“格式化”您的代码，您可以使其更易于阅读和更快地阅读。

以下是我们讨论的具体技术：

- 如果多段代码执行类似的操作，请尝试赋予它们相同的轮廓。
- 将代码的部分对齐成“列”可以使代码易于浏览。
- 如果代码在一个地方提到了 A、B 和 C，不要在另一个地方说 B、C 和 A。选择一个有意义的顺序并坚持下去。
- 使用空行将大的代码块分成逻辑上的“段落”。

知道该注释什么



本章的目标是帮助你意识到你应该注释什么。你可能认为注释的目的是“解释代码的作用”，但这只是一个很小的部分。

关键思想

注释的目的是帮助读者了解的和作者一样多。

当你编写代码时，你的脑海中有很多有价值的信息。当其他人阅读你的代码时，这些信息就丢失了——他们所拥有的只是摆在他们面前的代码。

在这一章中，我们将向您展示许多何时将信息记录在您脑海中的例子。我们省略了关于注释的更平凡的观点；相反，我们专注于注释中更有趣和“服务不足”的方面。

我们已将本章组织成以下几个方面：

- 知道什么不要注释
- 在编写代码时记录您的想法
- 设身处地为读者着想，想象他们需要知道什么

不该评论什么



阅读注释会占用阅读实际代码的时间，并且每个注释都会占用屏幕上的空间。也就是说，它最好是值得的。那么，您在哪里划定无价值的注释和好的注释之间的界限？

此代码中的所有注释都是无价值的：

```
// Account 的类定义
class Account {
public:
    // 构造函数
    账户();

    // 将利润成员设置为新值
    void SetProfit(double profit);

    // 返回此帐户的利润
    double GetProfit();
};
```

这些评论毫无价值，因为它们没有提供任何新的信息，也没有帮助读者更好地理解代码。

关键思想

不要评论可以从代码本身快速推导出的事实。

不过，“快速地”这个词是一个重要的区别。考虑一下这段 Python 代码的注释：

```
# 删除第二个 '*' 之后的所有内容
name = '*'.join(line.split('*')[:2])
```

从技术上讲，此注释也没有提供任何“新信息”。如果你查看代码本身，你最终会弄清楚它在做什么。但对于大多数程序员来说，阅读带注释的代码比理解没有注释的代码要快得多。

不要为了注释而注释



有些教授要求他们的学生在家庭作业代码中为每个函数添加注释。因此，一些程序员对让一个函数没有注释感到内疚，最终以句子形式重写函数的名称和参数：

```
// 使用给定的深度，在给定的子树中查找具有给定名称的节点。
Node* FindNodeInSubtree(Node* subtree, string name, int depth);
```

这一个属于“毫无价值的注释”类别——函数的声明和注释几乎相同。这个注释应该被删除或改进。

如果你想在这里添加注释，最好详细说明更重要的细节：

```
// 查找具有给定“名称”的节点，如果未找到则返回 NULL。  
// 如果 depth <= 0，则只检查 'subtree'。  
// 如果 depth == N，则只检查“子树”和 N 级以下的级别。  
Node* FindNodeInSubtree(Node* subtree, string name, int depth);
```

不要注释糟糕的名称——而是修复名称

注释不应该弥补一个糟糕的名称。例如，这是一个无辜的-
看起来像一个名为CleanReply()的函数的注释：

```
// 强制执行请求中声明的回复限制，  
// 例如返回的项目数或总字节大小等。  
void CleanReply(Request request, Reply reply);
```

大多数注释只是解释“clean”的含义。相反，短语“enforce limits”应该移动到函数名中：

```
// 确保“reply”满足来自“request”的计数/字节/等限制  
void EnforceLimitsFromRequest(Request request, Reply reply);
```

这个函数名更“自文档化”。一个好的名称胜过一个好的注释，因为它将在函数使用的任何地方被看到。

这是另一个命名不佳的函数的注释示例：

```
// 释放此键的句柄。这不会修改实际的注册表。  
void DeleteRegistry(RegistryKey* key);
```

名称DeleteRegistry(听起来像一个危险的函数（它 删除 注册表？！）。注释“这不会修改实际注册表”试图消除这种困惑。

相反，我们可以使用更具有自解释性的名称，例如：

```
void ReleaseRegistryHandle(RegistryKey* key);
```

一般来说，你不需要“拐杖注释”——试图弥补代码不可读性的注释。程序员通常将此规则表述为 好的代码 > 坏的代码 + 好的注释。

记录你的想法

现在你知道什么 不应该注释了，让我们讨论一下什么 应该被注释（但通常不是）。

许多好的注释可以简单地通过“记录你的想法”来产生——也就是说，你在编写代码时产生的重要想法。

包括“导演评论”

电影通常有一个“导演评论”音轨，电影制作人在其中提供他们的见解和讲述故事，以帮助你理解电影是如何制作的。类似地，你应该包含注释来记录关于代码的有价值的见解。

这是一个例子：

```
// 令人惊讶的是，对于这些数据，二叉树比哈希表快 40%。  
// 计算哈希的成本高于左右比较。
```

这个注释教会了读者一些东西，并阻止任何潜在的优化器浪费他们的时间。

这是另一个例子：

```
// 这个启发式方法可能会遗漏一些单词。没关系；100% 解决这个问题很难。
```

如果没有这个注释，读者可能会认为存在一个错误，并可能浪费时间尝试提出使它失败的测试用例，或者离开并尝试修复该错误。

注释还可以解释为什么代码状态不佳：

```
// 这个类变得混乱了。也许我们应该创建一个 'ResourceNode' 子类来  
// 帮助组织事情。
```

这个注释承认代码很混乱，但也鼓励下一个人修复它（具体说明如何开始）。如果没有注释，许多读者会被混乱的代码吓倒，不敢碰它。

注释代码中的缺陷

代码在不断发展，并且必然会有缺陷。不要不好意思记录这些缺陷。例如，记录何时应该进行改进：

```
// TODO: 使用更快的算法
```

或者当代码不完整时：

```
// TODO(dustin): 处理除 JPEG 之外的其他图像格式
```

有许多标记在程序员中变得流行：

标记	典型含义
TODO:	我还没来得及做的事情
FIXME:	此处已知损坏的代码
HACK:	无可否认的，解决问题的不优雅方案
XXX:	危险！这里有重大问题

您的团队可能对何时/是否使用这些标记有特定的约定。例如，TODO: 可能保留给严重的问题。如果是这样，那么对于较小的缺陷，您可以使用类似 todo: (小写) 或 maybe-later: 代替。

重要的是，您应该始终随意评论您对代码将来应该如何更改的想法。像这样的评论让读者对代码的质量和状态有宝贵的见解，甚至可能给他们一些关于如何改进它的方向。

评论你的常量

在定义常量时，通常会有一个关于该常量做什么或为什么它具有该特定值的“故事”。例如，您可能会在代码中看到这个常量：

```
NUM_THREADS = 8
```

这一行可能看起来不需要注释，但选择它的程序员可能更了解它：

```
NUM_THREADS = 8 # 只要它 >= 2 * num_processors, 就足够了。
```

现在，阅读代码的人可以获得有关如何调整该值的指导（例如，将其设置为 1 可能太低，而将其设置为 50 则矫枉过正）。

或者有时常量的确切值根本不重要。对此的评论仍然有用：

```
// 施加一个合理的限制 - 无论如何，没有人能读那么多。
```

```
const int MAX_RSS_SUBSCRIPTIONS = 1000;
```

有时这是一个经过高度调整的值，可能不应该进行太多调整：

```
image_quality = 0.72; // 用户认为 0.72 提供了最佳的尺寸/质量权衡
```

在所有这些示例中，您可能没有想到要添加注释，但它们非常有用。

有些常量不需要注释，因为它们的名称已经足够清晰（例如，SECONDS_PER_DAY，但在我们的经验中，大多数常量都可以通过添加注释来改进。这只是在您决定该常量的值时，记下您在想什么问题。

设身处地为读者着想

本书中我们使用的一种通用技术是想象一下你的代码对于一个局外人来说是什么样的——一个不像你那样熟悉你的项目的人。这种技术对于帮助你认识到什么需要注释特别有用。

预测可能出现的问题



“ARE THERE ANY QUESTIONS?...
...THAT AREN'T COVERED BY THAT SIGN.”

当其他人阅读你的代码时，有些部分可能会让他们想，嗯？这是怎么回事？你的工作就是注释这些部分。

例如，看看Clear()的定义：

```
struct Recorder {  
    vector<float> data;  
    ...  
    void Clear() {  
        vector<float>().swap(data); // 嗯？为什么不直接用 data.clear()?  
    }  
};
```

当大多数 C++ 程序员看到这段代码时，他们会想，为什么他不直接使用 `data.clear()` 而是用一个空向量进行交换呢？实际上，这是迫使 vector 真正将内存释放给内存分配器的唯一方法。这是一个不太为人所知的 C++ 细节。

最重要的是，应该对其进行注释：

```
// 强制向量放弃其内存（查找“STL交换技巧”）  
vector<float>().swap(data);
```


广告可能存在的陷阱



在记录一个函数或类时，一个好的问题是，这段代码有什么令人惊讶的地方？它可能被如何误用？基本上，你想要“提前思考”并预测人们在使用你的代码时可能遇到的问题。

例如，假设你编写了一个向给定用户发送电子邮件的函数：

```
void SendEmail(string to, string subject, string body);
```

这个函数的实现涉及到连接到一个外部邮件服务，这可能需要长达一秒钟，或者可能更长。一个正在编写Web应用程序的人可能没有意识到这一点，并在处理HTTP请求时错误地调用了这个函数。（如果电子邮件服务中断，这样做会导致他们的Web应用程序“挂起”。）

为了防止这种可能发生的意外，你应该对这个“实现细节”进行注释：

```
// 调用外部服务来发送电子邮件。（1分钟后超时。）  
void SendEmail(string to, string subject, string body);
```

这里有另一个例子：假设你有一个 `FixBrokenHtml()` 函数，它试图通过插入缺失的结束标签等来重写损坏的 HTML：

```
def FixBrokenHtml(html): ...
```

这个函数运行得很好，但有一个需要注意的地方是，当存在深度嵌套且不匹配的标签时，它的运行时间会急剧增加。对于糟糕的 HTML 输入，这个函数可能需要 分钟才能执行。

与其让用户稍后自己发现这一点，不如提前声明：

```
// 运行时复杂度为  $O(\text{number\_tags} * \text{average\_tag\_depth})$ ，所以要注意嵌套不良的输入。  
def FixBrokenHtml(html): ...
```

“大局观”注释



对于新团队成员来说，最难理解的事情之一就是“大局观”——类如何交互，数据如何流经整个系统，以及入口点在哪里。

设计系统的人经常忘记对此进行注释，因为他太密切地参与其中了。

考虑一下这个思想实验：有一个新人加入了你的团队，她坐在你旁边，你需要让她熟悉代码库。

当你带她浏览代码库时，你可能会指出某些文件或类，并说这样的话：

- “这是我们的业务逻辑和数据库之间的粘合代码。任何应用程序代码都不应直接使用它。”
- “这个类看起来很复杂，但它实际上只是一个智能缓存。它对系统的其他部分一无所知。”

经过一分钟的随意交谈，你的新团队成员会比她自己阅读源代码了解更多。

这正是应该作为高级

注释包含的信息类型。

这是一个文件级别注释的简单示例：

```
// 此文件包含辅助函数，这些函数为我们的
// 文件系统提供更方便的接口。它处理文件权限和其他细节。
```

不要因为你必须编写大量正式文档的想法而感到不知所措。几句精心挑选的句子胜过没有。

总结评论

即使在函数内部，对“大局”进行注释也是一个好主意。下面是一个注释的例子，它简洁地总结了下面的底层代码：

```
# 查找所有顾客为自己购买的商品。
for customer_id in all_customers:
    for sale in all_sales[customer_id].sales:
        if sale.recipient == customer_id:
            ...
```

如果没有这个注释，阅读每一行代码都会有点神秘。（“我看到我们正在迭代all_customers，但为了什么？”）

在较长的函数中，如果其中有几个大的“块”，那么这些摘要注释尤其有用：

```
def GenerateUserReport():
    # 获取此用户的锁
    ...

    # 从数据库读取用户信息
    ...

    # 将信息写入文件
    ...

    # 释放该用户的锁
```

这些注释也可以作为函数功能的要点总结，以便读者在深入细节之前了解函数的功能。（如果这些块很容易分离，您可以直接将它们作为自己的函数。正如我们之前提到的，好的代码胜过带有好的注释的坏代码。）

你应该评论什么、为什么或怎么做？

你可能听过这样的建议：“评论为什么，而不是什么（或怎么做）。”虽然很吸引人，但我们觉得这些说法过于简单化，对不同的人来说意味着不同的东西。

我们的建议是做任何有助于读者更容易理解代码的事情。这可能包括评论什么、怎么做或为什么（或全部三个）。

最后的想法——克服写作障碍

很多程序员不喜欢写注释，因为写好注释感觉像是一项繁重的工作。当作者遇到这种“写作障碍”时，最好的解决方案就是开始写作。

所以下次当你犹豫是否要写注释时，就直接写下你在思考的内容，无论它多么不成熟。

例如，假设你正在编写一个函数，并且心里想，糟糕，如果这个列表中出现重复项，事情就会变得棘手。把它写下来：

```
// 糟糕，如果这个列表中出现重复项，事情就会变得棘手。
```

看，有那么难吗？这实际上并不是一个糟糕的评论——肯定比什么都没有好。

不过，这种语言有点含糊。为了解决这个问题，我们可以遍历每个短语并将其替换为更具体的内容：

- “糟糕”的真正意思是“小心：这是需要注意的事情。”
- “这些东西”指的是“处理此输入的代码”。
- “会变得棘手”的意思是“很难实现”。

新的评论可能是：

```
// 小心：此代码不处理列表中的重复项（因为这很难做到）
```

请注意，我们将编写评论的任务分解为以下更简单的步骤：

1. 写下你脑海中浮现的任何评论。
2. 阅读评论，看看有什么（如果有的话）需要改进。
3. 做出改进。

随着你更频繁地进行注释，你会发现步骤 1 中注释的质量会越来越好，最终可能根本不需要修复。通过尽早和经常地进行注释，你可以避免在最后需要编写大量注释的不愉快情况。

总结

注释的目的是帮助读者了解作者在编写代码时所知道的内容。本章的全部内容都是关于意识到你所拥有的关于代码的所有不太明显的有价值的信息，并将这些信息记录下来。

什么不要注释：

- 可以从代码本身快速推导出的事实。
- 弥补糟糕代码的“拐杖注释”（例如糟糕的函数名称）——应该修复代码。

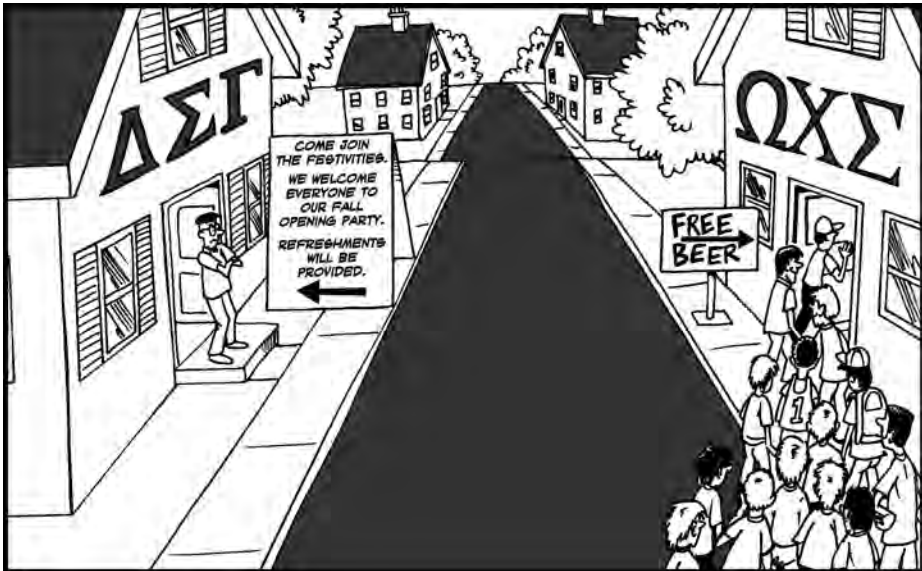
你应该记录的想法包括：

- 关于为什么代码是一种方式而不是另一种方式的见解（“导演评论”）。
- 代码中的缺陷，使用诸如 TODO: 或 XXX: 之类的标记。
- 常量如何获得其值的“故事”。

设身处地为读者着想：

- 预测代码的哪些部分会让读者说“Huh? ”，并对这些部分进行注释。
- 记录任何令人惊讶的行为，这是普通读者不会预料到的。
- 在文件/类级别使用“大局”注释来解释所有部分如何组合在一起。
- 用注释总结代码块，以便读者不会迷失在细节中。

使注释精确和简洁



前一章是关于意识到你应该注释什么。本章是关于如何编写精确而简洁的注释。

如果你要写评论，最好是*精确的*——尽可能具体和详细。

另一方面，评论会占用屏幕上的额外空间，并且需要额外的阅读时间。

因此，评论也应该是*紧凑的*。

关键思想

评论应该具有高的信息空间比。

本章的其余部分展示了如何做到这一点的例子。

保持注释简洁

这是一个C++类型定义的注释示例：

```
// int 是 CategoryType。  
// 内部对中的第一个浮点数是“分数”，  
// 第二个是“权重”。  
typedef hash_map<int, pair<float, float> > ScoreMap;
```

但是，当你可以用一行来解释它时，为什么要用三行呢？

```
// CategoryType -> (分数, 权重)  
typedef hash_map<int, pair<float, float> > ScoreMap;
```

有些评论需要三行空间，但这不是其中之一。

避免使用含义不清的代词

正如经典的小品《谁是第一？》所展示的那样，代词会使事情变得非常混乱。

读者需要额外的工作来“解决”代词。在某些情况下，不清楚“它”或“这”指的是什么。这是一个例子：

```
// 将数据插入缓存，但首先检查它是否太大。
```

在这个注释中，“它”可能指的是数据或缓存。您可能可以通过阅读代码的其余部分来弄清楚。

但是，如果您必须这样做，那么注释的意义何在？

最安全的方法是“填写”代词，如果有可能造成混淆。在前面的例子中，假设“它”是“数据”：

```
// 将数据插入缓存，但首先检查数据是否太大。
```

这是最简单的改变。您也可以重新组织句子，使“它”完全清楚：

```
// 如果数据足够小，则将其插入缓存。
```


润色粗糙的句子

在许多情况下，使注释更精确与使其更紧凑是齐头并进的。

这是一个来自网络爬虫的例子：

```
# 根据我们是否已经抓取过这个URL，赋予它不同的优先级。
```

这句话可能看起来还可以，但请将其与此版本进行比较：

```
# 优先抓取我们从未抓取过的URL。
```

这句话更简单、更短、更直接。它还解释了 *更高的* 优先级是给予未抓取的URL——之前的注释不包含该信息。

精确描述函数行为

想象一下，你刚刚编写了一个函数来计算文件中的行数：

```
// 返回此文件中的行数。
int CountLines(string filename) { ... }
```

这个注释不是很精确——有很多方法来定义“行”。以下是一些需要考虑的角落案例：

- ""（一个空文件）—0或1行？
- "hello"—0或1行？
- “你好”—1行还是2行？
- “你好 世界”—1或2行？
- “你好\r残酷的世界\r”—2、3或4行？

最简单的实现是计算换行符 (`\n`) 的数量。（这是 Unix 命令 `wc` 的工作方式。）以下是一个更好的注释来匹配此实现：

```
// 计算文件中包含多少个换行字节 ('\n')。
int CountLines(string filename) { ... }
```

这个注释并不比第一个版本长多少，但包含的信息更多。它告诉读者，如果没有换行符，该函数可能会返回 0。它还告诉读者，回车符 (`\r`) 会被忽略。

使用说明边界情况的输入/输出示例

说到注释，一个精心选择的输入/输出示例胜过千言万语。

例如，这是一个常见的函数，用于删除字符串的某些部分：

```
// 从输入 'src' 中删除 'chars' 的后缀/前缀。  
String Strip(String src, String chars) { ... }
```

这个评论不是很精确，因为它无法回答以下问题：

- chars是要删除的整个子字符串，还是实际上只是一个无序的字母集合？
- 如果在 src 的末尾有多个 字符 会怎么样？

相反，一个精心选择的例子可以回答这些问题：

```
// ...  
// 示例：Strip("abba/a/ba", "ab") 返回 "/a/"  
String Strip(String src, String chars) { ... }
```

这个例子“炫耀”了 Strip() 的全部功能。请注意，如果一个更简单的例子不能回答这些问题，那么它就没有什么用处：

```
// 示例：Strip("ab", "a") 返回 "b"
```

这里是另一个可以使用插图的函数的例子：

```
// 重新排列 'v'，使小于 pivot 的元素位于大于等于 pivot 的元素之前；  
// 然后返回最大的 'i'，使得 v[i] < pivot（如果没有小于 pivot 的元素，则返回 -1）  
int Partition(vector<int>* v, int pivot);
```

这个评论实际上非常精确，但有点难以想象。这是一个你可以包含的例子，以进一步说明：

```
// ...  
// 示例：Partition([8 5 9 8 2], 8) 可能会导致 [5 2 | 8 9 8] 并返回 1  
int Partition(vector<int>* v, int pivot);
```

关于我们选择的特定示例输入/输出，有几点需要提及：

- pivot 等于向量中的元素，以说明该边缘情况。
- 我们在向量中放入了重复项（8），以说明这是一个可接受的输入。
- 结果向量未排序——如果已排序，读者可能会产生错误的想法。
- 因为返回值是 1，所以我们确保 1 也不是向量中的值——那会令人困惑。

阐明你的代码意图

正如我们在前一章中提到的，注释通常是告诉读者你在编写代码时的想法。不幸的是，许多注释最终只是用字面术语描述代码的作用，而没有添加太多新信息。

这是一个这样的注释的例子：

```
void DisplayProducts(list<Product> products) {
    products.sort(CompareProductByPrice);

    // 以相反的顺序迭代列表
    for (list<Product>::reverse_iterator it = products.rbegin(); it != products.rend();
        ++it)
        DisplayPrice(it->price);

    ...
}
```

所有这些注释只是描述它下面的行。相反，考虑这个更好的注释：

```
// 从最高到最低显示每个价格
for (list<Product>::reverse_iterator it = products.rbegin(); ... )
```

这个注释解释了程序在更高层次上正在做什么。这更符合程序员编写代码时的想法。

有趣的是，这个程序中有一个错误！CompareProductByPrice 函数（未显示）已经首先对价格较高的商品进行了排序。这段代码正在做与作者

意图相反的事情。

这是一个很好的理由，说明为什么第二个评论更好。尽管存在错误，但第一个评论在技术上是正确的（循环确实以相反的顺序迭代）。但是有了第二个评论，读者更有可能注意到作者的意图（首先显示价格较高的商品）与代码实际执行的操作相矛盾。实际上，该评论充当了一个冗余检查。

最终，最好的冗余检查是单元测试（参见第 14 章，“测试和可读性”）。但拥有像这样的注释来解释你的程序意图仍然是有价值的。

“命名函数参数”注释

假设你看到了一个像这样的函数调用：

```
连接(10, false);
```

这个函数调用有点神秘，因为传入了那些整数和布尔字面量。

在像 Python 这样的语言中，你可以通过名称来分配参数：

```
def Connect(超时时间, use_encryption): ...

# 使用命名参数调用函数
Connect(超时 = 10, 使用加密 = False)
```

在 C++ 和 Java 等语言中，你不能这样做。但是，你可以使用行内注释来达到同样的效果：

```
void Connect(int timeout, bool use_encryption) { ... }

// 使用注释参数调用函数
Connect(/* timeout_ms = */ 10, /* use_encryption = */ false);
```

请注意，我们将第一个参数“命名”为 `timeout_ms` 而不是 `timeout`。理想情况下，函数的实际参数应该是 `timeout_ms`，但如果由于某种原因我们无法进行此更改，这是一种“改进”名称的便捷方法。

对于布尔参数，在值 前面 放置 `/* name = */` 尤其重要。将注释放在值 后面 非常令人困惑：

```
// 不要这样做！
Connect( ... , false /* use_encryption */);

// 也不要这样做！
Connect( ... , false /* = use_encryption */);
```

在这些示例中，不清楚 `false` 是指“使用加密”还是“不使用加密”。

大多数函数调用不需要这样的注释，但这是解释看起来很神秘的参数的一种方便（且紧凑）的方式。

使用信息密集型词语

一旦你编程了很多年，你会注意到同样的一般问题和解决方案会重复出现。通常，已经开发出特定的词语或短语来描述这些模式/习惯用法。使用这些词语可以使你的注释更加简洁。

例如，假设你的评论是：

```
// 这个类包含许多成员，它们存储与数据库中相同的信息，但为了提高速度而存储在这里。//
// 稍后从该类读取时，首先检查这些成员是否存在，如果存在则返回；否则从// 数据库中读取，并
// 将该数据存储在这些字段中，以便下次使用。
```

相反，你可以这样说：

```
// 这个类充当数据库的 缓存层。
```

再举一个例子，像这样的注释：

```
// 删除街道地址中多余的空格，并进行大量其他清理
// 比如将“Avenue”变成“Ave.”。这样，如果有两个不同的街道地址
// 输入略有不同，它们将具有相同的清理版本，并且
// 我们可以检测到这些地址是相等的。
```

可以改为：

// 规范化 街道地址（删除多余空格，“Avenue” -> “Ave.” 等）

有很多词语和短语包含很多含义，例如“启发式”、“蛮力”、“朴素解”等等。如果你有一个感觉有点冗长的注释，看看它是否可以被描述为一种典型的编程情况。

总结

本章是关于编写尽可能在小空间内包含尽可能多信息的注释。以下是一些具体技巧：

- 避免使用“它”和“这个”之类的代词，因为它们可能指代多个事物。
- 尽可能精确地描述函数的行为。
- 用精心挑选的输入/输出示例来说明你的注释。
- 说明你的代码的高级意图，而不是显而易见的细节。
- 使用内联注释（例如，`Function(/* arg = */ ...)`）来解释神秘的函数参数。
- 使用包含大量含义的词语来保持你的注释简洁。

第二部分

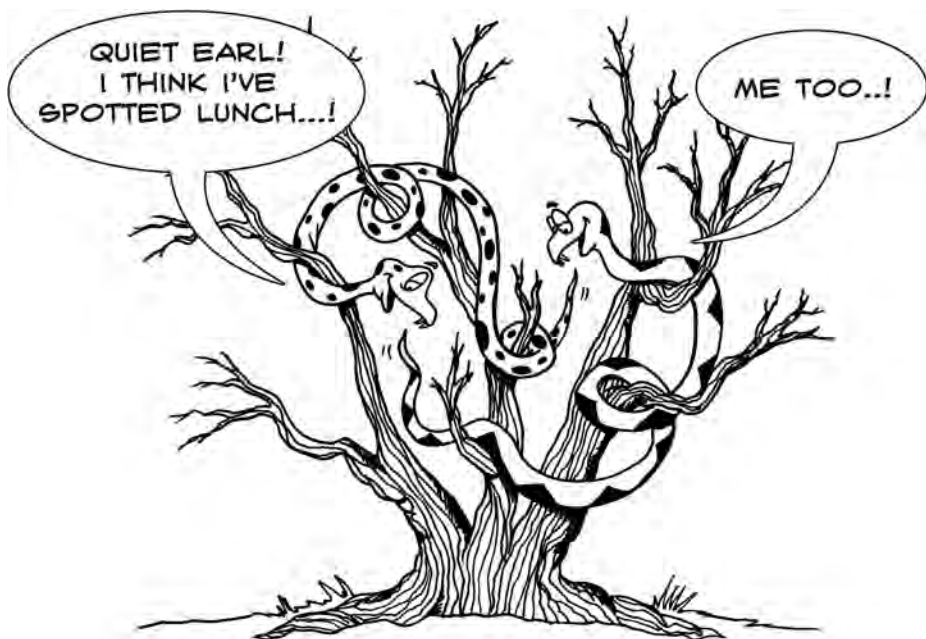
简化循环和逻辑

在[第一部分](#)中，我们介绍了表面上的改进——简单的方法来提高你的代码的可读性，一次一行，可以在没有太多风险或努力的情况下应用。

在接下来的部分中，我们将深入探讨程序的“循环和逻辑”：控制流、逻辑表达式和使代码工作的变量。与往常一样，我们的目标是使代码的这些部分易于理解。

我们通过尽量减少代码的“精神负担”来实现这一点。每次你看到一个复杂的循环、一个巨大的表达式或大量的变量，这都会增加你头脑中的精神负担。它要求你更努力地思考和记住更多。这与“易于理解”正好相反。当代码有很多精神负担时，bug更容易被忽视，代码变得更难修改，而且使用起来也不那么有趣。

使控制流易于阅读



如果代码没有条件语句、循环或任何其他*控制流*语句，它将非常容易阅读。这些跳转和分支是困难的部分，代码可能会很快变得混乱。
本章是关于使代码中的控制流易于阅读。

关键思想

使你的所有条件语句、循环和其他控制流的更改尽可能“自然”——以一种不会让读者停止并重新阅读你的代码的方式编写。

条件语句中参数的顺序

以下两段代码中哪一段更易读：

```
如果 (长度 >= 10)
```

或者

```
如果 (10 <= 长度)
```

对于大多数程序员来说，第一种方法更易读。但接下来的两种呢：

```
while (bytes_received < bytes_expected)
```

或者

```
while (bytes_expected > bytes_received)
```

同样，第一个版本更易读。但是为什么呢？通用规则是什么？你如何决定写 `a < b` 更好还是 `b > a` 更好？

以下是我们发现有用的一个指导原则：

左侧	右侧
“被询问” 的表达式，其值更不稳定。	被比较的表达式，其值更恒定。

这个指导原则符合英语用法——说“如果你每年至少赚 10 万美元”或“如果你至少 18 岁”是很自然的。说“如果 18 岁小于或等于你的年龄”是不自然的。

这解释了为什么 `while (bytes_received < bytes_expected)` 更具可读性。`bytes_received` 是我们正在检查的值，并且它随着循环的执行而增加。`bytes_expected` 是正在比较的更“稳定”的值。

“尤达表示法”：仍然有用吗？

在某些语言（包括 C 和 C++，但不包括 Java）中，将赋值放在 if

条件中是合法的：

```
if (obj = NULL) ...
```

很可能这是一个错误，程序员实际上想表达的是：

```
if (obj == NULL) ...
```

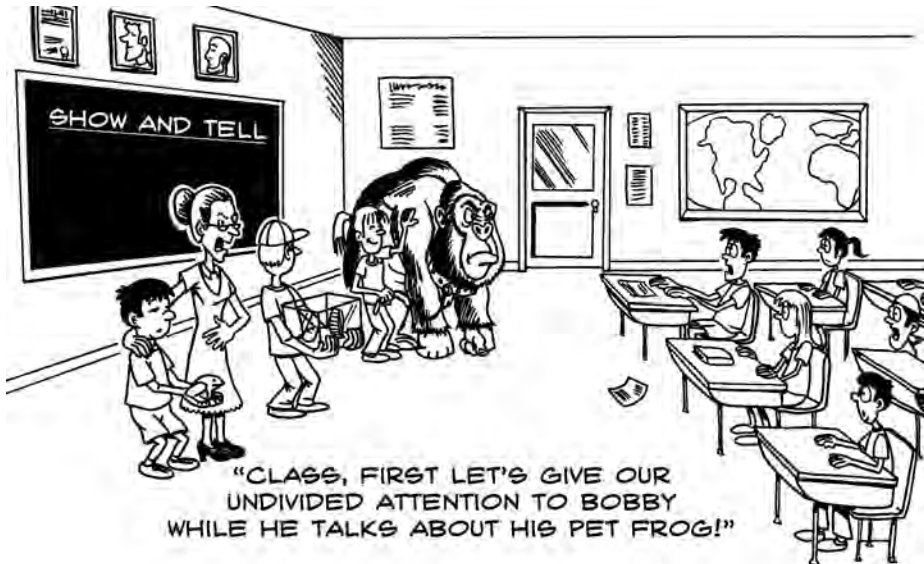
为了防止此类错误，许多程序员 切换参数的顺序：

```
if (NULL == obj) ...
```

这样，如果 == 意外地写成 =，表达式 if (NULL = obj 甚至无法编译。

不幸的是，切换顺序会使代码读起来有点不自然。（正如尤达会说的，“如果有什么要说的，我没有。”）值得庆幸的是，现代编译器会警告像 if (obj = 这样的代码 NULL)，所以“尤达表示法”正逐渐成为过去。

if/else 块的顺序



在编写 if/else 语句时，通常可以自由交换块的顺序。例如，您可以这样编写：

```
if (a == b) {  
    // 情况一 ...  
} else {  
    // 情况二 ...  
}
```

或者这样：

```
if (a != b) {  
    // 情况二 ...  
} else {  
    // 情况一 ...  
}
```

您以前可能没有过多考虑这个问题，但在某些情况下，有充分的理由选择一种顺序而不是另一种顺序：

- 优先处理正面情况，而不是负面情况——例如，if (debug) 而不是 if (!debug)。
- 优先处理更简单情况，以便尽快解决。这种方法也可能允许if和else同时显示在屏幕上，这很好。
- 优先处理更有趣或引人注目的情况。

有时这些偏好会发生冲突，你需要做出判断。但在很多情况下，都有一个明显的赢家。

例如，假设你有一个Web服务器，它正在构建一个响应，基于URL是否包含查询参数expand_all：

```
if (!url.HasQueryParameter("expand_all")) {  
    response.Render(items);  
    ...  
} else {  
    for (int i = 0; i < items.size(); i++) {  
        items[i].Expand();  
    }  
    ...  
}
```

当读者浏览第一行时，她的大脑立即想到expand_all的情况。这就像有人说，“不要想粉红色的象。”你忍不住会想关于它——“不要”被更不寻常的“粉红色的象”淹没了。

在这里，expand_all是我们的粉红色大象。因为它更有趣（而且是积极的情况），让我们先处理它：

```
if (url.HasQueryParameter("expand_all")) {  
    for (int i = 0; i < items.size(); i++) {  
        items[i].Expand();  
    }  
}
```

```

    ...
} else {
    response.Render(items);
    ...
}

```

另一方面，这里有一个负面情况更简单、更有趣/更危险的例子，所以我们首先处理它：

```

如果文件不存在：
    # 记录错误...
否则：
    # ...

```

同样，根据具体情况，这可能是一个判断。

总而言之，我们的建议是简单地关注这些因素，并注意你的if/else处于尴尬顺序的情况。

?: 条件表达式（又名“三元运算符”）

在类似 C 的语言中，你可以将条件表达式写成 `cond ? a : b`，这本质上是一种紧凑的编写 `if (cond) { a } else { b }` 的方式。

它对可读性的影响是有争议的。支持者认为这是一种用一行写出原本需要多行内容的好方法。反对者认为，它可能会让人感到困惑，并且难以在调试器中单步执行。

这是一个三元运算符可读且紧凑的例子：

```
time_str += (hour >= 12) ? "pm" : "am";
```

避免使用三元运算符，你可以这样写：

```

if (hour >= 12) {
    time_str += "pm";
} else {
    time_str += "am";
}

```

这有点冗长和多余。在这种情况下，条件表达式似乎是合理的。

然而，这些表达式很快就会变得难以阅读：

```
return exponent >= 0 ? mantissa * (1 << exponent) : mantissa / (1 << -exponent);
```

在这里，三元运算符不再只是在两个简单值之间进行选择。编写像这样的代码的动机通常是“将所有内容都挤在一行上”。

关键思想

与其最小化行数，不如使用更好的指标来最小化某人理解它所需的时间。

用 if/else 语句明确表达逻辑使代码更自然：

```
if (exponent >= 0) {  
    return mantissa * (1 << exponent);  
} else {  
    return mantissa / (1 << -exponent);  
}
```

建议

默认情况下，使用 if/else 三元 ?：只应在最简单的情况下使用。

避免 do/while 循环



许多受人尊敬的编程语言，以及 Perl，都有一个 `do { expression } while (condition)` 循环。expression 至少执行一次。这是一个例子：

```
// 在列表中搜索给定的 'name'，从 'node' 开始。  
// 不要考虑超过 'max_length' 个节点。  
public boolean ListHasNode(Node node, String name, int max_length) {  
    do {  
        if (node.name().equals(name))  
            return true;  
        node = node.next();  
    } while (node != null && --max_length > 0);  
  
    return false;  
}
```

do/while 循环的奇怪之处在于，一段代码块可能会或可能不会基于其下的条件重新执行。通常，逻辑条件位于它们所保护的代码之上——这与 if、while 和 for 语句的工作方式相同。因为您通常从上到下阅读代码，这使得 do/while 有点不自然。许多读者最终会阅读两次代码。

while 循环更容易阅读，因为在阅读内部的代码块之前，您已经知道所有迭代的条件。但是仅仅为了移除一个 do/while 而复制代码是很愚蠢的：

```
// 模仿 do/while — 不要这样做!  
主体  
  
while (条件) {  
    再次执行主体  
}
```

幸运的是，我们发现在实践中，大多数 do/while 循环一开始就可以写成 while 循环：

```
public boolean ListHasNode(Node node, String name, int max_length) {  
    while (node != null && max_length-- > 0) {  
        if (node.name().equals(name)) return true;  
        node = node.next();  
    }  
    return false;  
}
```

这个版本的好处还在于，如果 max_length 为 0 或者 node 为 null，它仍然有效。

避免 do/while 的另一个原因是 continue 语句在其中可能会令人困惑。例如，这段代码会做什么？

```
do {  
    continue;  
} while (false);
```

它会永远循环还是只循环一次？大多数程序员都必须停下来思考一下。（它应该只循环一次。）

总的来说，C++ 的创造者 Bjarne Stroustrup 在 *The C++ Programming Language* 中说得最好：

以我的经验，do 语句是错误和困惑的根源。... 我更喜欢“在前面我可以看见”的条件。因此，我倾向于避免 do 语句。

从函数中提前返回

有些程序员认为函数不应该有多个 return 语句。这是无稽之谈。从函数中提前返回是完全可以的，而且通常是可取的。例如：

```
public boolean Contains(String str, String substr) {  
    if (str == null || substr == null) return false;  
    if (substr.equals("")) return true;
```

```
    ...  
}
```

如果没有这些“保护子句”来实现这个函数，将会非常不自然。
想要单个退出点的一个动机是，函数底部的所有清理代码都保证被调用。但是现代语言提供了更复杂的方法来实现这种保证：

语言	用于清理代码的结构化习语
C++	析构函数
Java, Python	try finally
Python	with
C#	using

在纯 C 语言中，没有机制可以在函数退出时触发特定的代码。因此，如果有一个包含大量清理代码的大型函数，提前返回可能很难正确完成。在这种情况下，其他选项包括重构函数，甚至明智地使用 `goto cleanup;`

臭名昭著的 goto

在 C 语言以外的其他语言中，几乎不需要 `goto`，因为有很多更好的方法来完成这项工作。`goto` 也因很快失控并使代码难以理解而臭名昭著。
但是你仍然可以在各种 C 项目中看到 `goto` 的使用——最著名的是 Linux 内核。在你否定所有 `goto` 的使用为亵渎之前，分析一下为什么 `goto` 的一些使用比其他的更好是有用的。

`goto` 最简单、最无辜的用法是在函数底部使用单个 `exit`：

```
    if (p == NULL) goto exit;  
  
    ...  
  
exit:  
    fclose(file1);  
    fclose(file2);  
    ...  
  
    return;
```

如果这是唯一允许的 `goto` 形式，那么 `goto` 就不会有太大的问题。
当存在 *multiple* `goto` 目标时，问题就会出现，尤其是当它们的路径交叉时。
特别是，`goto` 向上 *upward* 移动会产生真正的意大利面条式代码，并且它们肯定可以被结构化循环替换。大多数情况下，应该避免使用 `goto`。

最小化嵌套

深度嵌套的代码难以理解。每一层嵌套都会在读者的“心理堆栈”上增加一个额外的条件。当读者看到一个右括号 (}) 时，很难“弹出”堆栈并记住下面的条件是什么。

这是一个相对简单的例子——看看你是否注意到自己回头检查你所在的块条件：

```
如果 (user_result == SUCCESS) {
    如果 (permission_result != SUCCESS) {
        reply.WriteErrors("error reading permissions");
        reply.Done();
    }
    return;
}
reply.WriteErrors("");
} else {
    reply.WriteErrors(user_result);
}
reply.Done();
```

当你看到第一个右括号时，你必须对自己说，*哦，permission_result != SUCCESS刚刚结束，所以现在permission_result == SUCCESS，这仍然在user_result == SUCCESS的块中。*

总的来说，你必须始终将user_result和permission_result的值记在脑海中。并且随着每个if { 块的关闭，你必须在你的脑海中切换相应的值。

这段特殊的代码甚至更糟糕，因为它一直在SUCCESS和非之间交替-成功情况。

嵌套如何累积

在我们尝试修复之前的示例代码之前，让我们讨论一下它是如何变成现在这样的。最初，代码很简单：

```
如果 (user_result == SUCCESS) {
    reply.WriteErrors("");
} else {
    reply.WriteErrors(user_result);
}
reply.Done();
```

这段代码完全可以理解——它计算出要写入的错误字符串，然后就完成了回复。

但是程序员又添加了第二个操作：

```
如果 (user_result == SUCCESS) {
    如果 (permission_result != SUCCESS) {
        reply.WriteErrors("error reading permissions");
    }
}
```

```

        reply.Done();
        返回;
    }
    reply.WriteErrors("");

```

...

这个改动是有道理的——程序员有一段新的代码要插入，她找到了最容易插入的地方。这段新代码在她脑海中是新鲜的，并且被“加粗”了。而且这个改动的“差异”非常干净——看起来像是一个简单的改动。

但是当其他人稍后遇到这段代码时，所有这些上下文都消失了。这就是你在本节开头第一次阅读代码时的感受——你必须一次性接受它。

关键思想

当你进行更改时，从全新的角度看待你的代码。退后一步，把它看作一个整体。

通过提前返回来移除嵌套

好的，让我们改进一下代码。像这样的嵌套可以通过尽快处理“失败情况”并从函数中提前返回来移除：

```

    if (user_result != SUCCESS) {
        reply.WriteErrors(user_result);
        reply.Done();
        return;
    }

    if (permission_result != SUCCESS) {
        reply.WriteErrors(permission_result);
        reply.Done();
        return;
    }

    reply.WriteErrors("");
    reply.Done();

```

这段代码只有一个嵌套级别，而不是两个。但更重要的是，读者永远不必从他的心理堆栈中“弹出”任何东西——每个 if 块都以 return 结尾。

移除循环内部的嵌套

提前返回的技术并非总是适用。例如，这里有一个嵌套在循环中的代码示例：

```

for (int i = 0; i < results.size(); i++) {
    if (results[i] != NULL) {
        non_null_count++;
    }
}

```

```

        if (results[i]->name != "") {
            cout << "正在考虑候选人..." << endl;

            ...
        }
    }
}

```

在循环中，类似于提前返回的技术是 `continue`

```

for (int i = 0; i < results.size(); i++) {
    if (results[i] == NULL) continue;
    non_null_count++;

    if (results[i]->name == "") continue;
    cout << "正在考虑候选者..." << endl;

    ...
}

```

正如 `if (...) return;` 作为一个函数的保护子句一样，这些 `if (...) continue;` 语句也充当循环的保护子句。

一般来说，`continue` 语句可能会让人感到困惑，因为它会让读者感到跳跃，就像循环中的 `goto` 一样。但在这种情况下，循环的每次迭代都是独立的（循环是一个“for each”），因此读者可以很容易地看到 `continue` 只是意味着“跳过此项”。

你能否追踪执行流程？



本章主要介绍底层控制流：如何使循环、条件和其他跳转易于阅读。但你也应该从更高的层面考虑程序的“流程”。理想情况下，应该很容易跟踪程序的整个执行路径——你从 `main()` 开始，并在脑海中逐步执行代码，就像一个函数调用另一个函数一样，直到程序退出。

但在实践中，编程语言和库具有一些构造，可以让代码在“幕后”执行，或者难以跟踪。以下是一些示例：

编程构造	高级程序流程如何被模糊化
线程	不清楚何时执行什么代码。
信号/中断处理程序	某些代码可能在任何时候执行。
异常	执行可以冒泡通过多个函数调用。
函数指针和匿名函数	很难确切知道要运行什么代码，因为在编译时这是未知的。
虚方法	<code>object.virtualMethod()</code> 可能会调用未知子类的代码。

其中一些结构非常有用，它们甚至可以使你的代码更具可读性，并减少冗余。但作为程序员，有时我们会得意忘形，过度使用它们，而没有意识到读者以后理解代码会有多困难。

此外，这些结构使错误更难追踪。

关键是不要让你的代码中过多的百分比使用这些结构。如果你滥用这些功能，它会使追踪你的代码就像玩三张牌蒙特游戏（如

卡通中所示）。

总结

你可以做很多事情来使你的代码的控制流程更容易阅读。

在编写比较时（`while (bytes_expected > bytes_received)`），最好将变化的值放在左边，更稳定的值放在右边（`while (bytes_received < bytes_expected)`）。

你也可以重新排序 `if/else` 语句的块。通常，尝试先处理肯定/更容易/更有趣的情况。有时这些标准会冲突，但当它们不冲突时，这是一个很好的经验法则。

某些编程结构，如三元运算符（`:` `?`），`do/while` 循环和 `goto` 通常会导致不可读的代码。通常最好不要使用它们，因为几乎总是存在更清晰的替代方案。

嵌套代码块需要更多的注意力才能跟上。每个新的嵌套都需要更多的上下文才能“推到”读者的“堆栈”上。相反，选择更“线性”的代码以避免深度嵌套。

提前返回可以删除嵌套并清理代码。“Guard 语句”（在函数顶部处理简单情况）特别有用。

分解巨大的表达式



巨型鱿鱼是一种令人惊叹且聪明的动物，但其近乎完美的身体设计有一个致命的缺陷：它有一个环绕其食道的甜甜圈形大脑。因此，如果它一次吞下太多的食物，它就会受到脑损伤。

这与代码有什么关系？嗯，以“块”形式出现且过大的代码可能会产生同样的效果。最近的研究表明，我们大多数人一次只能思考三到四件“事情”。*简而言之，代码表达式越大，就越难理解。

关键思想

将你的巨大表达式分解成更容易理解的部分。

在本章中，我们将介绍各种方法，你可以操纵和分解你的代码，使其更容易理解。

解释变量

分解表达式最简单的方法是引入一个额外的变量来捕获一个较小的子表达式。这个额外的变量有时被称为“解释变量”，因为它有助于解释子表达式的含义。

这是一个例子：

```
如果 line.split(':')[0].strip() == "root":  
    ...
```

这是相同的代码，现在带有一个解释变量：

```
用户名 = line.split(':')[0].strip()  
if 用户名 == "root":  
    ...
```

摘要变量

即使一个表达式不需要 *解释*（因为你可以弄清楚它的意思），将该表达式捕获到一个新的变量中仍然是有用的。如果它的目的仅仅是用一个更小的名称来替换一大块代码，以便更容易地管理和思考，我们称之为 *摘要变量*。

例如，考虑以下代码中的表达式：

```
if (request.user.id == document.owner_id) {  
    // 用户可以编辑此文档...  
}
```

* Cowan, N. (2001). 短期记忆中的神奇数字4：对心理存储的重新思考容量。 *行为和脑科学*, 24, 97–185.


```
...

if (request.user.id != document.owner_id) {
    // 文档是只读的...
}

```

表达式 `request.user.id == document.owner_id` 可能看起来不是很大，但它有五个变量，所以需要额外的时间来思考。

这段代码的主要概念是“用户是否拥有该文档？”通过添加一个摘要变量，可以更清楚地表达这个概念：

```
final boolean user_owns_document = (request.user.id == document.owner_id);

if (user_owns_document) {
    // 用户可以编辑此文档...
}

...

if (!user_owns_document) {
    // 文档是只读的...
}

```

这可能看起来不多，但语句 `if (user_owns_document)` 更容易思考。此外，在顶部定义 `user_owns_document` 会预先告诉读者“这是一个我们将在整个函数中引用的概念”。

使用德摩根定律

如果你曾经上过电路或逻辑课程，你可能会记得德摩根定律。它们是将布尔表达式重写为等效表达式的两种方法：

- 1) $\text{not } (a \text{ or } b \text{ or } c) \Leftrightarrow (\text{not } a) \text{ and } (\text{not } b) \text{ and } (\text{not } c)$
- 2) $\text{not } (a \text{ and } b \text{ and } c) \Leftrightarrow (\text{not } a) \text{ or } (\text{not } b) \text{ or } (\text{not } c)$

如果你难以记住这些定律，一个简单的总结是“分配 `not` 和切换 `and/or`。”（或者反过来，你“分解出 `not`。”）

有时你可以使用这些定律来使布尔表达式更具可读性。例如，如果你的代码是：

```
if (!(file_exists && !is_protected)) 错误("抱歉，无法读取文件。");

```

它可以被改写为：

```
if (!file_exists || is_protected) Error("Sorry, could not read file.");

```

滥用短路逻辑

在大多数编程语言中，布尔运算符执行短路求值。例如，语句 `if (a || b)` 如果 `a` 为真，则不会计算 `b`。这种行为非常方便，但有时会被滥用以完成复杂的逻辑。

这是一个作者曾经写过的一个语句的例子：

```
assert((!(bucket = FindBucket(key))) || !bucket->IsOccupied());
```

用英语来说，这段代码的意思是，“获取此键的存储桶。如果存储桶不为空，则确保它未被占用。”

即使它只是一行代码，它确实让大多数程序员停下来思考。现在将其与以下代码进行比较：

```
bucket = FindBucket(key);  
if (bucket != NULL) assert(!bucket->IsOccupied());
```

它做的事情完全相同，即使它是两行代码，也更容易理解。

那么，为什么这段代码一开始就被写成一个巨大的表达式呢？当时，感觉非常聪明。将逻辑简化为简洁的代码片段会带来一定的乐趣。这是可以理解的——就像解决一个微型难题一样，我们都喜欢在工作中获得乐趣。问题是，这段代码对于任何阅读代码的人来说都是一个心理障碍。

关键思想

当其他人稍后阅读

1991, 4:12 PM, 1991/12/12 - 1991/12/12

这是否意味着你应该避免使用短路行为？不。有很多情况下可以干净地使用它，例如：

```
if (object && object->method()) ...
```

还有一个值得一提的较新的习惯用法：在像 Python、JavaScript 和 Ruby 这样的语言中，“或”运算符返回它的一个参数（它不会转换为布尔值），所以像这样的代码：

```
x = a || b || c
```

可以用来从 `a`、`b` 或 `c` 中挑选出第一个“真值”。

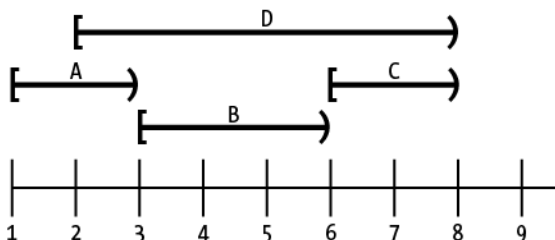
示例：与复杂逻辑作斗争

假设你正在实现以下 `Range` 类：

```
struct Range {  
    int begin;  
    int end;
```

```
// 例如, [0,5) 与 [3,8) 重叠
    bool OverlapsWith(Range other);
};
```

下图显示了一些示例范围:

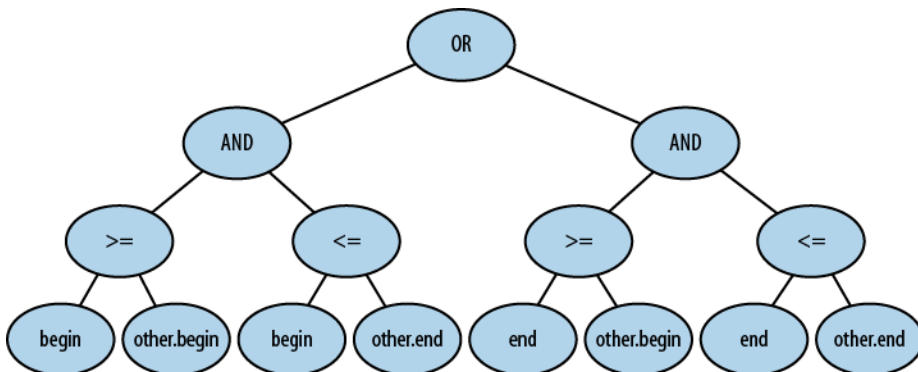


请注意, 结束 是非包含性的。因此, A、B 和 C 彼此不重叠, 但 D 与它们全部重叠。

这是一个实现 `OverlapsWith()` 的尝试——它检查其范围的任一端点是否落在 另一个 的范围内:

```
bool Range::OverlapsWith(Range other) {
    // 检查 'begin' 或 'end' 是否落在 'other' 内。
    return (begin >= other.begin && begin <= other.end) ||
           (end >= other.begin && end <= other.end);
}
```

即使代码只有两行长, 也有很多事情在发生。下图显示了所有涉及的逻辑。



有太多的情况和条件需要考虑, 以至于很容易出现错误。

说到这, 这里 有一个错误。之前的代码会声称 `Range [0,2)` 与 `[2,4)` 重叠, 但实际上并没有。

问题在于，在使用 `<=` 或仅使用

`<` 比较 `begin/end` 值时，必须小心。以下是此问题的修复方法：

```
return (begin >= other.begin && begin < other.end) ||
        (end > other.begin && end <= other.end);
```

现在它正确了吗？实际上，还有 另一个 错误。此代码忽略了 `begin/end` 完全包围 `other` 的情况。

这是一个处理这种情况的修复方案：

```
return (begin >= other.begin && begin < other.end) ||
        (end > other.begin && end <= other.end) ||
        (begin <= other.begin && end >= other.end);
```

糟糕——这段代码变得太复杂了。你不能指望任何人阅读这段代码并自信地知道它是正确的。那么我们该怎么办？我们如何分解这个巨大的表达式？

寻找更优雅的方法

这是你应该停下来并考虑完全不同的方法的时候之一。

最初是一个简单的问题（检查两个范围是否重叠），结果变成了一个令人惊讶的复杂逻辑。这通常是一个迹象，表明一定有更简单的方法。

但是找到更优雅的解决方案需要创造力。你如何做到这一点？一种技术是看看你是否可以用“相反”的方式解决问题。根据你所处的情况，这可能意味着以相反的顺序迭代数组，或者以向后的方式填充一些数据结构，而不是向前。

在这里，`OverlapsWith()` 的反面是“不重叠”。确定两个范围不重叠原来是一个简单得多的问题，因为只有两种可能性：

1. 另一个范围在此范围开始之前结束。
2. 另一个范围在此范围结束后开始。

我们可以很容易地将此转化为代码：

```
bool Range::OverlapsWith(Range other) {
    if (other.end <= begin) return false; // They end before we begin
    if (other.begin >= end) return false; // They begin after we end

    return true; // Only possibility left: they overlap
}
```

这里的每一行代码都简单得多——它只涉及一个比较。这让读者有足够的脑力来关注 `<=` 是否正确。

分解巨大的语句

本章是关于分解单个表达式，但相同的技术也适用于分解更大的语句。例如，以下 JavaScript 代码一次性包含了很多内容：

```
var update_highlight = function (message_num) {
    if ($("#vote_value" + message_num).html() === "Up") {
        $("#thumbs_up" + message_num).addClass("highlighted");
        $("#thumbs_down" + message_num).removeClass("highlighted");
    } else if ($("#vote_value" + message_num).html() === "Down") {
        $("#thumbs_up" + message_num).removeClass("highlighted");
        $("#thumbs_down" + message_num).addClass("highlighted");
    } else {
        $("#thumbs_up" + message_num).removeClass("highlighted");
        $("#thumbs_down" + message_num).removeClass("highlighted");
    }
};
```

此代码中的单个表达式并不大，但是当它们全部放在一起时，它会形成一个巨大的语句，一次性冲击您。

幸运的是，很多表达式都是相同的，这意味着我们可以将它们提取为函数顶部的摘要变量（这也是 DRY—Don't Repeat Yourself—原则的一个实例）：

```
var update_highlight = function (message_num) {
    var thumbs_up = $("#thumbs_up" + message_num);
    var thumbs_down = $("#thumbs_down" + message_num);
    var vote_value = ($("#vote_value" + message_num).html());
    var hi = "highlighted";

    如果 vote_value === "Up" {
        thumbs_up.addClass(hi);
        thumbs_down.removeClass(hi);
    } else if vote_value === "Down" {
        thumbs_up.removeClass(hi);
        thumbs_down.addClass(hi);
    } else {
        thumbs_up.removeClass(hi);
        thumbs_down.removeClass(hi);
    }
};
```

创建 `var hi = "highlighted"` 并非绝对必要，但由于它有 *six* 个副本，因此具有令人信服的优势：

- 它有助于避免输入错误。（事实上，你是否注意到在第一个例子中，字符串在第五种情况下被错误拼写为 "highighted" 吗？）
- 它进一步缩小了行宽，使代码更易于扫描。
- 如果需要更改类名，则只需在一个地方更改它。

另一种简化表达式的创造性方法

这是另一个例子，每个表达式中都有很多内容，这次是用 C++：

```
void AddStats(const Stats& add_from, Stats* add_to) {
    add_to->set_total_memory(add_from.total_memory() + add_to->total_memory());
    add_to->set_free_memory(add_from.free_memory() + add_to->free_memory());
    add_to->set_swap_memory(add_from.swap_memory() + add_to->swap_memory());
    add_to->set_status_string(add_from.status_string() + add_to->status_string());
    add_to->set_num_processes(add_from.num_processes() + add_to->num_processes());
    ...
}
```

再一次，你的眼睛面对的是长而相似，但又不完全相同的代码。
经过十秒钟的仔细审查，您可能会意识到每一行都在做同样的事情，
只是每次针对不同的字段：

```
add_to->setXXX(add_fromXXX() + add_to->XXX());
```

在 C++ 中，我们可以定义一个宏来实现这个：

```
void AddStats(const Stats& add_from, Stats* add_to) {
    #define ADD_FIELD(field) add_to->set_##field(add_from.field() + add_to->field())

    ADD_FIELD(total_memory);
    ADD_FIELD(free_memory);
    ADD_FIELD(swap_memory);
    ADD_FIELD(status_string);
    ADD_FIELD(num_processes);
    ...
    #undef ADD_FIELD
}
```

现在我们已经去掉了所有的混乱，你可以看看代码并立即理解正在发生的事情的本质。
很明显，每一行都在做同样的事情。

请注意，我们并不提倡经常使用宏——事实上，我们通常避免使用它们，因为它们会使代码混乱并引入细微的错误。但有时，就像在这种情况下一样，它们很简单，并且可以为可读性提供明显的好处。

总结

巨大的表达式很难考虑。本章展示了许多分解它们的方法，以便读者可以逐个消化它们。

一种简单的技术是引入“解释变量”，以捕获某些大型子表达式的值。这种方法有三个好处：

- 它将一个巨大的表达式分解成若干部分。
- 它通过用简洁的名称描述子表达式来记录代码。
- 它帮助读者识别代码中的主要“概念”。

另一种技术是使用德摩根定律来操作你的逻辑——这种技术有时可以用更简洁的方式重写布尔表达式（例如，`if (!(a && !b))` 变成 `if(!a || b)`）。

我们展示了一个例子，其中一个复杂的逻辑条件被分解成像“`if (a < b) .`”。这样的小语句。事实上，本章中所有改进的代码示例都有`if`语句，其中不超过`two`个值。这种设置是理想的。这样做可能并不总是可行——有时它需要“否定”问题或考虑与你的目标相反的情况。

最后，即使本章是关于分解单个表达式，这些相同的技术通常也适用于更大的代码块。因此，积极地分解你看到的任何复杂逻辑。

变量和可读性



在本章中，您将看到草率使用 变量 如何使程序更难理解。

具体来说，有三个问题需要解决：

1. 变量越多，就越难跟踪它们。
2. 变量的作用域越大，你就需要跟踪它的时间越长。
3. 变量更改的频率越高，就越难跟踪其当前值。

接下来的三个部分讨论如何处理这些问题。

消除变量

在第 8 章，[分解大型表达式](#)中，我们展示了引入“解释”或“摘要”变量如何使代码更具可读性。这些变量很有用，因为它们分解了大型表达式并充当了一种文档形式。

在本节中，我们有兴趣消除那些不能提高可读性的变量。当像这样的变量被删除时，新代码会更简洁，并且同样容易理解。

在下一节中，是一些关于这些不必要的变量如何出现的例子。

无用的临时变量

在下面的 Python 代码片段中，考虑现在变量：

```
now = datetime.datetime.now()
root_message.last_view_time = 现在
```

现在 是一个值得保留的变量吗？不是，原因如下：

- 它没有分解复杂的表达式。
- 它没有增加清晰度——表达式 `datetime.datetime.now()` 已经足够清晰了。
- 它只使用一次，因此不会压缩任何冗余代码。

没有 现在，代码同样易于理解：

```
root_message.last_view_time = datetime.datetime.now()
```

像 现在 这样的变量通常是代码编辑后留下的“残余”。变量现在 最初可能在多个地方使用。或者，编码人员可能预计会多次使用现在，但实际上从未需要它。

消除中间结果



这是一个从数组中删除值的 JavaScript 函数的示例：

```
var remove_one = function (数组, 要移除的值) {  
    var index_to_remove = null;  
    for (var i = 0; i < 数组.length; i += 1) {  
        if (数组[i] === 要移除的值) {  
            index_to_remove = i;  
            break;  
        }  
    }  
    if (index_to_remove !== null) {  
        数组.splice(index_to_remove, 1);  
    }  
};
```

变量 `index_to_remove` 只是用来保存一个 *中间结果*。像这样的变量可以通过在获得结果后立即处理来消除：

```
var remove_one = function (数组, 要移除的值) {  
    for (var i = 0; i < 数组.length; i += 1) {  
        if (数组[i] === 要移除的值) {  
            数组.splice(i, 1);  
            return;  
        }  
    }  
};
```

通过允许代码提前返回，我们完全摆脱了 `index_to_remove` 并简化了代码。

一般来说，尽快完成任务 是一个好策略。

消除控制流变量

有时你会在循环中看到这种代码模式：

```
boolean done = false;

while (/* condition */ && !done) {
    ...

    if (...) {
        done = true;
        continue;
    }
}
```

变量done 甚至可能在整个循环的多个位置设置为true。

这样的代码通常试图满足一些不成文的规则，即你不应该从循环的中间跳出。

没有这样的规则！

像done这样的变量就是我们所说的“控制流变量”。它们的唯一目的是引导程序的执行——它们不包含任何实际的程序数据。根据我们的经验，通过更好地利用结构化编程，通常可以消除控制流变量：

```
while (/* condition */) {
    ...
    如果(...) {
        break;
    }
}
```

这个案例很容易修复，但是如果存在多个嵌套循环，而一个简单的中断还不够吗？在更复杂的情况下，解决方案通常涉及将代码移动到一个新的函数中（循环内部的代码，或整个循环本身）。

你希望你的同事一直感觉像在面试吗？

微软的埃里克·布雷克纳曾谈到，一个好的面试问题应该至少包含三个变量。^{*}这可能是因为同时处理三个变量会迫使你认真思考！这对于面试来说是有意义的，因为你试图将候选人推向极限。

但是你想让你的同事在阅读你的代码时感觉像是在面试吗？

^{*} 埃里克·布雷克纳的*I. M. 赖特的“硬代码”*（微软出版社，2007年），第166页。

缩小变量的作用域

我们都听过“避免使用全局变量”的建议。这是一个很好的建议，因为很难跟踪所有这些全局变量在哪里以及如何被使用。并且通过“污染命名空间”（在那里放置一堆可能与你的局部变量冲突的名称），代码可能会在你打算使用局部变量时意外地修改全局变量，反之亦然。

事实上，缩小所有变量（不仅仅是全局变量）的作用域是个好主意。

关键思想

尽可能用最少的代码行数使你的变量可见。

许多编程语言提供多种作用域/访问级别，包括模块、类、函数和块作用域。使用更严格的访问通常更好，因为它意味着变量可以被更少的代码行“看到”。

为什么要这样做？因为它有效地减少了读者必须同时考虑的变量数量。如果将所有变量的作用域缩小一半，那么在任何给定时间，平均作用域内的变量数量将减少一半。

例如，假设你有一个非常大的类，其中一个成员变量只被两个方法使用，如下所示：

```
class LargeClass {
    string str_;

    void Method1() {
        str_ = ...;
        Method2();
    }

    void Method2() {
        // Uses str_
    }

    // Lots of other methods that don't use str_ ...
};
```

在某种意义上，类成员变量就像类范围内的“迷你全局变量”。特别是对于大型类，很难跟踪所有成员变量以及哪些方法修改了每个变量。迷你全局变量越少越好。

对于这种情况，将 `str_` “降级”为局部变量可能更有意义：

```
class LargeClass {
    void Method1() {
        string str = ...;
        Method2(str);
    }
}
```

```

    void Method2(string str) {
        // 使用 str
    }

    // 现在其他方法无法看到 str。
};

```

限制对类成员访问的另一种方法是尽可能多地将方法设为静态方法。静态方法是让读者知道“这些代码行与这些变量隔离”的好方法。

或者另一种方法是将大型类分解为较小的类。只有当较小的类实际上彼此隔离时，此方法才有用。如果您要创建两个访问彼此成员的类，那么您实际上什么也没有完成。

将大文件分解为较小的文件或将大型函数分解为较小的函数也是如此。这样做的主要动机是隔离数据（即变量）。

但是不同的语言对于什么构成作用域有不同的规则。我们想指出一些关于变量作用域的更有趣的规则。

C++ 中的 if 语句作用域

假设您有以下 C++ 代码：

```

PaymentInfo* info = database.ReadPaymentInfo();
if (info) {
    cout << "用户支付: " << info->amount() << endl;

    // 下面还有很多代码...
}

```

变量 `info` 将在函数的其余部分保持作用域，因此阅读此代码的人可能会记住 `info`，想知道它是否/将如何再次使用。

但在这种情况下，`info` 仅在 `if` 语句中使用。在 C++ 中，我们实际上可以在条件表达式中定义 `info`：

```

if (PaymentInfo* info = database.ReadPaymentInfo()) {
    cout << "用户支付: " << info->amount() << endl;
}

```

现在读者可以很容易地忘记 `info` 在它超出范围之后。

在 JavaScript 中创建“私有”变量

假设你有一个持久变量，只被一个函数使用：

```
submitted = false; // 注意：全局变量

var submit_form = function (form_name) {
    if (submitted) {
        return; // 不要重复提交表单
    }
    ...
    已提交 = true;
};
```

像submitted这样的全局变量可能会让阅读这段代码的人感到非常焦虑。看起来好像submit_form()是唯一使用submitted的函数，但你不能确定。事实上，另一个JavaScript文件可能也在使用一个名为submitted的全局变量，用于不同的目的！

你可以通过将submitted包装在一个闭包中来防止这个问题：

```
var submit_form = (function () {
    var submitted = false; // 注意：只能由下面的函数访问

    return function (form_name) {
        if (submitted) {
            return; // 不要重复提交表单
        }
        ...
        submitted = true;
    };
})();
```

注意最后一行的括号——匿名外部函数会立即执行，返回内部函数。

如果你以前没见过这种技术，乍一看可能会觉得很奇怪。它具有创建一个“私有”作用域的效果，只有内部函数才能访问。现在读者不必想知道提交还在哪里使用？或者担心与同名的其他全局变量冲突。（更多类似的技术，请参见Douglas Crockford的《[JavaScript: The Good Parts](#)》[O’Reilly, 2008]。）

JavaScript 全局作用域

在 JavaScript 中，如果你从变量定义中省略关键字 `var`（例如，`x = 1`而不是 `var x = 1`），则该变量将放入全局作用域，其中 *every* JavaScript 文件和 `<script>`块都可以访问它。这是一个例子：

```
<script>
  var f = function () {
    // 危险: 'i' 没有用 'var' 声明!
    for (i = 0; i < 10; i += 1) ...
  };

  f();
</script>
```

这段代码不小心将 `i` 放入了全局作用域，因此后面的代码块仍然可以看到它：

```
<script>
  alert(i); // 弹出 '10'。'i' 是一个全局变量!
</script>
```

许多程序员没有意识到这个作用域规则，这种令人惊讶的行为会导致奇怪的错误。这种错误的一个常见表现是，当两个函数都创建一个具有相同名称的局部变量，但忘记使用 `var` 时。这些函数会在不知不觉中“串扰”，可怜的程序员可能会得出结论，他的电脑被附身了，或者他的 RAM 坏了。

JavaScript 的常见“最佳实践”是始终使用 `var` 关键字 定义变量（例如，`var x = 1`）。这种做法将变量的作用域限制在定义它的（最内层）函数中。

Python 和 JavaScript 中没有嵌套作用域

像 C++ 和 Java 这样的语言具有 *块作用域*，其中在 `if`、`for`、`try` 或类似结构中定义的变量被限制在该块的嵌套作用域中：

```
if (...) {
  int x = 1;
}
x++; // 编译错误! 'x' 未定义。
```

但在 Python 和 JavaScript 中，在块中定义的变量会“溢出”到整个函数。

例如，请注意在此完全有效的 Python 代码中使用了 `example_value`

```
# 在此之前未使用 example_value。
if request:
    for value in request.values:
        if value > 0:
            example_value = value
            break

for logger in debug.loggers:
    logger.log("Example: %s", example_value)
```


这种作用域规则让许多程序员感到惊讶，并且这样的代码更难阅读。在其他语言中，更容易找到 `example_value` 首次定义的位置——您会沿着您所在的函数的“左边缘”查找。

之前的例子也有错误：如果在代码的第一部分没有设置 `example_value` 那么第二部分会引发一个异常：“`NameError: 'example_value' is not defined`”。我们可以修复这个问题，并通过在“最近公共祖先”（就嵌套而言）中定义 `example_value` 来使代码更具可读性，使其更接近使用它的地方：

```
example_value = None

如果请求:
    for value in request.values:
        if value > 0:
            example_value = value
            break

如果 example_value:
    对于调试记录器中的每个记录器:
        logger.log("示例: ", example_value)
```

然而，这是一个可以完全消除 `example_value` 的情况。`example_value` 只是持有中间结果，正如我们在“[消除中间](#)”中看到的那样第95页的“结果”，像这样的变量通常可以通过“尽快完成任务”来消除。在这种情况下，这意味着在我们找到示例值后立即记录它。

以下是新代码的样子：

```
def LogExample(value):
    对于调试记录器中的每个记录器:
        logger.log("示例: ", value)

if request:
    for value in request.values:
        if value > 0:
            LogExample(value) # 立即处理 'value'
            break
```

将定义下移

最初的 C 编程语言要求所有变量定义都位于函数或块的顶部。这个要求是不幸的，因为对于具有许多变量的长函数，它迫使读者立即考虑所有这些变量，即使它们直到很久以后才被使用。（C99 和 C++ 取消了此要求。）

在以下示例中，所有变量都无辜地定义在函数的顶部：

```
def ViewFilteredReplies(original_id):
    filtered_replies = []
    root_message = Messages.objects.get(original_id)
    all_replies = Messages.objects.select(root_id=original_id)
```

```

root_message.view_count += 1
root_message.last_view_time = datetime.datetime.now()
root_message.save()

    for reply in all_replies:
        如果 reply.spam_votes 小于等于 MAX_SPAM_VOTES:
            filtered_replies.append(reply)

    return filtered_replies

```

这个示例代码的问题在于，它迫使读者同时思考三个变量，并在它们之间切换。

因为读者不需要立即了解所有这些变量，所以很容易将每个定义移动到你首次使用之前：

```

def ViewFilteredReplies(original_id):
    root_message = Messages.objects.get(original_id)
    root_message.view_count += 1
    root_message.last_view_time = datetime.datetime.now()
    root_message.save()

    all_replies = Messages.objects.select(root_id=original_id)
    filtered_replies = []
    for reply in all_replies:
        如果 reply.spam_votes 小于等于 MAX_SPAM_VOTES:
            filtered_replies.append(reply)

    return filtered_replies

```

你可能想知道 `all_replies` 是否是一个必要的变量，或者是否可以通过以下方式消除它：

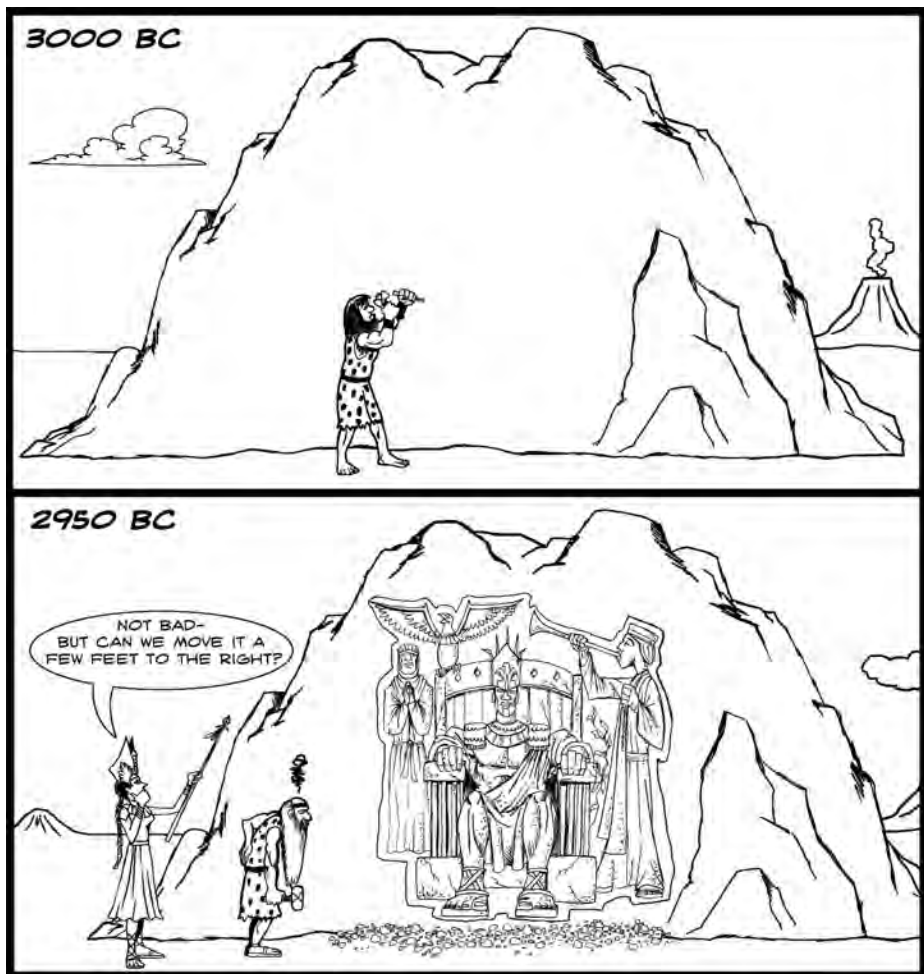
```

    for reply in Messages.objects.select(root_id=original_id)
    ...

```

在这种情况下，`all_replies` 是一个很好的解释变量，所以我们决定保留它。

倾向于只写一次的变量



到目前为止，在本章中，我们已经讨论了理解具有大量“正在运行”的变量的程序有多么困难。好吧，思考那些不断变化的变量甚至更难。跟踪它们的值会增加额外的难度。

为了解决这个问题，我们有一个听起来可能有点奇怪的建议：更喜欢一次写入变量。

作为“永久固定装置”的变量更容易思考。当然，像这样的常量：

```
static const int NUM_THREADS = 10;
```

不需要读者过多思考。出于同样的原因，强烈建议在 C++ 中使用 `const`（以及在 Java 中使用 `final`）。

事实上，在许多语言（包括 Python 和 Java）中，一些内置类型（如 `string`）是 *immutable*。正如 James Gosling（Java 的创建者）所说，“[不可变对象]往往更少出现问题。”

但是，即使你不能使你的变量只写一次，如果变量在更少的地方更改，仍然会有帮助。

关键思想

变量被操作的地方越多，就越难推断其当前值。

那么你该怎么做呢？如何将变量更改为只写一次？嗯，很多时候它需要稍微重构代码，正如你将在下一个例子中看到的那样。

最后一个例子

在本章的最后一个例子中，我们想展示一个例子，演示我们到目前为止讨论的许多原则。

假设你有一个网页，其中包含多个输入文本字段，像这样：

```
<input type="text" id="input1" value="Dustin">
<input type="text" id="input2" value="Trevor">
<input type="text" id="input3" value="">
<input type="text" id="input4" value="Melissa">
...
```

正如你所看到的，ids 以 `input1` 开始，并从那里递增。

你的任务是编写一个名为 `setFirstEmptyInput` 的函数，该函数接受一个字符串，并将其放在页面上的第一个空 `<input>` 中（在所示的示例中，为 `"input3"`）。该函数应返回已更新的 DOM 元素（如果没有任何剩余的空输入，则返回 `null`）。以下是一些不应用本章原则的代码：

```
var setFirstEmptyInput = function (new_value) {
    var found = false;
    var i = 1;
    var elem = document.getElementById('input' + i);
    while (elem !== null) {
        if (elem.value === '') {
            found = true;
            break;
        }
        i++;
        elem = document.getElementById('input' + i);
    }
}
```

```

        如果找到, elem.value = new_value;
        返回 elem;
    };

```

这段代码完成了工作,但并不漂亮。它有什么问题,我们如何改进它?

有很多方法可以考虑改进这段代码,但我们将从它使用的变量的角度来考虑:

- var found
- var i
- var elem

所有这三个变量都存在于整个函数中,并且被多次写入。让我们尝试改进它们的使用。

正如我们在本章前面讨论的那样,像 found 这样的中间变量通常可以通过提前返回来消除。这是改进后的代码:

```

var setFirstEmptyInput = function (new_value) {
    var i = 1;
    var elem = document.getElementById('input' + i);
    while (elem !== null) {
        如果 (elem.value === '') {
            elem.value = new_value;
            return elem;
        }
        i++;
        elem = document.getElementById('input' + i);
    }
    return null;
};

```

接下来,看看 elem。它在代码中以一种非常“循环”的方式多次使用,很难跟踪它的值。代码似乎 elem 是我们正在迭代的值,而实际上我们只是在递增 i。所以让我们重构 while 循环为 for 循环,遍历 i:

```

var setFirstEmptyInput = function (new_value) {
    for (var i = 1; true; i++) {
        var elem = document.getElementById('input' + i);
        if (elem === null)
            return null; // 搜索失败。未找到空输入。

        if (elem.value === '') {
            elem.value = new_value;
            return elem;
        }
    }
};

```

特别注意 `elem` 如何充当一个只写一次的变量，其生命周期包含在 `the` 循环中。使用 `true` 作为 `for` 循环条件是不寻常的，但作为交换，我们能够在单行中看到 `i` 的定义和修改。（传统的 `while (true)` 也是合理的。）

总结

本章讲述的是程序中的变量如何快速累积并变得难以跟踪。您可以通过减少变量的数量并使其尽可能“轻量级”来使您的代码更易于阅读。具体来说：

- 消除变量只是碍事。特别是，我们展示了一些示例，说明如何通过立即处理结果来消除“中间结果”变量。
- 尽可能缩小每个变量的作用域。将每个变量移动到最少的代码行可以看到它的位置。眼不见，心不烦。
- 首选只写一次的变量。只设置一次的变量（或 `const`、`final` 或其他不可变的变量）使代码更容易理解。

重组你的代码

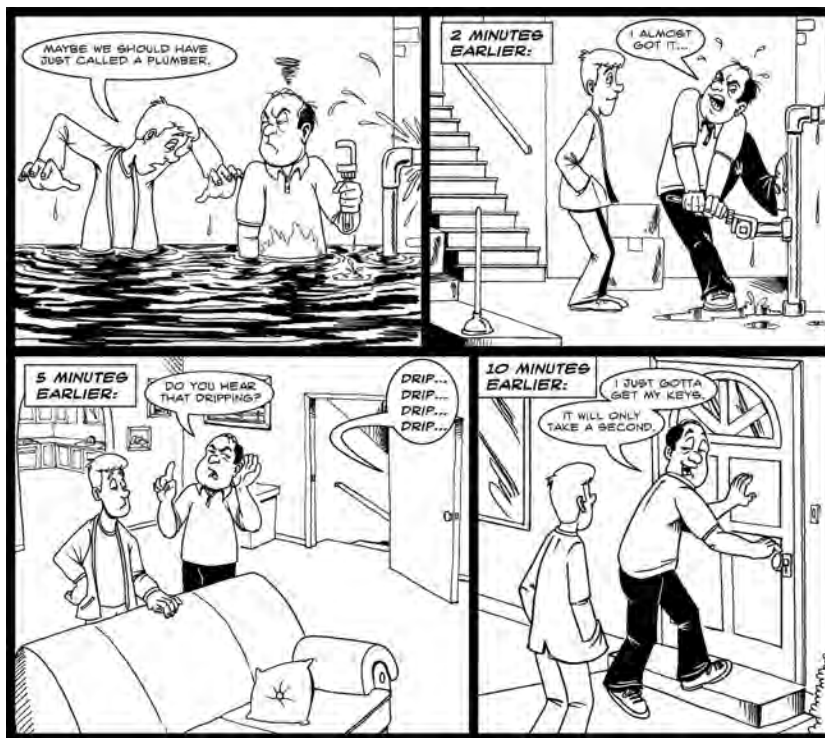
在第二部分中，我们讨论了如何更改程序的“循环和逻辑”以使您的代码更具可读性。我们描述了几种需要以较小的方式更改程序结构的技巧。

在这一部分中，我们将讨论您可以在函数级别对代码进行更大的更改。具体来说，我们将介绍三种重组代码的方法：

- 提取与程序主要目标无关的“无关子问题”。
- 重新安排你的代码，使其一次只执行一项任务。
- 首先用文字描述你的代码，并使用此描述来帮助你找到更简洁的解决方案。

最后，我们将讨论你可以完全删除代码或避免编写代码的情况——这是使代码易于理解的最好方法。

提取无关的子问题



工程就是将大问题分解成小问题，并将这些问题的解决方案重新组合在一起。将此原则应用于代码可以使其更健壮且更易于阅读。

本章的建议是积极地识别和提取无关的子问题。以下是我们的意思：

1. 查看给定的函数或代码块，并问自己，“此代码的高级目标是什么？”
2. 对于每一行代码，问：“它是否直接朝着该目标努力？还是它正在解决满足该目标所需的无关子问题？”
3. 如果有足够多的行在解决一个不相关的子问题，将该代码提取到一个单独的函数中。

将代码提取到单独的函数中是你可能每天都在做的事情。但对于本章，我们决定专注于提取*不相关的子问题*的具体情况，其中提取的代码完全不知道它为什么被调用。

正如你将看到的，这是一个很容易应用但可以大大改进你的代码的技术。然而，由于某种原因，许多程序员没有充分利用这种技术。诀窍是积极寻找这些不相关的子问题。

在本章中，我们将通过各种示例来说明这种技术在您可能遇到的不同情况下的应用。

入门示例：findClosestLocation()

以下 JavaScript 代码的高级目标是，*找到最接近给定点的位置*（不要陷入我们用斜体表示的高级几何学的泥潭）：

```
// 返回“array”中哪个元素最接近给定的纬度/经度。
// 将地球建模为一个完美的球体。
var findClosestLocation = function (lat, lng, array) {
    var closest;
    var closest_dist = Number.MAX_VALUE;
    for (var i = 0; i < array.length; i += 1) {
        // 将两个点都转换为弧度。
        var lat_rad = radians(纬度);
        var lng_rad = radians(经度);
        var lat2_rad = radians(array[i].latitude);
        var lng2_rad = radians(array[i].longitude);

        // 使用“球面余弦定律”公式。
        var dist = Math.acos(Math.sin(lat_rad) * Math.sin(lat2_rad) +
            Math.cos(lat_rad) * Math.cos(lat2_rad) *
            Math.cos(lng2_rad - lng_rad));

        if (dist < closest_dist) {
            closest = array[i];
        }
    }
}
```

```

        closest_dist = dist;
    }
    return closest;
};

```

循环中的大部分代码都在处理一个不相关的子问题：*计算两个经纬度点之间的球面距离*。因为太多的代码，所以将其提取到一个单独的spherical_distance函数中是有意义的：

```

var spherical_distance = function (lat1, lng1, lat2, lng2) {
    var lat1_rad = radians(lat1);
    var lng1_rad = radians(lng1);
    var lat2_rad = radians(lat2);
    var lng2_rad = radians(lng2);

    // 使用“球面余弦定律”公式。
    return Math.acos(Math.sin(lat1_rad) * Math.sin(lat2_rad) +
        Math.cos(lat1_rad) * Math.cos(lat2_rad) *
        Math.cos(lng2_rad - lng1_rad));
};

```

现在剩下的代码变成：

```

var findClosestLocation = function (lat, lng, array) {
    var closest;
    var closest_dist = Number.MAX_VALUE;
    for (var i = 0; i < array.length; i += 1) {
        var dist = 球面距离(lat, lng, array[i].latitude, array[i].longitude);
        if (dist < closest_dist) {
            closest = array[i];
            closest_dist = dist;
        }
    }
    return closest;
};

```

这段代码更具可读性，因为读者可以专注于高层次的目标，而不会被密集的几何方程式分散注意力。

作为额外的奖励，球面距离() 将更容易单独测试。而且球面距离() 是将来可以重用的函数类型。这就是为什么它是一个“不相关”的子问题——它完全是独立的，并且不知道应用程序是如何使用它的。

纯实用代码

大多数程序都有一组核心的基本任务，例如操作字符串、使用哈希表以及读取/写入文件。

通常，这些“基本实用程序”由编程语言中的内置库实现。例如，如果您想读取文件的全部内容，在PHP中您可以调用

`file_get_contents("filename")`，或者在Python中，您可以执行 `open("filename").read()`。

但有时你必须自己填补空白。例如，在C++中，没有简洁的方法来读取整个文件。相反，你不可避免地会写出这样的代码：

```
ifstream file(file_name);

// 计算文件的大小，并分配一个该大小的缓冲区。
file.seekg(0, ios::end);
const int file_size = file.tellg();
char* file_buf = new char [file_size];

// 将整个文件读入缓冲区。
file.seekg(0, ios::beg);
file.read(file_buf, file_size);
file.close();

...
```

这是一个经典的不相关子问题的例子，应该提取到一个新的函数中，比如 `ReadFileToString()`。现在，你的代码库的其余部分可以表现得好像C++确实有一个 `ReadFileToString()` 函数一样。

一般来说，如果你发现自己在想，“我希望我们的库有一个 `xyz()` 函数”，那就去编写它吧！（假设它还不存在。）随着时间的推移，你将建立一个不错的可以在各个项目中使用的实用代码集合。

其他通用代码

在调试JavaScript时，程序员经常使用 `alert()` 来弹出一个消息框，向程序员显示一些信息，这是Web版本的“`printf()` 调试”。例如，以下函数调用使用Ajax将数据提交到服务器，然后显示从服务器返回的字典：

```
ajax_post({
  url: 'http://example.com/submit',
  data: data,
  on_success: function (response_data) {
    var str = "";
    for (var key in response_data) {
      str += " " + key + " = " + response_data[key] + " ";
    }

    alert(str + "");

    // 继续处理 'response_data' ...
  }
});
```

这段代码的高级目标是，*向服务器发出 Ajax 调用，并处理响应*。但是很多代码都在解决不相关的子问题，*美化打印字典*。很容易将该代码提取到一个像 `format_pretty(obj)` 这样的函数中：

```
var format_pretty = function (obj) {
    var str = "{";
    for (var key in obj) {
        str += " " + key + " = " + obj[key] + ",";
    }

    return str + "}";
};
```

意外的好处

有很多理由说明提取 `format_pretty()` 是一个好主意。它使调用代码更简单，并且 `format_pretty()` 是一个方便的函数。

但还有一个不太明显的重要原因：当代码独立存在时，改进 `format_pretty()` 更容易。当您处理一个较小的独立函数时，添加功能、提高可靠性、处理边缘情况等都会感觉更容易。

以下是一些 `format_pretty(obj)` 无法处理的情况：

- 它期望 `obj` 是一个对象。如果它是一个普通字符串（或 `undefined`），当前代码将抛出一个异常。
- 它期望 `obj` 的每个值都是一个简单类型。如果它包含嵌套对象，当前代码将它们显示为 `[object Object]`，这不是很美观。

在我们把 `format_pretty()` 分离成它自己的函数之前，进行所有这些改进会感觉像很多工作。（事实上，在没有单独函数的情况下，递归打印嵌套对象非常困难。）

但是现在，添加此功能很容易。以下是改进后的代码的样子：

```
var format_pretty = function (obj, indent) {
    // 处理 null、undefined、字符串和非对象。
    如果 (obj === null) 返回 "null";
    如果 (obj === undefined) 返回 "undefined";
    如果 (typeof obj === "string") 返回 '"' + obj + '"';
    如果 (typeof obj !== "object") 返回 String(obj);

    如果 (indent === undefined) indent = "";

    // 处理（非空）对象。
    var str = "{";
    for (var key in obj) {
        str += indent + " " + key + " = "; str += f
        ormat_pretty(obj[key], indent + " ") + ",";
    }

    return str + indent + "}";
};
```

这涵盖了之前列出的缺点，并产生如下输出：

```
{
  key1 = 1
  key2 = true
  key3 = 未定义
  key4 = null
  key5 = {
    key5a = {
      key5a1 = "你好世界"
    }
  }
}
```

创建大量通用代码

函数 `ReadFileToString()` 和 `format_pretty()` 是不相关的子问题的很好的例子。它们非常基础且应用广泛，因此很可能在项目中重复使用。代码库通常有一个特殊的目录来存放这样的代码（例如，`util/`），以便它可以轻松共享。

通用代码非常棒，因为它与你的项目的其余部分完全分离。这样的代码更容易开发、更容易测试，也更容易理解。如果你的所有代码都能像这样就好了！

想想你使用的许多强大的库和系统，例如 SQL 数据库、JavaScript 库和 HTML 模板系统。你无需担心它们的内部结构——这些代码库与你的项目完全隔离。因此，你的项目的代码库保持很小。

你可以将项目的更多部分分解为独立的库，这样越好，因为其余的代码将更小，更容易思考。

这是自顶向下还是自底向上的编程？

自顶向下编程是一种风格，其中首先设计最高级别的模块和函数，然后根据需要进行较低级别的函数以支持它们。

自下而上的编程试图首先预测并解决所有子问题，然后使用这些部分构建更高级别的组件。

本章并不提倡一种方法优于另一种方法。大多数编程都涉及两者的结合。重要的是最终结果：子问题被移除并单独处理。

项目特定功能

理想情况下，您提取的子问题应该是完全与项目无关的。但即使不是，也没关系。分解子问题仍然有效。

这是一个来自商业评论网站的例子。这段 Python 代码创建了一个新的 `Business` 对象并设置其 `name`, `url` 和 `date_created`

```
business = Business()
business.name = request.POST["name"]

url_path_name = business.name.lower()
url_path_name = re.sub(r"[\.]", "", url_path_name)
url_path_name = re.sub(r"^[a-z0-9]+", "-", url_path_name)
url_path_name = url_path_name.strip("-")
business.url = "/biz/" + url_path_name

business.date_created = datetime.datetime.utcnow()
business.save_to_database()
```

The url is supposed to be a “clean” version of the name. For example, if the name is “A.C. Joe’s Tire & Smog, Inc.,” the url will be “/biz/ac-joes-tire-smog-inc”.

这段代码中不相关的子问题是：*将名称转换为有效的 URL*。我们可以很容易地提取这段代码。在这样做的时候，我们还可以预编译正则表达式（并给它们可读的名称）：

```
CHARS_TO_REMOVE = re.compile(r"[\.]+")
CHARS_TO_DASH = re.compile(r"^[a-z0-9]+")

def make_url_friendly(text):
    text = text.lower()
    text = CHARS_TO_REMOVE.sub("", text)
    text = CHARS_TO_DASH.sub('-', text)
    return text.strip("-")
```

现在，原始代码具有更“规则”的模式：

```
business = Business()
business.name = request.POST["name"]
business.url = "/biz/" + make_url_friendly(business.name)
business.date_created = datetime.datetime.utcnow()
business.save_to_database()
```

这段代码读起来轻松多了，因为你不会被正则表达式和深入的字符串操作分心。

你应该把 `make_url_friendly` 的代码放在哪里？它看起来像一个相当通用的函数，所以把它放在一个单独的 `util/` 目录下可能更有意义。另一方面，这些正则表达式是根据美国企业名称设计的，所以也许代码应该保留在它使用的同一个文件中。实际上这并不重要，你可以稍后轻松地移动定义。更重要的是 `make_url_friendly` 已经被提取出来了。

简化现有接口

当一个库提供一个干净的接口时，每个人都喜欢——这个接口只需要很少的参数，不需要太多的设置，而且通常只需要很少的努力就可以使用。它使你的代码看起来很优雅：简单而强大。

但是，如果您使用的接口不干净，您仍然可以创建自己的“包装”函数。

例如，在 JavaScript 中处理浏览器 cookie 远非理想。从概念上讲，cookie 是一组名称/值对。但是浏览器提供的接口呈现一个单一的 `document.cookie` 字符串，其语法为：

```
name1=value1; name2=value2; ...
```

要找到您想要的 cookie，您必须自己解析这个巨大的字符串。这是一个读取名为“max_results”的 cookie 值的代码示例：

```
var max_results;
var cookies = document.cookie.split(';');
for (var i = 0; i < cookies.length; i++) {
    var c = cookies[i];
    c = c.replace(/^\[ ]+/, ''); // 删除前导空格
    if (c.indexOf("max_results=") === 0)
        max_results = Number(c.substring(12, c.length));
}
```

哇，这真是一些丑陋的代码。显然，有一个 `get_cookie()` 函数正在等待被创建，以便我们可以直接写：

```
var max_results = Number(get_cookie("max_results"));
```

创建或更改 cookie 值甚至更奇怪。您必须将 `document.cookie` 设置为一个具有精确语法的值：

```
document.cookie = "max_results=50; expires=Wed, 1 Jan 2020 20:53:47 UTC; path=/";
```

该语句看起来会覆盖所有其他现有 cookie，但（神奇地）它不会！

一个更理想的设置 cookie 的接口应该是这样的：

```
set_cookie(name, value, days_to_expire);
```

删除 cookie 也很不直观：您必须将 cookie 设置为在过去过期。相反，一个理想的接口应该是：

```
delete_cookie(name);
```

这里的教训是，您永远不应该满足于一个不太理想的接口。您总是可以创建自己的包装函数来隐藏您所困扰的接口的丑陋细节。

根据您的需求重塑接口

程序中的很多代码只是为了支持其他代码——例如，设置函数的输入或后处理输出。这种“胶水”代码通常与程序的真正逻辑无关。像这样的平凡代码非常适合提取到单独的函数中。

例如，假设你有一个包含敏感用户信息（如用户名和密码）的 Python 字典，如下所示：

```
{ "username": "...", "password": ".."} 并且你需要将所有这些信息放入一个 URL 中。
因为它是敏感的，你决定首先使用 Cipher 类加密字典。
```

但是 Cipher 期望一个字节字符串作为输入，而不是一个字典。并且 Cipher 返回一个字节字符串，但我们需要一些 URL 安全的东西。Cipher 还需要一些额外的参数，并且使用起来非常麻烦。

一个简单的任务变成了很多粘合代码：

```
user_info = { "username": "...", "password": "..."}
user_str = json.dumps(user_info)
cipher = Cipher("aes_128_cbc", key=PRIVATE_KEY, init_vector=INIT_VECTOR, op=ENCODE)
encrypted_bytes = cipher.update(user_str)
encrypted_bytes += cipher.final() # 刷新当前的 128 位块
url = "http://example.com/?user_info=" + base64.urlsafe_b64encode(encrypted_bytes)
...
```

即使我们正在解决的问题是 *将用户的信息加密到 URL 中*，大部分代码只是在做 *将这个 Python 对象加密成一个 URL 友好的字符串*。很容易提取那个子问题：

```
def url_safe_encrypt(obj):
    obj_str = json.dumps(obj)
    cipher = Cipher("aes_128_cbc", key=PRIVATE_KEY, init_vector=INIT_VECTOR, op=ENCODE)
    encrypted_bytes = cipher.update(obj_str)
    encrypted_bytes += cipher.final() # 刷新当前的 128 位块
    return base64.urlsafe_b64encode(encrypted_bytes)
```

然后，执行程序的 *真实* 逻辑的结果代码很简单：

```
user_info = { "username": "...", "password": "..."}
url = "http://example.com/?user_info=" + url_safe_encrypt(user_info)
```

做得太过分

正如我们在本章开头所说，我们的目标是“*积极* 识别和提取不相关的子问题。”我们说“*积极*”，因为大多数程序员不够积极。但也有可能过于兴奋，做得太过火。

例如，前一节中的代码可以进一步分解，
像这样：

```
user_info = { "username": "...", "password": "..." }
url = "http://example.com/?user_info=" + url_safe_encrypt_obj(user_info)

def url_safe_encrypt_obj(obj):
    obj_str = json.dumps(obj)
    return url_safe_encrypt_str(obj_str)

def url_safe_encrypt_str(data):
    encrypted_bytes = encrypt(data)
    return base64.urlsafe_b64encode(encrypted_bytes)

def encrypt(data):
    cipher = make_cipher()
    encrypted_bytes = cipher.update(data)
    encrypted_bytes += cipher.final() # 冲刷掉所有剩余的字节
    return encrypted_bytes

def make_cipher():
    return Cipher("aes_128_cbc", key=PRIVATE_KEY, init_vector=INIT_VECTOR, op=ENCODE)
```

引入所有这些小函数实际上会损害可读性，因为读者需要跟踪更多的内容，并且跟踪执行路径需要跳来跳去。

向代码中添加新函数会带来很小（但切实存在）的可读性成本。在前一种情况下，没有任何收益可以抵消这种成本。如果项目的其他部分需要这些较小的函数，那么添加它们可能是有意义的。但在那之前，没有必要。

总结

思考本章的一个简单方法是将通用代码与项目特定代码分开。事实证明，大多数代码都是通用的。通过构建大量的库和辅助函数来解决一般问题，剩下的将是使您的程序独一无二的一小部分核心。

这种技术的主要原因是它可以让程序员专注于更小、定义明确的问题，这些问题与项目的其余部分是分离的。因此，这些子问题的解决方案往往更彻底和正确。您以后也可能能够重用它们。

延伸阅读

Martin Fowler的 *重构：改进现有代码的设计* (Fowler et al., Addison-Wesley Professional, 1999) 描述了重构的“提取方法”，并列出了许多其他重构代码的方法。

Kent Beck的 *Smalltalk 最佳实践模式* (Prentice Hall, 1996) 描述了“组合方法模式”，其中列出了将代码分解为许多小函数的若干原则。特别是，其中一个原则是“将单个方法中的所有操作保持在同一级别的抽象”。

这些想法与我们“提取不相关的子问题”的建议类似。我们在本章中讨论的是何时提取方法的一个简单而特殊的例子。

第十一章

一次只做一项任务

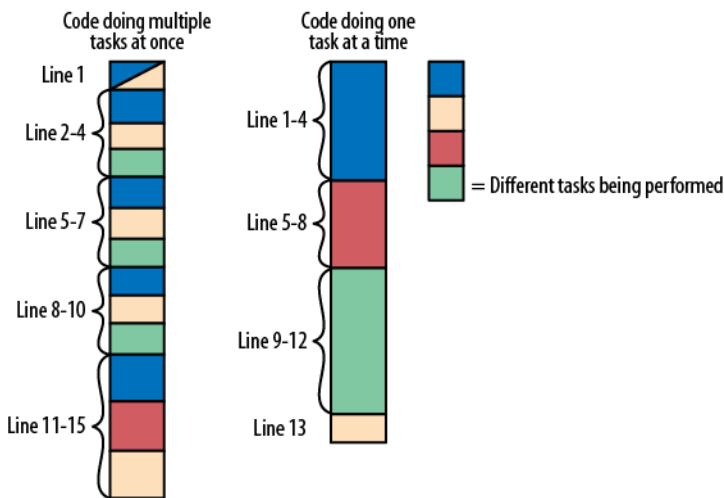


一次执行多项任务的代码更难理解。单个代码块可能同时初始化新对象、清理数据、解析输入和应用业务逻辑。如果所有这些代码都交织在一起，那么它将比每个“任务”单独启动和完成更难理解。

关键思想

代码的组织方式应使其一次只执行一项任务。

换句话说，本章是关于“整理”你的代码。下图说明了这个过程：左边显示了一段代码正在执行的各种任务，右边显示了相同的代码在经过组织后一次只执行一个任务。



你可能听说过“函数应该只做一件事”的建议。我们的建议类似，但并不总是关于函数边界。当然，将一个大型函数分解成多个较小的函数可能很好。但即使你不这样做，你仍然可以组织大型函数内部的代码，使其感觉像有独立的逻辑部分。

以下是我们用来使代码“一次执行一个任务”的过程：

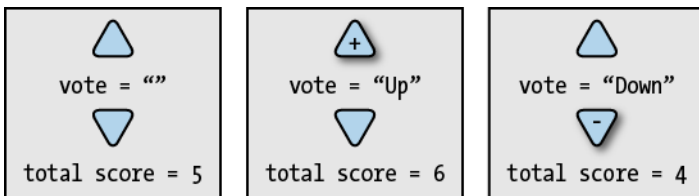
1. 列出你的代码正在执行的所有“任务”。我们非常宽泛地使用“任务”这个词——它可以小到“确保这个对象是有效的”，也可以模糊到“遍历树中的每个节点”。
2. 尽量将这些任务尽可能地分离到不同的函数或至少不同的代码段中。

在本章中，我们将向你展示一些如何做到这一点的例子。

任务可以很小

假设博客上有一个投票小部件，用户可以对评论投“赞成”或“反对”票。评论的总得分是所有投票的总和：每个“赞成”票+1，每个“反对”票-1。

以下是用户的投票可以处于的三种状态，以及它将如何影响总得分：



当用户点击其中一个按钮（进行/更改她的投票）时，将调用以下 JavaScript：

```
vote_changed(old_vote, new_vote); // 每个投票都是 "Up", "Down", 或 ""
```

此函数更新总得分，适用于old_vote/new_vote的所有组合：

```
var vote_changed = function (old_vote, new_vote) {  
    var score = get_score();  
  
    if (new_vote !== old_vote) {  
        if (new_vote === 'Up') {  
            score += (old_vote === 'Down' ? 2 : 1);  
        } else if (new_vote === 'Down') {  
            score -= (old_vote === 'Up' ? 2 : 1);  
        } else if (new_vote === '') {  
            score += (old_vote === 'Up' ? -1 : 1);  
        }  
    }  
  
    set_score(score);  
};
```

即使代码很短，它也做了很多事情。有很多错综复杂的细节，而且很难一眼看出是否存在任何差一错误、拼写错误或其他错误。

这段代码可能看起来只做了一件事（更新分数），但实际上同时执行了两个任务：

1. old_vote和new_vote正在被“解析”为数值。
2. score正在被更新。

我们可以通过分别解决每个任务来使代码更易于阅读。以下代码解决了第一个任务，即将投票解析为数值：

```
var vote_value = function (vote) {  
    如果 (vote === 'Up') {
```

```

        return +1;
    }
    如果 (vote === 'Down') {
        return -1;
    }
    return 0;
};

```

现在，代码的其余部分可以解决第二个任务，更新 分数：

```

var vote_changed = function (old_vote, new_vote) {
    var score = get_score();

    分数 -= vote_value(old_vote); // 移除旧的投票
    分数 += vote_value(new_vote); // 添加新的投票

    set_score(score);
};

```

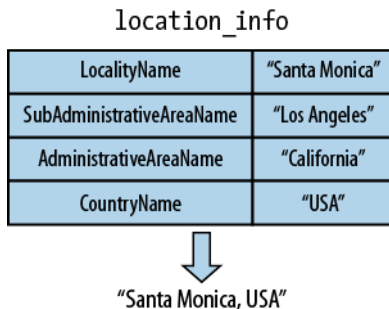
正如你所看到的，这个版本的代码需要更少的脑力来让你相信它可以工作。这是使代码“易于理解”的重要组成部分。

从对象中提取值

我们曾经有一些 JavaScript 将用户的位置格式化为友好的字符串 “城市，”

“国家” 比如 “美国圣莫尼卡” 或 “法国巴黎”。我们得到了一个 `location_info` 字典，其中包含大量结构化信息。我们所要做的就是从所有字段中选择一个 “城市” 和一个 “国家”，并将它们连接在一起。

以下图示显示了输入/输出示例：

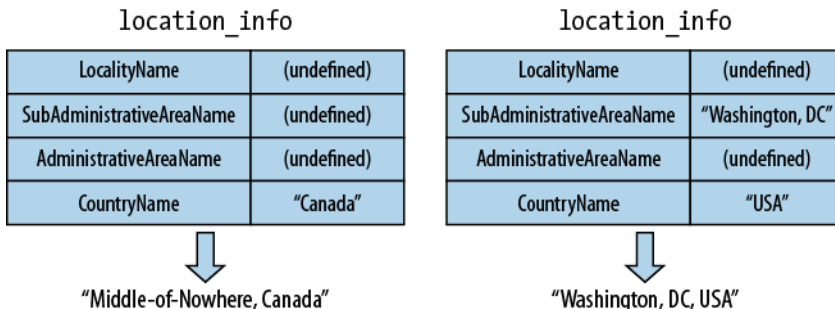


到目前为止，这似乎很容易，但棘手的部分是这些四个值中的任何一个或全部都可能缺失。这是我们处理这个问题的方法：

- 在选择 “城市” 时，如果可用，我们更倾向于使用 “LocalityName”（城市/城镇），然后是 “SubAdministrativeAreaName”（较大的城市/县），然后是 “AdministrativeAreaName”（州/地区）。

- 如果这三个都缺失，“城市”会被亲切地赋予默认值“荒郊野外”。
- 如果“CountryName”缺失，则使用“地球”作为默认值。

下图显示了处理缺失值的两个示例。



这是我们编写的用于实现此任务的代码：

```
var place = location_info["LocalityName"]; // 例如 "Santa Monica"
if (!place) {
    place = location_info["SubAdministrativeAreaName"]; // 例如 "Los Angeles"
}
if (!place) {
    place = location_info["AdministrativeAreaName"]; // 例如 "California"
}
if (!place) {
    place = "荒郊野外";
}
if (location_info["CountryName"]) {
    place += ", " + location_info["CountryName"]; // 例如 "USA"
} else {
    place += ", 地球";
}

return place;
```

当然，这有点乱，但它完成了工作。

但几天后，我们需要改进功能：对于美国的位置，

我们希望显示 州 而不是国家（如果可能）。因此，它将返回“圣莫尼卡，加利福尼亚”，而不是“圣莫尼卡，USA”。

将此功能添加到之前的代码会使其更加难看。

应用“一次只做一件事”

我们没有屈服于这段代码，而是停下来意识到它已经在同时执行多个任务：

1. 从字典 `location_info` 中提取值
2. 按照“城市”的偏好顺序进行查找，如果找不到任何内容，则默认为“荒郊野外”
3. 获取“国家”，如果没有，则使用“地球”
4. 更新 地点

因此，我们重写了原始代码，以独立解决这些任务。

第一个任务（从 `location_info` 中提取值）很容易单独解决：

```
var 城镇 = location_info["LocalityName"]; // 例如 "Santa Monica"
var 城市 = location_info["SubAdministrativeAreaName"]; // 例如 "Los Angeles"
var 状态 = location_info["AdministrativeAreaName"]; // 例如 "CA"
var 国家 = location_info["CountryName"]; // 例如 "USA"
```

在这一点上，我们完成了使用 `location_info` 并且不必记住那些又长又不直观的键。相反，我们有四个简单的变量可以使用。

接下来，我们必须弄清楚返回值的“后半部分”是什么：

```
// 从默认值开始，并不断用最具体的值覆盖。
var second_half = "地球";
if (country) {
    second_half = country;
}
if (state && country === "USA") {
    second_half = state;
}
```

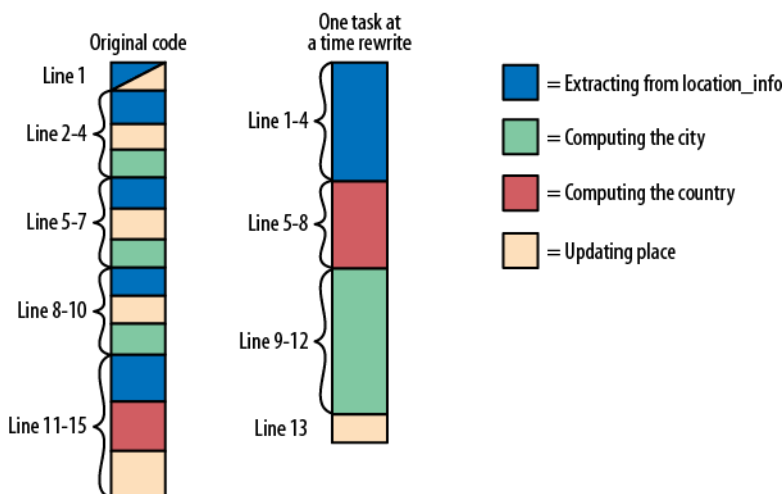
同样，我们可以算出“前半部分”：

```
var first_half = "穷乡僻壤";
if (state && country !== "USA") {
    first_half = state;
}
if (city) {
    first_half = city;
}
if (town) {
    first_half = town;
}
```

最后，我们将信息拼凑在一起：

```
return first_half + ", " + second_half;
```

本章开头的“碎片整理”图实际上是原始解决方案和这个新版本的表示。这是同一张图，填充了更多细节：



正如你所看到的，第二个解决方案中的四个任务已经被碎片整理成不同的区域。

另一种方法

在重构代码时，通常有多种方法可以实现，这种情况也不例外。一旦你分离了一些任务，代码就会变得更容易理解，你可能会想出更好的重构方法。

例如，之前的一系列 if 语句需要仔细阅读才能知道每个 case 是否正常工作。实际上，在该代码中同时进行了两个子任务：

1. 遍历变量列表，并选择最可用的首选变量。
2. 使用不同的列表，具体取决于国家是否为“USA”。

回顾一下，你可以看到之前的代码将“if USA”逻辑与其余逻辑交织在一起。相反，我们可以分别处理美国和非美国的情况：

```
var first_half, second_half;

if (country === "USA") {
  first_half = town || city || "Middle-of-Nowhere";
  second_half = state || "USA";
} else {
  first_half = town || city || state || "Middle-of-Nowhere";
  second_half = country || "Planet Earth";
}

返回 first_half + ", " + second_half;
```

如果你不熟悉 JavaScript，`a || b || c` 是一种惯用表达，其结果为第一个“真值”（在本例中，是一个已定义、非空的字符串）。这段代码的好处是它非常

易于检查偏好列表并更新它。大多数的 if 语句已被清除，业务逻辑由更少的代码行表示。

一个更大的例子

在我们构建的一个网络爬虫系统中，一个名为 UpdateCounts() 的函数被调用来递增在每个网页下载后，各种统计数据：

```
void UpdateCounts(HttpDownload hd) {
    counts["退出状态"][hd.exit_state()]++; // 例如 "SUCCESS" 或 "FAILURE"
    counts["Http Response"][hd.http_response()]++; // 例如 "404 NOT FOUND"
    counts["Content-Type" ][hd.content_type()]++; // 例如 "text/html"
}
```

好吧，这就是我们 *希望* 代码的样子！

实际上，HttpDownload 对象没有这里显示的任何方法。相反，HttpDownload 是一个非常庞大和复杂的类，包含许多嵌套类，我们必须自己找出这些值。更糟糕的是，有时这些值完全缺失——在这种情况下，我们只使用 "unknown" 作为默认值。

由于这一切，真正的代码非常混乱：

```
// 警告：请勿长时间直视此代码。
void UpdateCounts(HttpDownload hd) {
    // 如果可用，则找出退出状态。
    if (!hd.has_event_log() || !hd.event_log().has_exit_state()) {
        counts["Exit State"]["unknown"]++;
    } else {
        string state_str = ExitStateTypeName(hd.event_log().exit_state());
        counts["Exit State"][state_str]++;
    }

    // 如果根本没有 HTTP 标头，则对剩余元素使用 "unknown" 。
    if (!hd.has_http_headers()) {
        counts["Http Response"]["unknown"]++;
        counts["Content-Type"]["unknown"]++;
        return;
    }

    HttpHeaders headers = hd.http_headers();

    // 如果已知，则记录 HTTP 响应，否则记录 "unknown"
    if (!headers.has_response_code()) {
        counts["Http Response"]["unknown"]++;
    } else {
        string code = StringPrintf("%d", headers.response_code());
        counts["Http Response"][code]++;
    }

    // 如果已知，则记录 Content-Type，否则记录 "unknown"
    if (!headers.has_content_type()) {
        counts["Content-Type"]["unknown"]++;
    }
}
```

```

    } else {
        string content_type = ContentTypeMime(headers.content_type());
        counts["Content-Type"][content_type]++;
    }
}

```

正如你所看到的，这里有很多代码，很多逻辑，甚至还有一些重复的代码行。这段代码读起来并不有趣。

特别是，这段代码在不同的任务之间来回切换。以下是贯穿代码的不同任务：

1. 使用 "unknown" 作为每个键的默认值
2. 检测 `HttpDownload` 的成员是否缺失
3. 提取值并将其转换为字符串
4. 更新 `counts[]`

我们可以通过将其中一些任务分离到代码中的不同区域来改进代码：

```

void UpdateCounts(HttpDownload hd) {
    // 任务：为我们要提取的每个值定义默认值
    string exit_state = "unknown";
    string http_response = "unknown";
    string content_type = "unknown";

    // 任务：尝试从 HttpDownload 中逐个提取每个值
    if (hd.has_event_log() && hd.event_log().has_exit_state()) {
        exit_state = ExitStateTypeName(hd.event_log().exit_state());
    }
    如果 (hd.has_http_headers() && hd.http_headers().has_response_code()) {
        http_response = StringPrintf("%d", hd.http_headers().response_code());
    }
    如果 (hd.has_http_headers() && hd.http_headers().has_content_type()) {
        content_type = ContentTypeMime(hd.http_headers().content_type());
    }

    // 任务：更新 counts[]
    counts["退出状态"][exit_state]++;
    counts["Http Response"][http_response]++;
    counts["Content-Type"][content_type]++;
}

```

正如你所看到的，代码有三个独立的区域，其目标如下：

1. 为我们感兴趣的三个键定义默认值。
2. 提取每个键的值（如果可用），并将它们转换为字符串。
3. 为每个键/值更新 `counts[]`。

这些区域的优点在于它们彼此隔离——当您阅读一个区域时，您不需要考虑其他区域。

请注意，虽然我们列出了四个任务，但我们只能分离出其中的三个。这完全没问题：你最初列出的任务只是一个起点。即使只分离出其中的一些任务，也能起到很大的帮助，就像这里一样。

进一步的改进

这个新版本的代码比最初的怪物有了显著的改进。请注意，我们甚至不需要创建其他函数来执行此清理。正如我们之前提到的，“一次一个任务”的想法可以帮助你清理代码，而不管函数边界如何。

但是，我们也可以通过引入三个辅助函数来改进此代码：

```
void UpdateCounts(HttpDownload hd) {
    counts["Exit State"][ExitState(hd)]++;
    counts["Http Response"][HttpResponse(hd)]++;
    counts["Content-Type"][ContentType(hd)]++;
}
```

这些函数将提取相应的值，或返回“unknown”。例如：

```
string ExitState(HttpDownload hd) {
    if (hd.has_event_log() && hd.event_log().has_exit_state()) {
        return ExitStateTypeName(hd.event_log().exit_state());
    } else {
        return "unknown";
    }
}
```

请注意，这种替代解决方案甚至没有定义任何变量！正如我们在第 9 章“变量和可读性”中提到的，保存中间结果的变量通常可以完全消除。

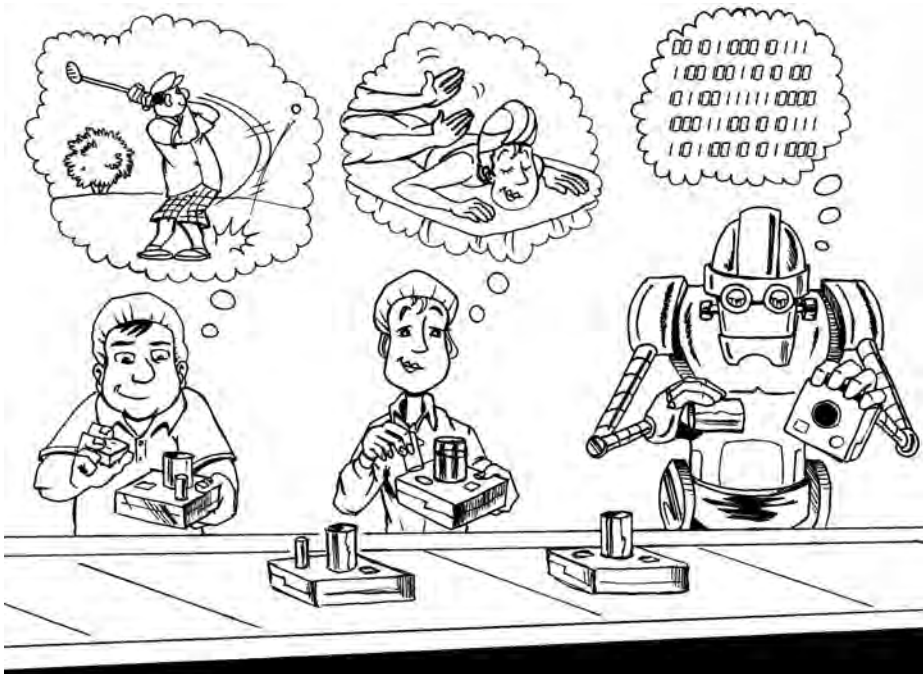
在这个解决方案中，我们只是以不同的方向“切分”了问题。两种解决方案都具有很高的可读性，因为它们要求读者一次只考虑一个任务。

总结

本章阐述了一种组织代码的简单技术：一次只做一项任务。

如果你有难以阅读的代码，试着列出它正在做的所有任务。其中一些任务可能很容易变成单独的函数（或类）。其他的可能只是变成逻辑上的“段落”在一个函数中。你如何分离这些任务的具体细节并不像它们被分离的事实那么重要。困难的部分是准确地描述你的程序正在做的所有小事情。

将想法转化为代码



除非你能向你的祖母解释清楚，否则你并不真正理解某件事。

—阿尔伯特·爱因斯坦

当向某人解释一个复杂的想法时，很容易用所有的小细节让他们感到困惑。能够用“简单的英语”解释一个想法是一项很有价值的技能，这样比你知识少的人也能理解。这需要将一个想法提炼成最重要的概念。这样做不仅能帮助对方理解，还能帮助你更清楚地思考自己的想法。

在向读者“展示”代码时，也应使用同样的技巧。我们认为源代码是解释程序正在做什么的主要方式。因此，代码应该用“通俗易懂的英语”编写。

在本章中，我们将使用一个简单的流程，它可以帮助你更清晰地编写代码：

1. 用通俗易懂的英语描述代码需要做什么，就像你对同事说的那样。
2. 注意描述中使用的关键词和短语。
3. 编写代码以匹配此描述。

清晰地描述逻辑

这是一个来自 PHP 网页的代码片段。此代码位于安全页面的顶部。它检查用户是否有权查看该页面，如果没有，则立即返回一个页面，告诉用户她未获得授权：

```
$is_admin = is_admin_request();
if ($document) {
    if (!$is_admin && ($document['username'] != $_SESSION['username'])) {
        return not_authorized();
    }
} else {
    if (!$is_admin) {
        return not_authorized();
    }
}

// continue rendering the page ...
```

这段代码中有很多逻辑。正如你在第二部分“简化循环和逻辑”中看到的，像这样的大型逻辑树不容易理解。这段代码中的逻辑可能可以简化，但如何简化呢？让我们首先用简单的英语描述一下这个逻辑：

您可以通过两种方式获得授权：

- 1) 您是管理员
- 2) 您拥有当前文档（如果存在）

否则，您未获得授权。

以下是受此描述启发的替代解决方案：

```
if (is_admin_request()) {  
    // 已授权  
} elseif ($document && ($document['username'] == $_SESSION['username'])) {  
    // 已授权  
} else {  
    return not_authorized();  
}  
  
// continue rendering the page ...
```

这个版本有点不寻常，因为它有两个空主体。但代码更小，并且逻辑更简单，因为没有否定。（之前的解决方案有三个“not”。）最重要的是，它更容易理解。

了解你的库会有帮助

我们曾经有一个网站，其中包含一个“提示框”，向用户显示有用的建议，例如：

提示：登录以查看您过去的查询。[给我看另一个提示!] _____

有几十条提示，所有这些提示都隐藏在HTML中：

```
<div id="tip-1" class="tip">提示：登录以查看您过去的查询。</div>  
<div id="tip-2" class="tip">提示：点击图片可以近距离观看。</div>  
...
```

当用户访问页面时，这些divs中的一个会随机变为可见，其余的保持隐藏。

如果点击“给我看另一个提示！”链接，它会循环到下一个提示。以下是使用jQuery JavaScript库实现该功能的一些代码：

```
var show_next_tip = function () {  
    var num_tips = $('tip').size();  
    var shown_tip = $('tip:visible');  
  
    var shown_tip_num = Number(shown_tip.attr('id').slice(4));  
    if (shown_tip_num === num_tips) {  
        $('#tip-1').show();  
    } else {  
        $('#tip-' + (shown_tip_num + 1)).show();  
    }  
    shown_tip.hide();  
};
```

这段代码还可以。但它可以做得更好。让我们首先用文字描述这段代码试图做什么：

查找当前可见的提示并隐藏它。
然后找到它之后的下一个提示并显示出来。
如果我们已经用完了所有提示，则循环回到第一个提示。

基于此描述，这是另一种解决方案：

```
var show_next_tip = function () {  
    var cur_tip = $('.tip:visible').hide(); // 找到当前可见的提示并隐藏它  
    var next_tip = cur_tip.next('.tip'); // 找到它之后的下一个提示  
    if (next_tip.size() === 0) { // 如果我们已经用完了所有提示，  
        next_tip = $('.tip:first'); // 循环回到第一个提示  
    }  
    next_tip.show(); // 显示新的提示  
};
```

此解决方案包含更少的代码行，并且不必直接操作整数。它更符合一个人对代码的思考方式。

在这种情况下，jQuery 有一个 `.next()` 方法可以帮助我们使用。编写简洁代码的一部分是了解你的库所提供的功能。

将此方法应用于更大的问题

之前的例子已经将我们的流程应用于小代码块。在下一个例子中，我们将把它应用到一个更大的函数。正如你将看到的，这种方法可以帮助你分解你的代码，帮助你识别可以分离的部分。

假设我们有一个记录股票购买的系统。每笔交易都有四个数据：

- 时间（购买的精确日期和时间）
- 股票代码（例如，GOOG）
- 价格（例如，600美元）
- 股票数量（例如，100）

由于某些奇怪的原因，数据分布在三个独立的数据库表中，如此处所示。在每个数据库中，时间是唯一的primary key（主键）。

time	ticker_symbol	time	price	time	number_of_shares
3:45	IBM	3:45	\$120	3:45	50
3:59	IBM	4:30	\$600	3:59	200
4:30	GOOG	5:00	\$25	4:10	75
5:20	AAPL	5:20	\$200	4:30	100
6:00	MSFT	6:00	\$25	5:20	80

现在我们需要编写一个程序将这三个表连接在一起（就像SQLJOIN操作会做的那样）。这一步应该很容易，因为这些行都按时间排序，但不幸的是有些行丢失了。你想要找到所有三个时间匹配的行，并忽略任何无法对齐的行，如上图所示。

这是一些Python代码，用于查找所有匹配的行：

```
def PrintStockTransactions():
    stock_iter = db_read("SELECT time, ticker_symbol FROM ...")
    price_iter = ...
    num_shares_iter = ...

    # 并行遍历3个表的所有行。
    while stock_iter and price_iter and num_shares_iter:
        stock_time = stock_iter.time
        price_time = price_iter.time
        num_shares_time = num_shares_iter.time

        # 如果所有3行的时间不相同，则跳过最旧的行
        # 注意：下面的“<=”不能只是“<”，以防有两个最旧的。
        if stock_time != price_time or stock_time != num_shares_time:
            if stock_time <= price_time and stock_time <= num_shares_time:
                stock_iter.NextRow()
            elif price_time <= stock_time and price_time <= num_shares_time:
                price_iter.NextRow()
            elif num_shares_time <= stock_time and num_shares_time <= price_time:
                num_shares_iter.NextRow()
        else:
            assert False # 不可能
            继续

    assert stock_time == price_time == num_shares_time

    # 打印对齐的行。
    print "@", stock_time,
        print stock_iter.ticker_symbol,
        print price_iter.price,
    print num_shares_iter.number_of_shares

    stock_iter.NextRow()
    price_iter.NextRow()
    num_shares_iter.NextRow()
```

这段示例代码可以工作，但是循环如何跳过不匹配的行有很多问题。

你脑海中可能会出现一些警告标志：这会错过任何行吗？

对于任何迭代器，它是否会读取到流的末尾？

那么我们如何才能使其更具可读性呢？

解决方案的英文描述

再次，让我们退一步，用简单的英语描述我们要做什么：

我们正在并行读取三个行迭代器。

每当行的时间不一致时，推进这些行，使它们对齐。

然后打印对齐的行，并再次推进这些行。

持续这样做，直到没有更多匹配的行留下。

回顾原始代码，最混乱的部分是处理“推进行使它们对齐”的块。为了更清楚地呈现代码，我们可以将所有这些混乱的逻辑提取到一个名为 AdvanceToMatchingTime() 的新函数中。

这是代码的新版本，使用了这个新函数：

```
def PrintStockTransactions():
    stock_iter = ...
    price_iter = ...
    num_shares_iter = ...

    while True:
        time = AdvanceToMatchingTime(stock_iter, price_iter, num_shares_iter)
        if time is None:
            return

    # 打印对齐的行。
    print "@", time,
        print stock_iter.ticker_symbol,
        print price_iter.price,
        print num_shares_iter.number_of_shares

    stock_iter.NextRow()
    price_iter.NextRow()
    num_shares_iter.NextRow()
```

正如你所看到的，这段代码更容易理解，因为我们隐藏了所有脏乱细节，将行对齐。

递归地应用该方法

很容易想象你会如何编写 AdvanceToMatchingTime()—在最坏的情况下，它会看起来非常类似于第一个版本中的那个丑陋的代码块：

```
def AdvanceToMatchingTime(stock_iter, price_iter, num_shares_iter):
    # 并行遍历3个表的所有行。
    while stock_iter and price_iter and num_shares_iter:
        stock_time = stock_iter.time
        price_time = price_iter.time
        num_shares_time = num_shares_iter.time

        # 如果所有 3 行的时间不相同，则跳过最旧的行
        if stock_time != price_time or stock_time != num_shares_time:
            if stock_time <= price_time and stock_time <= num_shares_time:
                stock_iter.NextRow()
            elif price_time <= stock_time and price_time <= num_shares_time:
                price_iter.NextRow()
            elif num_shares_time <= stock_time and num_shares_time <= price_time:
                num_shares_iter.NextRow()
        else:
            assert False # 不可能
    继续
```

```
assert stock_time == price_time == num_shares_time
return stock_time
```

但是，让我们通过将我们的方法应用于 `AdvanceToMatchingTime()` 来改进该代码。这是对这个函数需要做什么的描述：

查看每个当前行的时间：如果它们对齐，我们就完成了。
否则，推进任何“落后”的行。
持续这样做，直到行对齐（或者其中一个迭代器结束）。

这个描述比之前的代码清晰和优雅得多。需要注意的一件事是，该描述从未提及 `stock_iter` 或特定于我们问题的其他细节。这意味着我们也可以将变量重命名为更简单和更通用的名称。这是结果代码：

```
def AdvanceToMatchingTime(row_iter1, row_iter2, row_iter3):
    while row_iter1 and row_iter2 and row_iter3:
        t1 = row_iter1.time
        t2 = row_iter2.time
        t3 = row_iter3.time

        if t1 == t2 == t3:
            return t1

    tmax = max(t1, t2, t3)

    # 如果任何行“落后”，则前进它。
    # 最终，这个while循环会将它们全部对齐。
    if t1 < tmax: row_iter1.NextRow()
    if t2 < tmax: row_iter2.NextRow()
    if t3 < tmax: row_iter3.NextRow()

    return None # 找不到对齐
```

正如你所看到的，这段代码比以前清晰多了。该算法变得更简单，现在有了更少的棘手比较。我们使用了像 `t1` 这样的短名称，不再需要考虑涉及的具体数据库列。

总结

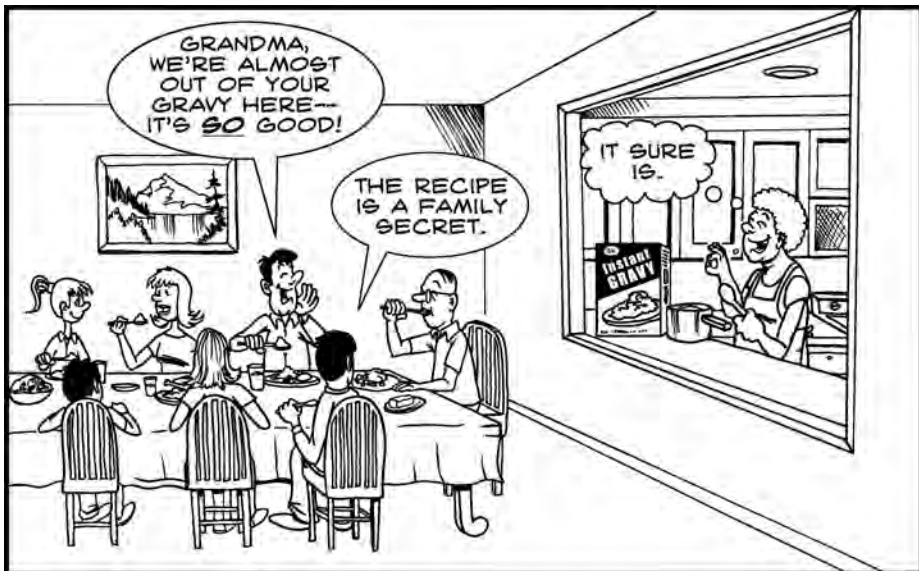
本章讨论了一种简单的技术，即用简单的英语描述你的程序，并使用该描述来帮助你编写更自然的代码。这种技术具有欺骗性简单，但非常强大。查看你的描述中使用的单词和短语也可以帮助你确定要分解哪些子问题。

但是，这种“用简单的英语表达事物”的过程不仅适用于编写代码。例如，一所大学的计算机实验室政策规定，当学生需要帮助调试他的程序时，他首先必须向房间角落里的一只专门的泰迪熊解释问题。令人惊讶的是，仅仅大声描述问题通常可以帮助学生找到解决方案。这种技术被称为“橡皮鸭调试法”。

另一种看待问题的方式是：如果你无法用语言描述问题或你的设计，可能缺少或未定义某些内容。用语言表达一个程序（或任何想法）可以真正地使其成形。

第十三章

编写更少的代码



知道何时不编写代码可能是程序员可以学到的最重要的技能。你编写的每一行代码都是必须经过测试和维护的一行代码。通过重用库或消除功能，你可以节省时间并保持代码库的精简和高效。

关键思想

最具可读性的代码就是没有代码。

不要费心实现那个功能——你不会需要它的

当你开始一个项目时，很自然地会感到兴奋，并想到所有你想要实现的很酷的功能。但是程序员往往会高估有多少功能是真正对他们的项目至关重要的。很多功能都没有完成或未使用，或者只是使应用程序复杂化。

程序员也倾向于低估实现一个功能需要多少努力。我们乐观地估计实现一个粗略的原型需要多长时间，但忘记了未来的维护、文档和添加到代码库中的“重量”需要花费多少额外的时间。

质疑并分解你的需求

并非所有程序都需要快速、100% 正确，并且能够处理每一种可能的输入。如果你真的仔细审查你的需求，有时你可以开辟一个更简单的问题，需要更少的代码。让我们看一些这样的例子。

示例：商店定位器

假设您正在为一个企业编写一个“商店定位器”。您认为您的需求是：

对于任何给定用户的纬度/经度，找到纬度/经度最接近的商店。

要100%正确地实现这一点，您需要处理：

- 当位置位于国际日期变更线的两侧时
- 当位置靠近北极或南极时
- 调整地球的曲率，因为“每英里的经度”会发生变化

处理所有这些情况需要相当多的代码。

但是，对于您的应用程序，德克萨斯州只有30家商店。在这个较小的区域中，列表中的三个问题并不那么重要。因此，您可以将您的需求降低为：

对于德克萨斯州附近的用户，找到（大约）德克萨斯州最近的商店。

解决这个问题更容易，因为你可以只遍历每个商店并计算纬度/经度之间的欧几里得距离。

示例：添加缓存

我们曾经有一个 Java 应用程序，经常从磁盘读取对象。应用程序的速度受到这些读取的限制，因此我们希望实现某种缓存。一个典型的读取序列如下所示：

```
读取对象 A
读取对象 A
读取对象 A
读取对象 B
读取对象 B
读取对象 C
读取对象 D
读取对象 D
```

正如你所看到的，有很多对同一对象的重复访问，所以缓存肯定会有帮助。

当面对这个问题时，我们的第一反应是使用一个丢弃最近最少使用的项目的缓存。我们的库中没有可用的缓存，所以我们必须自己实现它。但这并不是问题，因为我们以前已经实现过这样的数据结构（它涉及一个哈希表和一个单链表——总共大约 100 行代码）。

然而，我们注意到重复的访问总是在一行中。因此，我们没有实现 LRU 缓存，而是实现了一个单项缓存：

```
DiskObject lastUsed; // 类成员

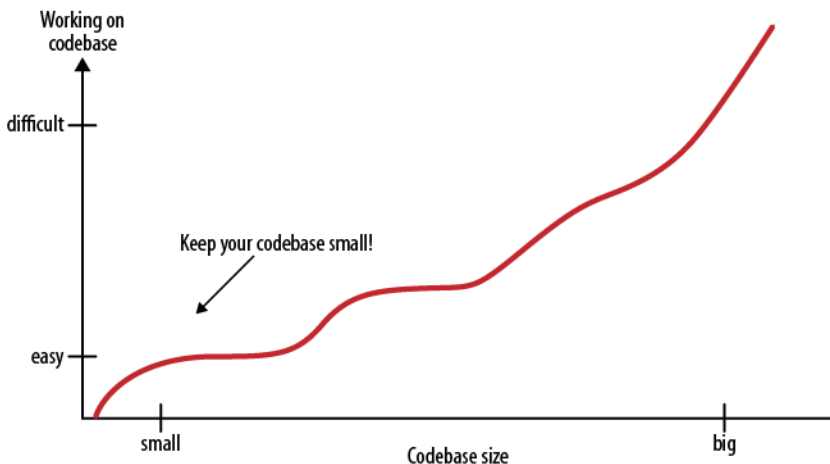
DiskObject lookUp(String key) {
    if (lastUsed == null || !lastUsed.key().equals(key)) {
        lastUsed = loadDiskObject(key);
    }

    return lastUsed;
}
```

这使我们获得了 90% 的好处，而无需太多编码，并且该程序也具有很小的内存占用。

“消除需求”和“解决更简单的问题”的好处怎么强调都不为过。需求常常以微妙的方式相互干扰。这意味着解决一半的问题可能只需要四分之一的编码工作。

保持你的代码库小巧



当你刚开始一个软件项目，只有一两个源文件时，一切都很棒。编译和运行代码很容易，修改也很容易，而且很容易记住每个函数或类在哪里定义。

然后，随着项目的增长，你的目录里充满了越来越多的源文件。很快你就需要多个目录来组织它们。记住哪些函数调用了哪些其他函数变得更加困难，而且追踪错误需要更多的工作。

最终，你会有大量的源代码分布在许多不同的目录中。项目非常庞大，没有人能完全理解它。添加新功能变得很痛苦，而且使用代码很麻烦和令人不快。

我们所描述的是宇宙的一条自然法则——随着任何协调系统的增长，将其粘合在一起所需的复杂性增长得更快。

最好的应对方法是尽可能保持你的代码库小而轻量级，即使随着你的项目增长。因此，你应该：

- 创建尽可能多的通用“实用”代码，以删除重复的代码。（参见第 10 章，[提取不相关的子问题](#)。）
- 删除未使用的代码或无用的功能。（参见下面的侧边栏。）
- 保持你的项目被分隔成不相关的子项目。
- 总的来说，要注意代码库的“重量”。保持其轻量和灵活。

移除未使用的代码

园丁经常修剪植物，以保持它们的生命力和生长。同样，修剪任何妨碍工作的未使用代码也是一个好主意。

一旦代码编写完成，程序员通常不愿删除它，因为它代表了大量实际工作。删除它意味着承认花在它上面的时间被浪费了。好吧，克服它！这是一个创意领域——摄影师、作家和电影制作人也不会保留他们所有的作品。

删除孤立的函数很容易，但有时“未使用的代码”实际上贯穿于你的整个项目，你却不知道。这里有一些例子：

- 你最初设计你的系统来处理国际文件名，现在代码中到处都是转换代码。然而，这些代码并不完全起作用，而且你的应用程序无论如何都不会与国际文件名一起使用。

为什么不移除这个功能呢？

- 您希望您的程序即使在系统内存耗尽时也能正常工作，因此您编写了大量巧妙的逻辑来尝试从内存不足的情况中恢复。这曾经是个好主意，但实际上，当系统内存耗尽时，您的程序只会变成一个不稳定的僵尸——所有核心功能都不可用，而且只需一次鼠标点击就会崩溃。

为什么不直接用一句简单的“系统内存不足，抱歉”来终止程序，然后删除所有这些内存不足的代码呢？

熟悉你周围的库

很多时候，程序员只是没有意识到现有的库可以解决他们的问题。

或者有时他们忘记了库能做什么。重要的是了解你的库代码的功能，这样你才能利用它。

这里有一个小小的建议：时不时地，花15分钟阅读你的标准库中所有函数/模块/类型的名称。这些包括C++标准模板库（STL）、Java API、内置的Python模块等。

目标不是记住整个库。只是为了了解有什么可用的，这样下次你编写新代码时，你会想，“等等，这听起来像我在API中看到的.....”我们相信预先做这项工作会很快得到回报，因为你更倾向于首先使用这些库。

示例：Python中的列表和集合

假设你在Python中有一个列表（如`[2,1,2]`），并且你想要一个包含唯一元素的列表（在本例中，`[2,1]`）。你可以使用字典来实现这个任务，字典有一个保证唯一的键列表：

```
def unique(elements):
    temp = {}
    for element in elements:
        temp[element] = None # 这个值无关紧要。
    返回 temp.keys()
```

```
unique_elements = unique([2,1,2])
```

但实际上你可以使用不太为人所知的 `set` 类型：

```
unique_elements = set([2,1,2]) # 移除重复项
```

这个对象是可迭代的，就像一个普通的 `list`。如果你真的想要一个 `list` 对象，你可以直接使用：

```
unique_elements = list(set([2,1,2])) # 移除重复项
```

显然，`set` 在这里是正确的工具。但是如果你不知道 `set` 类型，你可能会编写像上面的 `unique()` 这样的代码。

为什么重用库如此重要

一个普遍引用的统计数据是，平均软件工程师每天产生十行可交付的代码。当程序员第一次听到这个时，他们不相信——“十行代码？我一分钟就能写完！”

关键词是 *shippable*。成熟库中的每一行代码都代表了相当数量的设计、调试、重写、文档编写、优化和测试。任何经过这种达尔文过程的代码行都非常有价值。这就是为什么重用库如此重要，既可以节省时间，又可以减少编写的代码。

示例：使用 Unix 工具而不是编码

当 Web 服务器频繁返回 4xx 或 5xx HTTP 响应代码时，这表明存在潜在问题（4xx 是客户端错误；5xx 是服务器错误）。因此，我们想编写一个程序，该程序解析 Web 服务器的访问日志并确定哪些 URL 导致了最多的错误。

访问日志通常看起来像这样：

```
1.2.3.4 example.com [24/Aug/2010:01:08:34] "GET /index.html HTTP/1.1" 200 ...
2.3.4.5 example.com [24/Aug/2010:01:14:27] "GET /help?topic=8 HTTP/1.1" 500 ...
3.4.5.6 example.com [24/Aug/2010:01:15:54] "GET /favicon.ico HTTP/1.1" 404 ...
...
```

一般来说，它们包含这种形式的行：

浏览器-IP 主机 [日期] "GET /url-path HTTP/1.1" HTTP-响应代码 ...

编写一个程序来查找最常见的 url-paths，其具有 4xx 或 5xx 响应代码，在 C++ 或 Java 这样的语言中可能很容易需要 20 行代码。

相反，在 Unix 中，您可以键入以下命令行：

```
cat access.log | awk '{ print $5 " " $7 }' | egrep "[45]..$" \
| sort | uniq -c | sort -nr
```

它产生的输出如下：

```
95 /favicon.ico 404
13 /help?topic=8 500
11 /login 403
...
<计数> <路径> <HTTP 响应代码>
```

这个命令行的优点在于我们避免了编写任何“真正的”代码或将任何东西检入源代码控制。



总结

冒险，刺激——绝地武士不渴望这些东西。

—尤达

本章是关于编写尽可能少的新代码。每一行新代码都需要经过测试、记录和维护。此外，代码库中的代码越多，它就越“重”，开发起来就越困难。

你可以通过以下方式避免编写新的代码行：

- 从你的产品中消除非必要的功能，并且不要过度设计
- 重新思考需求，以解决仍然可以完成工作的最简单版本的问题
- 通过定期阅读其整个 API 来熟悉标准库

第四部分

精选主题

在前三个部分中，我们介绍了许多使代码易于理解的技术。在这一部分中，我们将把其中的一些技术应用到两个选定的主题。

首先，我们将讨论测试——如何编写既有效又可读的测试。

然后，我们将逐步完成一个专用数据结构（一个“分钟/小时计数器”）的设计和实现，以了解性能、良好设计和可读性如何相互作用的示例。

测试和可读性



在本章中，我们将向您展示编写简洁有效的测试的简单技术。

测试对不同的人意味着不同的事情。在本章中，我们使用“测试”来表示任何代码，其唯一目的是检查另一段（“真实”）代码的行为。我们将重点关注测试的可读性方面，而不讨论您是否应该在编写真实代码之前编写测试代码（“测试驱动开发”）或测试开发的其他哲学方面。

使测试易于阅读和维护

测试代码的可读性与非测试代码同样重要。其他程序员经常会把测试代码看作是对真实代码如何工作以及应该如何使用的非官方文档。因此，如果测试易于阅读，用户将更好地理解真实代码的行为方式。

关键思想

测试代码应该是可读的，以便其他程序员可以轻松地进行更改或添加测试。

当测试代码庞大而可怕时，会发生以下情况：

- 程序员害怕修改真实代码。哦，我们不想搞乱那些代码——更新所有测试将是一场噩梦！
- 程序员在添加新代码时不会添加新测试。随着时间的推移，你的模块测试越来越少，你不再确信它都能正常工作。

相反，你希望鼓励你的代码用户（尤其是你！）对测试代码感到满意。他们应该能够诊断出为什么新的更改会破坏现有的测试，并且觉得添加新的测试很容易。

这个测试有什么问题？

在我们的代码库中，我们有一个函数可以对评分的搜索结果列表进行排序和过滤。这是函数声明：

```
// 按分数（最高分在前）对“docs”进行排序，并删除负分文档。  
void SortAndFilterDocs(vector<ScoredDocument>* docs);
```

此函数的测试最初看起来像这样：

```
void Test1() {  
    vector<ScoredDocument> docs;  
    docs.resize(5);  
    docs[0].url = "http://example.com";  
    docs[0].score = -5.0;  
    docs[1].url = "http://example.com";  
    docs[1].score = 1;  
}
```

```

docs[2].url = "http://example.com";
docs[2].score = 4;
docs[3].url = "http://example.com";
docs[3].score = -99998.7;
docs[4].url = "http://example.com";
docs[4].score = 3.0;

SortAndFilterDocs(&docs);

assert(docs.size() == 3);
assert(docs[0].score == 4);
assert(docs[1].score == 3.0);
assert(docs[2].score == 1);
}

```

这段测试代码至少有八个不同的问题。在本章结束时，你将能够识别并修复所有这些问题。

使这个测试更具可读性

作为一般的设计原则，你应该向用户隐藏不太重要的细节，以便更重要的细节最为突出。

上一节的测试代码明显违反了这一原则。测试的每一个细节都摆在最前面，就像设置`vector<ScoredDocument>`的不重要的细节一样。大多数示例代码涉及`url`、`score`和`docs[]`，这些都只是关于如何设置底层C++对象的细节，而不是关于这个测试在高层次上做什么。

作为清理工作的第一步，我们可以创建一个像这样的辅助函数：

```

void MakeScoredDoc(ScoredDocument* sd, double score, string url) {
    sd->score = score;
    sd->url = url;
}

```

使用这个函数，我们的测试代码变得稍微紧凑一些：

```

void Test1() {
    vector<ScoredDocument> docs;
    docs.resize(5);
    MakeScoredDoc(&docs[0], -5.0, "http://example.com");
    MakeScoredDoc(&docs[1], 1, "http://example.com");
    MakeScoredDoc(&docs[2], 4, "http://example.com");
    MakeScoredDoc(&docs[3], -99998.7, "http://example.com");
    ...
}

```

但这还不够好——仍然有一些不重要的细节摆在我们面前。例如，参数`"http://example.com"`只是一个眼中钉。它总是相同的，而且确切的URL甚至都不重要——它只是需要填写一个有效的`ScoredDocument`

我们被迫看到的另一个不重要的细节是docs.resize(5)和docs[0] & docs[1]等等。让我们改变我们的辅助函数，为我们做更多的工作，并称之为AddScoredDoc()

```
void AddScoredDoc(vector<ScoredDocument>& docs, double score) {
    ScoredDocument sd;
    sd.score = score;
    sd.url = "http://example.com";
    docs.push_back(sd);
}
```

使用这个函数，我们的测试代码更加紧凑：

```
void Test1() {
    vector<ScoredDocument> docs;
    AddScoredDoc(docs, -5.0);
    AddScoredDoc(docs, 1);
    AddScoredDoc(docs, 4);
    AddScoredDoc(docs, -99998.7);
    ...
}
```

这段代码更好，但仍然没有通过“高度可读和可写”的测试。如果你想用一组新的评分文档添加另一个测试，这将需要大量的复制和粘贴。那么我们该如何进一步改进它呢？

创建最小测试语句

为了改进这个测试代码，让我们使用第12章“将想法转化为代码”中的技术。让我们用简单的英语描述我们的测试想要做什么：

我们有一个文档列表，其分数是[-5, 1, 4, -99998.7, 3]。在SortAndFilterDocs()之后，剩余的文档应该具有[4, 3, 1]的分数，按照这个顺序。

正如你所看到的，在那段描述中，我们没有提到vector<ScoredDocument>这里最重要的是分数数组。理想情况下，我们的测试代码看起来像这样：

```
CheckScoresBeforeAfter("-5, 1, 4, -99998.7, 3", "4, 3, 1");
```

我们能够将这个测试的本质提炼成一行代码！

不过，这并不罕见。大多数测试的本质都归结为：对于这个输入/情况，期望这种行为/输出。而且很多时候，这个目标可以用一行代码来表达。此外，使代码非常紧凑和可读，保持测试语句简短使得添加更多测试用例非常容易。

实现自定义“微型语言”

请注意，CheckScoresBeforeAfter() 接受两个字符串参数，用于描述分数数组。在更高版本的 C++ 中，您可以像这样传入数组字面量：

```
CheckScoresBeforeAfter({-5, 1, 4, -99998.7, 3}, {4, 3, 1});
```

因为当时我们无法做到这一点，所以我们将分数放在一个字符串中，用逗号分隔。为了使这种方法有效，`CheckScoresBeforeAfter()` 将不得不解析这些字符串参数。

一般来说，定义自定义微型语言可能是一种强大的方式，可以用少量空间表达大量信息。其他例子包括 `printf()` 和正则表达式库。

在这种情况下，编写一些辅助函数来解析逗号分隔的数字列表应该不会太难。以下是 `checkScoresBeforeAfter` 的样子：

```
void CheckScoresBeforeAfter(string input, string expected_output) {
    vector<ScoredDocument> docs = ScoredDocsFromString(input);
    SortAndFilterDocs(&docs);
    string output = ScoredDocsToString(docs);
    assert(output == expected_output);
}
```

为了完整起见，以下是在 `string` 和

`vector<ScoredDocument>` 之间转换的辅助函数：

```
vector<ScoredDocument> ScoredDocsFromString(string scores) {
    vector<ScoredDocument> docs;

    replace(scores.begin(), scores.end(), ',', ' ');

    // 从空格分隔的分数字符串填充 'docs'。
    istringstream stream(scores);
    double score;
    while (stream >> score) {
        AddScoredDoc(docs, score);
    }

    return docs;

string ScoredDocsToString(vector<ScoredDocument> docs) {
    ostringstream stream;
    for (int i = 0; i < docs.size(); i++) {
        if (i > 0) stream << ", ";
        stream << docs[i].score;
    }

    return stream.str();
}
```

乍一看，这似乎有很多代码，但它让你做的事情非常强大。

因为你只需调用一次 `CheckScoresBeforeAfter` 就可以编写一个完整的测试，所以你会更倾向于添加更多测试（我们将在本章后面进行）。

使错误消息可读



前面的代码很好，但是当 `assert(output == expected_output)` 行失败时会发生什么？它会产生如下错误消息：

```
断言失败: (output == expected_output),  
function CheckScoresBeforeAfter, file test.cc, line 37.
```

显然，如果你看到这个错误，你会想，`output` 和

`expected_output` 的值是什么？

使用更好的 `assert()` 版本

幸运的是，大多数语言和库都有更复杂的 `assert()` 版本，你可以使用。所以不用写：

```
assert(output == expected_output);
```

你可以使用 Boost C++ 库：

```
BOOST_REQUIRE_EQUAL(output, expected_output)
```

现在，如果测试失败，你会得到更详细的消息，例如：

```
test.cc(37): fatal error in "CheckScoresBeforeAfter": critical check  
output == expected_output failed ["1, 3, 4" != "4, 3, 1"]
```

这更有帮助。

你应该使用这些更有帮助的断言方法，如果它们可用的话。每次你的测试失败时，它都会有所回报。

其他语言中更好的 ASSERT()

在 Python 中，内置语句 `assert a == b` 会产生一个简单的错误消息，例如：

```
文件 "file.py", 第 X 行, 在 <module> 中
    assert a == b
断言错误
```

相反，您可以使用 `unittest` 模块中的 `assertEqual()` 方法：

```
import unittest

class MyTestCase(unittest.TestCase):
    def testFunction(self):
        a = 1
        b = 2
        self.assertEqual(a, b)

if __name__ == '__main__':
    unittest.main()
```

这将产生类似以下的错误消息：

```
文件 "MyTestCase.py", 第 7 行, 在 testFunction 中
    self.assertEqual(a, b)
AssertionError: 1 != 2
```

无论你使用哪种语言，可能都有一个库/框架（例如，XUnit）可以帮助你。了解你的库是有好处的！

手工制作的错误消息

使用 `BOOST_REQUIRE_EQUAL()` 我们能够获得更好的错误消息：

```
output == expected_output 失败 ["1, 3, 4" != "4, 3, 1"]
```

然而，这个消息可以进一步改进。例如，看到触发此失败的原始输入将很有用。理想的错误消息应该是这样的：

```
CheckScoresBeforeAfter() 失败,
  输入:      "-5, 1, 4, -99998.7, 3"
  预期输出: "4, 3, 1"
  实际输出: "1, 3, 4"
```

如果这是你想要的，那就继续写吧！

```

void CheckScoresBeforeAfter(...) {
    ...

    if (output != expected_output) {
        cerr << "CheckScoresBeforeAfter() 失败," << endl;
        cerr << "输入:      \"" << input << "\"" << endl;
        cerr << "预期输出: \"" << expected_output << "\"" << endl;
        cerr << "实际输出: \"" << output << "\"" << endl;
        abort();
    }
}

```

这个故事的寓意是，错误信息应该尽可能地有帮助。有时候，通过构建“自定义断言”来打印你自己的信息是最好的方法。

选择好的测试输入

为你的测试选择好的输入值是一门艺术。我们现在拥有的这些看起来有点随意：

```
CheckScoresBeforeAfter("-5, 1, 4, -99998.7, 3", "4, 3, 1");
```

我们如何选择好的输入值？好的输入应该彻底地测试代码。但它们也应该很简单，以便于阅读。

关键思想

一般来说，你应该选择最简单的输入集合，以完全执行代码。

例如，假设我们刚刚写了：

```
CheckScoresBeforeAfter("1, 2, 3", "3, 2, 1");
```

虽然这个测试很简单，但它并没有测试SortAndFilterDocs()的“过滤负分”行为。SortAndFilterDocs()如果代码的那部分存在错误，这个输入不会触发它。

另一方面，假设我们像这样编写测试：

```
CheckScoresBeforeAfter("123014, -1082342, 823423, 234205, -235235",
                        "823423, 234205, 123014");
```

这些值不必要地复杂。（而且它们甚至没有彻底地测试代码。）

简化输入值

那么我们能做些什么来改进这些输入值呢？

```
CheckScoresBeforeAfter("-5, 1, 4, -99998.7, 3", "4, 3, 1");
```


嗯，你可能首先注意到的是非常“响亮”的值-99998.7。那个值只是意味着“任何负数”，所以一个更简单的值就是-1。（如果-99998.7意味着“一个非常负的数”，一个更好的值应该是像-1e100这样清晰的值。）

关键思想

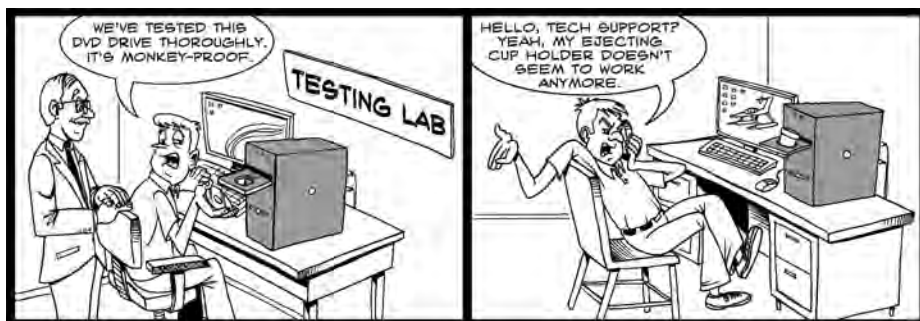
首选干净简单的测试值，这些值仍然可以完成工作。

我们测试中的其他值还不错，但既然我们在这里，我们可以将它们简化为最简单的整数。此外，只需要一个负值来测试是否删除了负值。这是我们测试的新版本：

```
CheckScoresBeforeAfter("1, 2, -1, 3", "3, 2, 1");
```

我们简化了测试值，但并没有降低它们的有效性。

大型“粉碎机”测试



针对大型、疯狂的输入测试你的代码绝对是有价值的。例如，你可能会想要包含一个像这样的测试：

```
CheckScoresBeforeAfter("100, 38, 19, -25, 4, 84, [大量的值] ...",  
                        "100, 99, 98, 97, 96, 95, 94, 93, ...");
```

像这样的大型输入在暴露诸如缓冲区溢出或你可能没有预料到的其他错误方面做得很好。

但是像这样的代码看起来很大很可怕，并且在压力测试代码方面并不完全有效。相反，以编程方式构造大型输入更有效，构造一个（比如）100,000个值的大型输入。

功能的多次测试

与其构建一个单一的“完美”输入来彻底测试你的代码，不如编写多个较小的测试，这通常更容易、更有效且更具可读性。

每个测试都应该朝着某个方向推动你的代码，试图找到一个特定的错误。例如，以下是 `SortAndFilterDocs` 的四个测试：

```
CheckScoresBeforeAfter("2, 1, 3", "3, 2, 1"); // 基本排序
CheckScoresBeforeAfter("0, -0.1, -10", "0"); // 所有值 < 0 被移除
CheckScoresBeforeAfter("1, -2, 1, -2", "1, 1"); // 重复不是问题
CheckScoresBeforeAfter("", ""); // 空输入 OK
```

如果你想非常彻底，你可以编写更多的测试。拥有单独的测试用例也使得下一个处理代码的人更容易。如果有人不小心引入了一个错误，测试失败将精确指出哪个特定测试失败。

命名测试函数

测试代码通常被组织成函数——每个函数对应一个你正在测试的方法和/或情况。例如，测试 `SortAndFilterDocs` 的代码在一个名为 `Test1()` 的函数内部：

```
void Test1() {
    ...
}
```

为测试函数选择一个好名字可能看起来很乏味且无关紧要，但不要使用像 `Test1()` `Test2()` 等无意义的名字。

相反，你应该使用名称来描述测试的细节。特别是，如果阅读测试代码的人可以快速弄清楚，这将非常方便：

- 正在测试的类（如果有）
- 正在测试的函数
- 被测试的情况或错误

构建一个好的测试函数名称的简单方法是将这些信息连接在一起，可能带有“Test_”前缀。

例如，我们可以使用 `Test_<FunctionName>()` 格式，而不是将其命名为 `Test1()`：

```
void Test_SortAndFilterDocs() {
    ...
}
```

根据此测试的复杂程度，您可以考虑为正在测试的每种情况使用单独的测试函数。您可以使用 `Test_<FunctionName>_<Situation>` 格式：

```
void Test_SortAndFilterDocs_BasicSorting() {
    ...
}
```

```

void Test_SortAndFilterDocs_NegativeValues() {
    ...
}

...

```

不要害怕这里有一个长或笨拙的名字。这不是一个将在整个代码库中调用的函数，因此避免长函数名称的原因不适用。测试函数名称实际上就像一个注释。此外，如果该测试失败，大多数测试框架将打印出断言失败的函数名称，因此描述性名称特别有用。

请注意，如果您使用的是测试框架，则可能已经存在关于方法如何命名的规则或约定。例如，Python `unittest` 模块希望测试方法名称以 “test” 开头。

在命名测试代码中的 *helper* 函数时，突出显示该函数是否执行任何断言本身，或者只是一个普通的 “测试不知情” 助手，这很有用。例如，在本章中，任何调用 `assert()` 的辅助函数都命名为 `Check...`，但是函数 `AddScoreDoc` 的命名方式就像一个普通的辅助函数。

那个测试有什么问题？

在本章的开头，我们声称这个测试至少有八个问题：

```

void Test1() {
    vector<ScoredDocument> docs;
    docs.resize(5);
    docs[0].url = "http://example.com";
    docs[0].score = -5.0;
    docs[1].url = "http://example.com";
    docs[1].score = 1;
    docs[2].url = "http://example.com";
    docs[2].score = 4;
    docs[3].url = "http://example.com";
    docs[3].score = -99998.7;
    docs[4].url = "http://example.com";
    docs[4].score = 3.0;

    SortAndFilterDocs(&docs);

    assert(docs.size() == 3);
    assert(docs[0].score == 4);
    assert(docs[1].score == 3.0);
    assert(docs[2].score == 1);
}

```

现在我们已经学习了一些编写更好测试的技巧，让我们来识别它们：

1. 测试非常长，并且充满了不重要的细节。你可以用一句话描述这个测试在做什么，所以测试语句不应该太长。
2. 添加另一个测试并不容易。你可能会想复制/粘贴/修改，这会使代码更长，并且充满重复。
3. 测试失败消息不是很有用。如果这个测试失败，它只会说 `Assertion failed: docs.size() == 3` 这没有给你足够的信息来进一步调试。
4. 测试试图一次测试所有内容。它试图同时测试负面过滤和排序功能。将其分解为多个测试会更具可读性。
5. 测试输入并不简单。特别是，示例分数 `-99998.7` 非常“醒目”，即使该特定值没有任何意义，也会引起您的注意。一个更简单的负值就足够了。
6. 测试输入没有彻底地执行代码。例如，它没有测试当分数是 0 时的情况。（该文档会被过滤掉吗？）
7. 它没有测试其他极端输入，例如空输入向量、非常大的向量或具有重复分数的向量。
8. 名称 `Test1()` 没有意义——名称应该描述正在测试的函数或情况。

测试友好的开发

有些代码比其他代码更容易测试。理想的测试代码具有定义良好的接口，没有太多状态或其他“设置”，并且没有太多隐藏数据可供检查。

如果你在编写代码时知道稍后会为它编写测试，那么会发生一件有趣的事情：你开始设计你的代码，使其易于测试！幸运的是，以这种方式编码也意味着你通常会创建更好的代码。对测试友好的设计通常自然而然地导致组织良好的代码，各个部分分别执行不同的操作。

测试驱动开发

测试驱动开发 (TDD) 是一种编程风格，在这种风格中，您在编写实际代码之前编写测试。TDD 的支持者认为，与在编写代码后编写测试相比，此过程可以极大地提高非测试代码的质量。

这是一个备受争议的话题，我们不会深入探讨。至少，我们发现，在编写代码时牢记测试有助于使代码更好。

但无论你是否采用 TDD，最终的结果是你拥有测试其他代码的代码。本章的目标是帮助你使你的测试更容易阅读和编写。

在将程序分解为类和方法的各种方式中，解耦程度最高的通常最容易测试。另一方面，假设你的程序非常互连，类之间有很多方法调用，并且所有方法都有很多参数。不仅该程序会有难以理解的代码，而且测试代码也会同样丑陋，难以阅读和编写。

拥有大量的“外部”组件（需要初始化的全局变量、需要加载的库或配置文件等）也会使编写测试更加麻烦。

一般来说，如果你在设计你的代码时意识到，嗯，这将是一个测试的噩梦，这是一个停止并重新思考设计的好理由。表14-1显示了一些典型的测试和设计问题。

表14-1. 可测试性较差的代码的特征，以及这如何导致设计问题

特征	可测试性问题	设计问题
使用全局变量	所有全局状态都需要为每个测试重置（否则，不同的测试可能会相互干扰）。	很难理解哪些函数具有哪些副作用。不能孤立地考虑每个函数；需要考虑整个程序才能理解一切是否正常。
代码依赖于许多外部组件	编写任何测试都更加困难，因为首先需要设置大量的脚手架。测试编写起来不太有趣，所以人们避免编写测试。	当其中一个依赖项失败时，系统更有可能失败。更难理解任何给定的更改可能产生的影响。更难重构类。系统有更多的故障模式和恢复路径需要考虑。
代码具有非确定性行为	测试不稳定且不可靠。偶尔失败的测试最终会被忽略。	程序更可能出现竞争条件或其他无法重现的错误。程序更难推理。 生产环境中的错误很难追踪和修复。

另一方面，如果你的设计很容易编写测试，这是一个好兆头。

表 14-2列出了一些有益的测试和设计特性。

表 14-2. 更易于测试的代码的特性，以及这如何带来好的设计

特征	可测试性优势	设计优势
类几乎没有或没有内部状态	测试更容易编写，因为测试一个方法所需的设置更少，需要检查的隐藏状态也更少。	状态较少的类更简单，更容易理解。
类/函数只做一件事	需要更少的测试用例来完全测试它。	更小v/更简单的组件更模块化，并且系统通常更解耦。

类依赖于很少的其他类；高度解耦	每个类都可以独立测试（比一次测试多个类容易得多）。	系统可以并行开发。类可以轻松修改或删除，而不会扰乱系统的其余部分。
函数具有简单、定义明确的接口	有明确的行为需要测试。简单的接口需要更少的工作来测试。	接口更容易让程序员学习，并且更有可能被重用。

走得太远

也有可能过于关注测试。以下是一些例子：

- **为了测试而牺牲你的真实代码的可读性。**
设计你的真实代码使其可测试应该是一个双赢的局面：你的真实代码变得更简单、更解耦，并且你的测试也更容易编写。但是，如果你必须要在你的真实代码中插入大量丑陋的管道，仅仅为了能够测试它，那就说明有问题了。
- 痴迷于 100% 的测试覆盖率。测试代码的前 90% 通常比测试剩下的 10% 要少花精力。剩下的 10% 可能涉及用户界面，或者是一些愚蠢的错误情况，这些错误的代价实际上并不高，而测试它的努力根本不值得。

事实是，你永远无法获得 100% 的覆盖率。如果不是遗漏的错误，那可能是遗漏的功能，或者你可能没有意识到应该更改规范。

根据你的错误代价有多高，在测试代码上花费多少开发时间有一个最佳点。如果你正在构建一个网站原型，那么编写任何测试代码可能都不值得。另一方面，如果你正在为宇宙飞船或医疗设备编写控制器，那么测试可能就是你的主要重点。

- 让测试妨碍产品开发。我们已经看到过这样的情况：测试本应只是项目的一个方面，却主导了整个项目。测试变成某种需要安抚的神，程序员只是走过场，而没有意识到他们宝贵的工程时间可能花在其他地方会更好。

总结

在测试代码中，可读性仍然非常重要。如果你的测试非常易读，那么它们也会非常易写，因此人们会添加更多测试。此外，如果你将你的真实代码设计成易于测试，你的代码总体上会有一个更好的设计。

以下是关于如何改进测试的具体要点：

- 每个测试的顶层应该尽可能简洁；理想情况下，每个测试输入/输出可以用一行代码来描述。

- 如果你的测试失败，它应该发出一个错误信息，使bug易于追踪和修复。
- 使用最简单的测试输入，以完全测试您的代码。
- 给你的测试函数一个完全描述性的名称，以便清楚地知道每个函数都在测试什么。不要使用Test1()，而是使用像Test_<FunctionName>_<Situation>这样的名称。

最重要的是，使其易于修改和添加新测试。

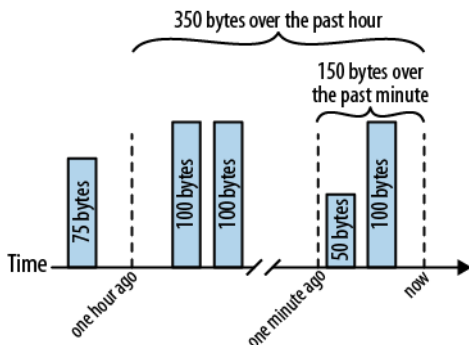
设计和实现 “分钟/小时计数器”



让我们来看看在实际生产代码中使用的数据结构：一个“分钟/小时计数器”。我们将带您了解工程师可能经历的自然思考过程，首先尝试解决这个问题，然后提高其性能并添加功能。最重要的是，我们还将努力保持代码易于阅读，使用本书中的原则。我们可能会走一些错误的弯路或犯其他错误。看看你是否能跟上并抓住它们。

问题

我们需要跟踪 Web 服务器在过去一分钟和过去一小时内传输了多少字节。以下是如何维护这些总数的说明：



这是一个相当简单的问题，但正如你将看到的，高效地解决它是一个有趣的挑战。让我们从定义类接口开始。

定义类接口

这是我们在 C++ 中的第一个类接口版本：

```
class MinuteHourCounter {
public:
    // 添加计数
    void Count(int num_bytes);

    // 返回此分钟内的计数
    int MinuteCount();

    // 返回此小时内的计数
    int HourCount();
};
```

在我们实现这个类之前，让我们先看看名称和注释，看看是否有任何我们想要更改的地方。

改进名称

类名 `MinuteHourCounter` 非常好。它非常具体、实在且易于理解。

鉴于类名，方法名 `MinuteCount()` 和 `HourCount()` 也是合理的。你可能会称它们为 `GetMinuteCount()` 和 `GetHourCount()`，但这无济于事。正如我们在 [第 3 章，名称不会被误解](#) 中所说，“get” 对很多人来说意味着“轻量级访问器”。正如你将看到的，实现不会是轻量级的，所以最好去掉“get”。

方法名 `Count()` 有问题。我们问了我们的同事他们认为 `Count()` 会做什么，有些人认为它意味着“返回所有时间的总计数”。这个名字有点违反直觉（没有双关语的意思）。问题在于 `Count` 既是名词又是动词，可能意味着“我想要你看到的样本数量的计数”或“我希望你计算这个样本”。

以下是替代 `Count()` 的名称，供您考虑：

- `Increment()`
- `Observe()`
- `Record()`
- `Add()`

`Increment()` 具有误导性，因为它暗示存在一个仅增加的值。（在我们的例子中，小时计数随时间波动。）

`Observe()` 还可以，但有点含糊。

`Record()` 也有名词/动词的问题，所以这也不好。

`Add()` 很有意思，因为它既可以表示“数值上相加”，也可以表示“添加到数据列表中”——在我们的例子中，两者都有点，所以这行得通。所以我们将方法重命名为 `void Add(int num_bytes)`。

但是参数名 `num_bytes` 太具体了。是的，我们的主要用例是计数字节，但是 `MinuteHourCounter` 不需要知道这一点。其他人可能会使用这个类来计数查询或数据库事务。我们可以使用更通用的名称，如 `delta`，但术语 `delta` 通常用在值可以为负数的地方，这不是我们想要的。名称 `count` 应该可以——它简单、通用，并且暗示“非负”。此外，它让我们可以在一个不太模糊的上下文中偷偷地使用“count”这个词。

改进注释

这是我们目前为止的类接口：

```
class MinuteHourCounter {  
    public:
```

```

// 添加计数
void Add(int count);

// 返回此分钟内的计数
int 分钟计数();

// 返回此小时内的计数
int HourCount();
};

```

让我们来逐一改进这些方法注释。考虑第一个：

```

// 添加计数
void Add(int count);

```

这个注释现在完全是多余的——应该删除或改进。这是一个改进的版本：

```

// 添加一个新的数据点 (count >= 0)。
// 在接下来的分钟内，MinuteCount() 将增加 +count。
// 在接下来的小时内，HourCount() 将增加 +count。
void Add(int count);

```

现在让我们考虑 MinuteCount() 的注释：

```

// 返回此分钟内的计数
int MinuteCount();

```

当我们询问同事这个评论是什么意思时，出现了两种相互矛盾的解释：

1. 返回当前时钟分钟内的计数，例如下午 12:13。
2. 返回过去 60 秒内的计数，无论时钟分钟边界如何。

第二种解释是它实际的工作方式。所以，让我们用更精确和详细的语言来消除这种困惑：

```

// 返回过去 60 秒内的累计计数。
int MinuteCount();

```

（同样，我们应该改进 HourCount() 的注释。）

这是包含所有更改的类定义，以及类级别的注释：

```

// 跟踪过去一分钟和过去一小时的累计计数。
// 例如，可用于跟踪最近的带宽使用情况。
class MinuteHourCounter {
    // 添加一个新的数据点 (计数 >= 0)。
    // 在接下来的1分钟内，MinuteCount() 将增加 +count。
    // 在接下来的一个小时内，HourCount() 将增加 +count。
    void Add(int count);

    // 返回过去 60 秒内的累计计数。
    int 分钟计数();
};

```

```

        // 返回过去 3600 秒内的累计计数。
        int HourCount();
};

```

(为了简洁起见，从现在开始，我们将从代码清单中删除注释。)

获得外部视角

您可能已经注意到，我们已经有几个案例让我们的同事参与其中。寻求外部视角是测试您的代码是否“用户友好”的好方法。尽量对他们的第一印象持开放态度，因为其他人可能会得出相同的结论。并且那些“其他人”可能包括六个月后的您。

尝试 1：一个幼稚的解决方案

让我们继续解决问题。我们将从一个简单的解决方案开始：只需保留一个时间戳“事件”的列表：

```

class MinuteHourCounter {
    struct Event {
        Event(int count, time_t time) : count(count), time(time) {}
        int count;
        time_t time;
    };

    list<Event> events;

public:
    void Add(int count) {
        events.push_back(Event(count, time()));
    }

    ...
};

```

然后我们可以根据需要计算最近的事件：

```

class MinuteHourCounter {
    ...

    int MinuteCount() {
        int count = 0;
        const time_t now_secs = time();
        for (list<Event>::reverse_iterator i = events.rbegin();
             i != events.rend() && i->time > now_secs - 60; ++i) {
            count += i->count;
        }
        return count;
    }
};

```

```

        int小时计数() {
            int count = 0;
            const time_t now_secs = time();
            for (list<Event>::reverse_iterator i = events.rbegin();
                 i != events.rend() && i->time > now_secs - 3600; ++i) {
                count += i->count;
            }
            return count;
        }
};

```

代码是否容易理解？

虽然这个解决方案是“正确的”，但存在一些可读性问题：

- for 循环有点冗长。大多数读者在阅读这部分代码时会明显放慢速度（至少他们应该这样做，如果他们要确保没有错误）。
- **MinuteCount()** 和 **HourCount()** 几乎相同。如果他们可以共享重复的代码，代码会更小。这个细节尤其重要，因为冗余代码相对复杂。（最好将所有困难的代码限制在一个地方。）

一个更易于阅读的版本

MinuteCount() 和 **HourCount()** 的代码仅相差一个常数（60 与 3600）。显而易见的重构是引入一个辅助方法来处理这两种情况：

```

class MinuteHourCounter {
    list<Event> events;

    int CountSince(time_t cutoff) {
        int count = 0;
        for (list<Event>::reverse_iterator rit = events.rbegin();
             rit != events.rend(); ++rit) {
            if (rit->time <= cutoff) {
                break;
            }
            count += rit->count;
        }
        return count;
    }

public:
    void Add(int count) {
        events.push_back(Event(count, time()));
    }

    int MinuteCount() {
        return CountSince(time() - 60);
    }
};

```

```

    }

    int HourCount() {
        return CountSince(time() - 3600);
    }
};

```

关于这段新代码，有几点值得指出。

首先，请注意 `CountSince()` 接受的是一个绝对的 `cutoff` 参数，而不是一个相对的 `secs_ago` 值（60 或 3600）。两种方式都可以，但这样 `CountSince()` 的工作会稍微轻松一些。

其次，我们将迭代器从 `i` 重命名为 `rit`。名称 `i` 更常用于整数索引。我们曾考虑使用名称 `it`，这对于迭代器来说很典型。但在这种情况下，我们有一个 *reverse* 迭代器，并且这个事实对于代码的正确性至关重要。通过使用以 `r` 为前缀的变量名，它为像 `rit != events.rend()` 这样的语句增加了一种令人欣慰的对称性。

最后，我们从 `for` 循环中提取了条件 `rit->time <= cutoff`，并将其作为一个单独的 `if` 语句。为什么？因为 `for(begin; end; advance)` 形式的“传统”`for` 循环最容易阅读。读者可以立即将其理解为“遍历所有元素”，而无需进一步思考。

性能问题

虽然我们改进了代码的外观，但这种设计存在两个严重的性能问题：

1. 它一直在增长。

该类保留了它所见过的所有事件——它使用了无限量的内存！理想情况下，`MinuteHourCounter` 应该自动删除一小时前的事件，因为它们不再需要。

2. `MinuteCount()` 和 `HourCount()` 太慢了。

方法 `CountSince()` 需要 $O(n)$ 时间，其中 n 是相关时间窗口中的数据点数。想象一下，一个高性能服务器每秒调用 `Add()` 数百次。每次调用 `HourCount()` 都必须计算一百万个数据点！理想情况下，`MinuteHourCounter` 应该保持单独的 `minute_count` 和 `hour_count` 变量，这些变量会随着每次调用 `Add()` 而更新。

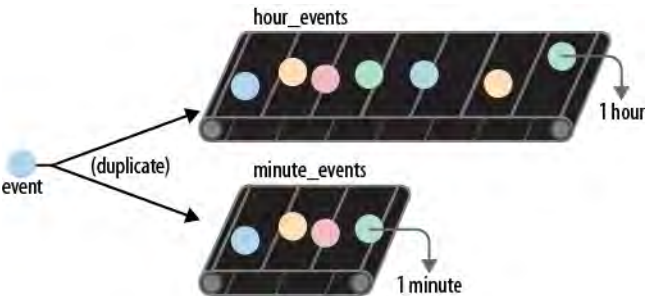
尝试 2：传送带设计

我们需要一个能够解决之前两个问题的设计：

1. 删除我们不再需要的数据。
2. 保持预先计算的 `minute_count` 和 `hour_count` 总数是最新的。

我们将这样实现：我们将像传送带一样使用我们的 list。当新数据到达一端时，我们将其添加到我们的总数中。当数据太旧时，它会从另一端“掉落”，然后我们从总数中减去。

我们可以通过几种方式来实现这种传送带设计。一种方法是维护两个独立的 lists，一个用于过去一分钟内的事件，一个用于过去一小时内的事件。当一个新事件进入时，将副本添加到两个列表中。



这种方式非常简单，但效率低下，因为它会复制每个事件两次。
另一种方法是维护两个 lists，事件最初进入第一个 list（“最近一分钟事件”），然后将其馈送到第二个 list（“最近一小时[但不是最近一分钟]事件”）。



这种“两阶段”传送带设计似乎更有效，所以让我们实现这个设计。

实施两阶段传送带设计

让我们从列出我们班的成员开始：

```
class MinuteHourCounter {
    list<Event> minute_events;
    list<Event> hour_events; // 仅包含不在 minute_events 中的元素

    int minute_count;
    int hour_count; // 统计过去一小时内的所有事件，包括过去一分钟内的事件
};
```

这种传送带设计的关键在于能够随着时间的推移“移动”事件，以便事件从 minute_events 移动到 hour_events，并且 minute_count 和 hour_count 得到更新

相应地。为此，我们将创建一个名为 `ShiftOldEvents()` 的辅助方法。一旦我们有了该方法，该类的其余部分就很容易实现：

```
void Add(int count) {
    const time_t now_secs = time();
    ShiftOldEvents(now_secs);

    // 馈送到分钟列表（而不是小时列表——这将在稍后发生）
    minute_events.push_back(Event(count, now_secs));

    minute_count += count;
    hour_count += count;
}

int MinuteCount() {
    ShiftOldEvents(time());
    return minute_count;
}

int HourCount() {
    ShiftOldEvents(time());
    return hour_count;
}
```

显然，我们将所有的脏活都推迟到了 `ShiftOldEvents()`

```
// 查找并删除旧事件，并相应地减少 hour_count 和 minute_count。
void ShiftOldEvents(time_t now_secs) {
    const int minute_ago = now_secs - 60;
    const int hour_ago = now_secs - 3600;

    // 将超过一分钟的事件从 'minute_events' 移动到 'hour_events'
    // （超过一小时的事件将在第二个循环中删除。）
    while (!minute_events.empty() && minute_events.front().time <= minute_ago) {
        hour_events.push_back(minute_events.front());

        minute_count -= minute_events.front().count;
        minute_events.pop_front();
    }

    // 从 'hour_events' 中移除超过一小时的事件
    while (hour_events.empty() && hour_events.front().time <= hour_ago) {
        hour_count -= hour_events.front().count;
        hour_events.pop_front();
    }
}
```

我们完成了吗？

我们已经解决了之前提到的两个性能问题，并且我们的解决方案有效。对于许多应用程序来说，这个解决方案已经足够好了。但是也有一些不足之处。

首先，这个设计非常不灵活。假设我们想要统计过去24小时的数据。那将需要对代码进行大量的修改。而且你可能已经注意到了，`ShiftOldEvents()` 是一个非常密集的函数，分钟和小时数据之间存在微妙的交互。

其次，这个类有一个相当大的内存占用。假设你有一个高流量的服务器每秒调用 `Add()` 100次。因为我们保留过去一小时的所有数据，这段代码最终将需要大约5MB的内存。

一般来说，`Add()` 被调用的频率越高，我们使用的内存就越多。在生产环境中，使用大量、不可预测内存的库是不好的。理想情况下，`t MinuteHourCounter` 应该使用固定数量的内存，无论 `Add()` 被调用的频率如何。

尝试 3：基于时间桶的设计

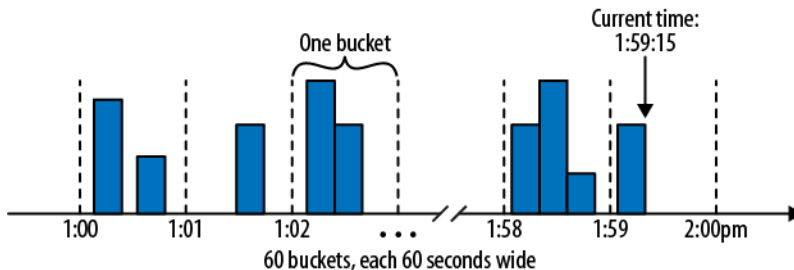
你可能没有注意到，但之前的两个实现都有一个小错误。我们使用了 `time_t` 来存储时间戳，它存储的是整数秒。由于这种舍入，`MinuteCount()` 实际上返回的是59到60秒的数据，具体取决于你调用它的时间。

例如，如果一个事件发生在 `time = 0.99`秒，那么这个时间将被四舍五入到 `t=0`秒。如果你在 `time = 60.1` 秒调用 `MinuteCount()`，它将返回 `t=1,2,3,...60`的事件总数。因此，第一个事件将被错过，即使它在技术上不到一分钟前。

平均而言，`MinuteCount()` 将返回59.5秒的数据。而 `HourCount()` 将返回3599.5秒的数据（一个可以忽略的误差）。

我们可以通过使用具有亚秒级粒度的时间来解决所有这些问题。但有趣的是，大多数使用 `MinuteHourCounter` 的应用程序首先不需要那么高的精度。我们将利用这一事实来设计一个新的 `MinuteHourCounter`，它更快并且使用更少的空间。这是精度与性能之间的权衡，这是非常值得的。

关键的想法是将小时间窗口内的所有事件 *bucket* 在一起，并用一个总数来概括这些事件。例如，过去一分钟内的事件可以插入到60个离散的桶中，每个桶的宽度为1秒。过去一小时内的事件也可以插入到60个离散的桶中，每个桶的宽度为1分钟。



使用如上所示的桶，方法 `MinuteCount()` 和 `HourCount()` 的精度将达到 $1/60$ ，这是合理的。*

如果需要更高的精度，可以使用更多的桶来换取更大的内存占用空间。但重要的是，这种设计具有固定的、可预测的内存使用量。

实施时间分段设计

仅用一个类来实现这个设计会产生大量复杂的代码，难以理解。相反，我们将遵循第 11 章“一次只做一件事”中的建议，创建单独的类来处理这个问题的不同部分。

首先，让我们创建一个单独的类来跟踪单个时间跨度（如过去一小时）的计数。我们将其称为 `TrailingBucketCounter`。它本质上是一个通用版本 `MinuteHourCounter` 来处理一个时间跨度。这是接口：

```
// 一个类，用于保存过去 N 个时间段的计数。
class TrailingBucketCounter {
public:
    // 示例：TrailingBucketCounter(30, 60) 跟踪最后 30 分钟的时间桶。
    TrailingBucketCounter(int num_buckets, int secs_per_bucket);

    void Add(int count, time_t now);

    // 返回过去 num_buckets 时间段内的总计数
    int TrailingCount(time_t now);
};
```

你可能想知道为什么 `Add()` 和 `TrailingCount()` 需要当前时间 (`time_t now`) 作为一个参数——如果这些方法自己计算当前的 `time()` 不是更容易吗？

* 与之前的解决方案类似，最后一个桶平均只会半满。通过这种设计，我们可以通过保持 61 个桶而不是 60 个桶并忽略当前“进行中”的桶来纠正低估。但这会导致数据部分“陈旧”。一个更好的修复方法是将进行中的桶与最旧桶的互补部分结合起来，以获得既无偏见又最新的计数。此实现留给读者作为练习。

虽然这可能看起来很奇怪，但传入当前时间有几个好处。首先，它使 `TrailingBucketCounter` 成为一个“无时钟”类，这通常更容易测试且不易出错。其次，它将所有对 `time()` 的调用都保存在 `MinuteHourCounter` 中。对于时间敏感的系统，如果你可以将所有获取时间的调用放在一个地方，这将有所帮助。

假设 `TrailingBucketCounter` 已经实现，那么 `MinuteHourCounter` 很容易实现：

```
class MinuteHourCounter {
    TrailingBucketCounter minute_counts;
    TrailingBucketCounter hour_counts;

public:
    MinuteHourCounter() :
        minute_counts(/* num_buckets = */ 60, /* secs_per_bucket = */ 1),
        hour_counts( /* num_buckets = */ 60, /* secs_per_bucket = */ 60) {
    }

    void Add(int count) {
        time_t now = time();
        minute_counts.Add(count, now);
        hour_counts.Add(count, now);
    }

    int MinuteCount() {
        time_t now = time();
        return minute_counts.TrailingCount(now);
    }

    int HourCount() {
        time_t now = time();
        return hour_counts.TrailingCount(now);
    }
};
```

这段代码更具可读性，而且更灵活——如果我们想增加桶的数量（以提高精度但增加内存使用量），这将很容易做到。

实现 `TrailingBucketCounter`

现在剩下要做的就是实现 `TrailingBucketCounter` 类。再一次，我们将创建一个辅助类来进一步分解这个问题。

我们将创建一个名为 `ConveyorQueue` 的数据结构，其工作是处理底层计数及其总数。`TrailingBucketCounter` 类可以专注于根据经过的时间移动 `ConveyorQueue` 的任务。

这是 `ConveyorQueue` 接口：

```
// 一个具有最大槽数的队列，旧数据会从末尾“掉落”。
class ConveyorQueue {
    ConveyorQueue(int max_items);
```

```

// 增加队列末尾的值。
    void AddToBack(int count);

// 队列中的每个值都向前移动 'num_shifted'。
// 新项目初始化为 0。
// 最旧的项目将被删除，因此 <= max_items。
    void Shift(int num_shifted);

// 返回队列中当前所有项目的总值。
    int 总和();
};

```

假设这个类已经实现，看看实现 `TrailingBucketCounter` 有多容易：

```

class TrailingBucketCounter {
    ConveyorQueue buckets;
    const int secs_per_bucket;
    time_t last_update_time; // 上次调用Update()的时间

    // 计算有多少个时间桶已经过去，并相应地Shift()。
    void Update(time_t now) {
        int current_bucket = now / secs_per_bucket;
        int last_update_bucket = last_update_time / secs_per_bucket;

        buckets.Shift(current_bucket - last_update_bucket);
        last_update_time = now;
    }

public:
    TrailingBucketCounter(int num_buckets, int secs_per_bucket) :
        buckets(num_buckets),
        secs_per_bucket(每个桶的秒数) {
    }

    void Add(int 计数, time_t 现在) {
        更新(现在);
        buckets.AddToBack(计数);
    }

    int TrailingCount(time_t 现在) {
        更新(现在);
        return buckets.TotalSum();
    }
};

```

这种分解成两个类（`TrailingBucketCounter` 和 `ConveyorQueue`）是另一个实例我们在 [第11章，一次一个任务](#) 中讨论的内容。我们也可以不用 `ConveyorQueue` 并在 `TrailingBucketCounter` 中直接实现所有内容。但这样，代码更容易理解。

实现 ConveyorQueue

现在剩下要做的就是实现 ConveyorQueue 类：

```
// 一个具有最大槽数的队列，旧数据会从末尾移出。
class ConveyorQueue {
    queue<int> q;
    int max_items;
    int total_sum; // q中所有项目的总和

public:
    ConveyorQueue(int max_items) : max_items(max_items), total_sum(0) {
    }

    int TotalSum() {
        return total_sum;
    }

    void Shift(int num_shifted) {
        // 如果移动的项目过多，只需清除队列。
        if (num_shifted >= max_items) {
            q = queue<int>(); // 清除队列
            total_sum = 0;
            return;
        }

        // 推送所有需要的零。
        while (num_shifted > 0) {
            q.push(0);
            num_shifted--;
        }

        // 让所有多余的物品掉落。
        while (q.size() > max_items) {
            total_sum -= q.front();
            q.pop();
        }
    }

    void AddToBack(int count) {
        if (q.empty()) Shift(1); // 确保q至少有1个项目。
        q.back() += count;
        total_sum += count;
    }
};
```

现在我们完成了！我们有一个快速且内存高效的MinuteHourCounter 以及一个更灵活的TrailingBucketCounter 它很容易重用。例如，创建一个更通用的RecentCounter来计算各种间隔，例如最后一天或最后十分钟，将非常容易。

三种解决方案的比较

让我们比较一下本章中我们研究的解决方案。下表显示了代码大小和性能统计信息（假设每秒 100 个 Add() 的高流量用例）：

解决方案	代码行数	每小时计数成本()	内存使用	每小时计数错误()
朴素解法	33	O(每小时事件数) (~360万)	无界	1/3600
传送带设计	55	O(1)	O(每小时事件数) (~5MB)	1/3600
时间分桶设计 (60个桶)	98	O(1)	O(桶数) (~500 字节)	每 60 个一份

请注意，我们最终的三类解决方案的代码总量比其他任何尝试都要多。但是，性能要优越得多，而且设计也更灵活。

此外，每个类单独来看都更容易阅读。这始终是一个积极的变化：拥有 100 行易于阅读的代码比拥有 50 行不易阅读的代码更好。

有时，将一个问题分解成多个类可能会引入类间复杂性（这是单类解决方案所没有的）。但在这种情况下，从一个类到下一个类，存在一个简单的“线性”使用链，并且只有一个类暴露给最终用户。总的来说，分解这个问题的好处使这成为一个胜利。

总结

让我们回顾一下我们为获得最终的 MinuteHourCounter 设计所经历的步骤。这个过程是其他代码演变的典型过程。

首先，我们从编写一个简单的解决方案开始。这帮助我们意识到了两个设计挑战：速度和内存使用。

接下来，我们尝试了一种“传送带”设计。这种设计提高了速度和内存使用率，但对于高性能应用程序来说仍然不够好。此外，这种设计非常不灵活：调整代码以处理其他时间间隔将需要大量工作。

我们的最终设计通过将问题分解为子问题来解决之前的问题。以下是我们创建的三个类，按自下而上的顺序排列，以及每个类解决的子问题：

传送带队列

一个可以“移动”并保持其总和的最大长度队列

尾随桶计数器

根据经过的时间移动 传送带队列 并保持单个（最新）时间间隔的计数，具有给定的精度

分钟小时计数器

简单地包含两个 尾随桶计数器，一个用于分钟计数，一个用于小时计数

进一步阅读



我们通过分析来自生产代码的数百个代码示例来创建这本书，以找出在实践中有效的方法。但我们也阅读了许多书籍和文章，这些书籍和文章对我们的追求有所帮助。

如果您有兴趣了解更多信息，这里有一些您可能会喜欢的资源。以下列表绝不是完整的，但它们是一个很好的起点。

关于编写高质量代码的书籍

代码大全：软件构建实践手册，第2版，作者：Steve McConnell（微软出版社，2004年）

一本严谨且研究充分的关于软件构建各个方面的著作，包括代码质量等。

重构：改善现有代码的设计，作者：Martin Fowler 等。（Addison-Wesley Professional，1999年）

一本很棒的书，讲述了增量代码改进的理念，并包含了许多不同重构的详细目录，以及采取步骤以减少破坏事情的机会。

编程实践，作者：Brian Kernighan 和 Rob Pike (Addison-Wesley Professional，1999)

讨论了编程的各个方面，包括调试、测试、可移植性和性能，并提供了各种编码示例。

务实的程序员：从学徒到大师，作者：Andrew Hunt 和 David Thomas（Addison-Wesley Professional，1999）

汇集了许多优秀的编程和工程原则，组织成简短的讨论。

代码整洁之道：敏捷软件工艺手册，作者：Robert C. Martin（Prentice Hall，2008）

一本与我们的书类似（但特定于 Java）的书；探讨了其他主题，如错误处理和并发。

关于各种编程主题的书籍

JavaScript: The Good Parts，作者：Douglas Crockford（O’ Reilly，2008）

我们认为这本书与我们的书有相似的精神，即使这本书不是明确地关于可读性的。它是关于使用 JavaScript 语言的一个干净的子集，这个子集更不容易出错，也更容易推理。

Effective Java，第2版，作者：Joshua Bloch（Prentice Hall，2008）

一本关于使您的 Java 程序更易于阅读且更少错误的杰出书籍。虽然它是关于 Java 的，但许多原则适用于所有语言。强烈推荐。

设计模式：可复用面向对象软件的要素，作者：Erich Gamma、Richard Helm、Ralph Johnson 和 John Vlissides (Addison-Wesley Professional, 1994 年)

关于软件工程师讨论面向对象编程的通用“模式”的原始书籍。作为一个常见、有用的模式目录，它可以帮助程序员避免在人们第一次尝试自己解决棘手问题时经常遇到的陷阱。

编程珠玑，第二版，作者：Jon Bentley (Addison-Wesley Professional, 1999 年)

一系列关于实际软件问题的文章。每一章都对解决现实世界的问题有深刻的见解。

高性能网站，作者：Steve Souders (O’ Reilly, 2007 年)

虽然不是一本关于编程的书，但这本书值得注意，因为它描述了许多无需编写太多代码即可优化网站的方法（与第 13 章“编写更少的代码”一致）。

Joel 谈软件：以及关于多样性和...，作者：Joel Spolsky

一些来自 <http://www.joelonsoftware.com/> 的最佳文章。Spolsky 撰写了关于软件工程的许多方面，并对许多相关主题有深刻的见解。请务必阅读“你永远不应该做的事情，第一部分”和“Joel 测试：改进代码的 12 个步骤”。

具有历史意义的书籍

编写可靠的代码，作者：Steve Maguire (Microsoft Press

，1993 年) 不幸的是，这本书已经有些过时了，但它确实以关于如何使你的代码更少错误的绝佳建议影响了我们。如果你阅读它，你会注意到很多与我们推荐的内容重叠。

Smalltalk 最佳实践模式，作者：Kent Beck (Prentice Hall, 1996 年)

虽然示例是用 Smalltalk 编写的，但这本书有很多好的编程原则。

编程风格的要素，作者：Brian Kernighan 和 P.J. Plauger (Computing McGraw-Hill, 1978 年)

最早处理“编写事物最清晰的方式”问题的书籍之一。大多数示例都是用 Fortran 和 PL1 编写的。

文学编程，作者：Donald E. Knuth (语言与信息研究中心，

1992)

我们完全同意高德纳的观点：“与其想象我们的主要任务是指导计算机做什么，不如集中精力向人类解释我们希望计算机做什么”（第 99 页）。但请注意：本书的大部分内容是关于高德纳用于文档的 WEB 编程环境。WEB 实际上是一种将程序编写成文学作品的语言，代码只是附带的。

我们自己使用过一个 WEB 衍生系统，我们认为当代码不断变化时（通常是这样），保持所谓的“文学程序”的新鲜度比使用我们推荐的做法保持代码的新鲜度更难。

符号

4xx HTTP 响应代码, 144
5xx HTTP 响应代码, 144
?: 条件表达式, 73-74

A

缩写, 使用名称, 19
抽象名称, 与具体名称相比, 13-15
首字母缩略词, 带有名称, 19
美学, 34-43
 将代码分成段落, 41-42
 列对齐, 38-39
 声明组织成块, 40-41
 重要性, 35
 为了保持一致性和紧凑性而换行,
 35-37
 清理不规则性的方法, 37-38
 代码顺序, 39-40
 个人风格与一致性, 42
 与设计, 34
Ajax, 使用它向服务器提交数据, 112
alert() (JavaScript), 112
不明确的名称, 24
带有不明确代词的注释, 60
匿名函数, 80
参数
 按名称赋值, 63
 条件语句中的顺序, 70
数组, 用于从数组中删除值的 JavaScript 函数,
 95
assert() 方法, 154-155
assertEqual() 方法 (Python), 155
赋值, 在 if 语句中, 71
属性, 在名称中编码, 16-17
网页用户授权, PHP 用于, 132

B

Beck, Kent, 《Smalltalk 最佳实践模式》, 119

开始和结束, 使用包含/排除范围,
 26-27
大局观注释, 55
块作用域, 100
代码块, 组织成声明, 40-41
布尔值
 布尔值的名称, 27
 重写表达式, 85
Boost C++ 库, 154
自下而上的编程, 114
Brehner, Eric, 96
在小时时间窗口中进行事件分桶, 174-178
错误
 评论和, 50
 差一错误, 25

C

C 编程语言, 变量定义
 位置, 101-102
C#, 用于清理代码的结构化习语, 76
C++
 块作用域, 100
 读取文件的代码, 112
 if 语句作用域, 98
 命名函数参数的内联注释,
 64
 宏, 90
 简化表达式, 90
 标准库, 28
 用于清理代码的结构化习语, 76
缓存, 添加, 141
大写, 带大写的名称, 20
Cipher 类 (Python), 117
类接口, 用于分钟/小时计数器, 166-169
类成员变量, 97
类成员, 限制访问, 98
类
 来自多个类的类间复杂性, 179
 名称, 8
清理代码, 结构化习语, 76

我们希望听到您对改进索引的建议。请发送电子邮件至 index@oreilly.com。

- 巧妙的代码，引起的困惑，86
- Clip() 函数，24
- JavaScript 中的闭包，99
- 代码，viii, 150
 - (另请参见测试代码)
 - 消除重复，38
 - 隔离区域，129
 - 更少与更多，3
 - 多任务与单任务，122-130
 - 优秀代码的品质，2
 - 冗余，170
 - 移除未使用的，143
 - 测试友好的开发，160
 - 将想法转化为，132-138
 - 可理解的，2
 - 编写更少的代码，140-145
- 代码库
 - 通用代码目录，114
 - 保持小巧，142
- 列对齐，38-39
- 命令行标志，名称，14
- 注释，3, 46-57, 60-65
 - 含糊不清的代词，60
 - 大局观，54
 - 代码缺陷描述，50-51
 - 紧凑性，60
 - 常量解释，51
 - 函数行为描述，61
 - 信息密集型词语，64
 - 输入/输出示例，用于说明角落
 - 案例，61-62
 - 关于代码的见解，50
 - 代码的意图声明，62-63
 - 对齐，36-37
 - 分钟/小时计数器改进，167-169
 - 命名函数参数，63-64
 - 名称和，49
 - 精确性，60, 61
 - 目的，46
 - 读者的角度，51
 - 总结，42, 55
 - 什么、为什么或如何，56
 - 何时不使用，47-49
 - 写作障碍，56
- 复杂想法，解释能力，132
- 复杂性，142
- 复杂逻辑，分解，86-88
- 具体名称，与抽象名称，13-15
- 条件表达式(?:), 73-74
- 条件，参数顺序，70
- 一致的布局，34
 - 换行符，35-37
 - 个人风格与，42
- 常量，103
 - 解释性注释，51

- 构造函数，格式化名称，21
- continue 语句，75
- 控制流，70-81
 - ?: 条件表达式，73-74
 - 函数提前返回，75-76
 - 消除变量，96
 - 跟踪执行流程，80
 - goto 语句，76
 - 嵌套，77-79
- ConveyorQueue 接口，176
 - 实现，178
- JavaScript 中的 cookies
 - ，116复制构造函数，默认，
- 13极端情况，输入/输出注释示例，用于说明，61-62
- 拐杖注释，49

D

- 带破折号的名称，20
- 连接数据库表的程序，134-137
- 德摩根定律，85
- 声明，组织成块，40-41
- 代码碎片整理，122
- 删除未使用的代码，143
- 设计，与美学，34
- 开发时间，最佳点，162
- Python中的字典，144
 - 敏感信息，117
- DISALLOW_COPY_AND_ASSIGN 宏，14
- DISALLOW_EVIL_CONSTRUCTOR 宏，13
- do-while 循环，避免，74-75
- DRY（不要重复自己）原则，89
- 重复代码，消除，38

E

- Eclipse，单词补全命令，19
- Emacs，单词补全命令，19
- end，使用包含/排除范围，26-27
- 错误消息
 - 手工制作，155-156
 - 可读性，154-156
- 异常，80
- 执行流程，跟踪，80
- 用户期望，匹配，27-28
- 解释变量，84
- 表达式
 - 分解，84-91
 - 复杂的逻辑，86-88
 - 单行与多行，3
 - 短路逻辑滥用，86
 - 简化，90
- 外部组件
 - 测试问题，161
- 提取，110

(另请参见子问题代码提取)
对象中的值, 124–128

F

假, 27
特性, 决定不实现, 140
文件内容, 读取, 112
Filter() 函数, 24
findClosestLocation() 示例, 110–111
first 和 last, 使用包含范围, 26
FIXME: 标记, 50
执行流程, 跟踪, 80
for 循环, 170, 171
 移除内部嵌套, 78–79
格式化名称, 从中获取含义, 20–21
format_pretty() 函数, 113
Fowler, Martin, Refactoring: Improving the Design
 of Existing Code, 119
函数指针, 80
功能, 项目特定的, 115
函数
 匿名的, 80
 行为描述的注释, 61
 从函数中提前返回, 75–76, 78
 将代码提取到单独的函数中, 110–118
 函数的名称, 8
 包装函数, 116
可读性的基本理论, 3

G

通用代码, 112–114
 创建, 114
通用名称, 10–12
get*() 方法, 用户期望, 27
全局作用域, JavaScript, 100
全局变量
 避免使用, 97
 可测试性, 161
Google
 DISALLOW_EVIL_CONSTRUCTOR 宏, 14开
 源项目的格式约定, 20–21

Gosling, James, 104
goto 语句, 76

H

HACK: 标记, 50
辅助方法, 37, 130
 测试代码中的名称, 159
 minute/hour 计数器中的 ShiftOldEvents(), 173
 使用测试代码清理, 151
高级注释, 55
HTML 标签, id 或 class 属性名称, 21
HttpDownload 对象, 128

匈牙利命名法, 17

I

if 语句
 内部赋值, 71
 单独处理, 127–128
 索引名称, 12
 参数顺序, 70
 C++中的作用域, 98
if/else 块, 顺序, 72–73
不可变数据类型, 104
实现功能, 决定不实现, 140
包含范围, 第一个和最后一个, 26
包含/排除范围, 开始和结束, 26–
 27
索引名称, 12
信息密集型词语, 带有注释, 64
内联注释, 命名函数参数,
 64
输入值, 为测试选择好的, 156–158
输入/输出注释示例, 用于说明
 极端情况, 61–62
IntelliJ IDEA, 单词补全命令, 19
接口
 重塑, 117
 简化现有, 116
中间结果变量, 消除, 95, 101,
 105
隔离代码区域, 129

J

Java
 块作用域, 100–101
 命名函数参数的内联注释,
 64
 用于清理代码的结构化习语, 76
JavaScript
 alert(), 112
 cookies, 116
 findClosestLocation() 示例, 110–111
 格式化名称, 21
 从数组中删除值的函数, 95
 全局作用域, 100
 无嵌套作用域, 100–101
 或运算符, 86
 私有变量, 99
jQuery JavaScript 库, 133
jQuery 库函数, 格式化名称, 21

L

last, 使用包含范围, 26
库, 116
 知识, 133–134, 143–144
 正则表达式, 153

- 限制, 名称, 25
- 代码中的换行, 35-37
- 代码行数, 最小化, 与时间要求相比, 73
- list::size() 方法, 用户期望, 28
- Python 中的列表, 144
- 逻辑
 - 分解复杂问题, 86-88
 - 清晰描述, 132
- 循环迭代器, 12
- 循环, 移除内部嵌套, 78-79

M

- 宏 (C++), 90
- 匹配数据库行, 用于查找的 Python 代码, 135-137
- max, 用于包含性限制, 25
- 内存泄漏, 14
- 内存需求, 174
- 精神负担, 67
- 混乱的代码, 注释, 50
- min, 用于包含性限制, 25
- 微型语言, 实现自定义, 152-153
- 分钟/小时计数器, 166-180
 - 类接口, 166-169
 - 注释, 167-169
 - 比较解决方案, 179
 - 传送带设计, 171-174
 - 朴素解决方案, 169-171
 - 性能问题, 171
 - 时间分段设计, 174-178
 - TrailingBucketCounter 实现, 176-177

N

- 命名函数参数注释, 63-64
- 名称
 - 首字母缩略词或缩写词, 19
 - 避免误解, 24-31
 - 布尔值, 27
 - 注释和, 49
 - 具体与抽象, 13-15
 - 编码属性, 16-17
 - 评估多个候选对象, 29-31
 - 为意义格式化, 20-21
 - 泛型, 10-12
 - 信息, 8, 16-17
 - 长度, 18-20
 - 限制, 25
 - 循环迭代器选项, 12
 - 测量单位, 16
 - MinuteHourCounter 类改进, 167
 - Python 通过以下方式进行参数赋值, 63
 - 词语的特指性, 8-10

- 对于测试函数, 158-159
- if/else 中的否定情况, 与肯定情况相比, 72-73
- 嵌套, 77-79
 - 累积, 77
 - 通过提前返回移除, 78
 - 在循环内部移除, 78-79
- 非确定性行为, 161

O

- 差一错误, 25
- OpenBSD 操作系统, 向导模式, 29
- 或运算符, 86
- 代码顺序, 39-40

P

- 段落, 将代码分成, 41-42
- 性能, 与精度相比, 174
- 个人风格与一致性, 42
- 他人的视角, 169
- PHP
 - 读取文件内容, 112
 - 网页用户授权, 132
- 陷阱, 用注释预测, 53-54
- 纯英文
 - 代码解释, 132
 - 测试描述, 152
- 纯文本, 名称中的指示器, 17
- if/else 中的肯定情况, 与否定情况, 72-73
- 精度, 与性能, 174
- printf(), 153
- JavaScript 中的私有变量, 99
- 问题
 - 用注释预测, 53-54
 - 在测试代码中, 150
- 产品开发, 测试作为限制, 162
- 项目特定功能, 115
- 原型继承模式, 评估名称
 - 对于, 29-31
- 实体目的, 名称选择和, 10-12, 10
- Python
 - 按名称分配参数, 63
 - assert 语句, 155
 - 查找匹配数据库行的代码, 135-137
 - 包含敏感用户信息字典, 117
 - 列表和集合, 144
 - 无嵌套作用域, 100
 - 或运算符, 86
 - 读取文件内容, 112
 - 用于清理代码的结构化习惯用法, 76
 - unittest 模块和测试方法名称, 159

Q

- 问题, 通过评论进行预测, 52

R

范围

- 包含, 第一个和最后一个, 26
- 包含/排除, 开始和结束, 26-27

可读性

- 错误消息和, 154-156
- 基本理论, 3
- 测试代码和, 150-153
- 变量和, 94-106

读取文件内容, 112

冗余检查, 作为注释, 63

冗余代码, 170

重构: 改善现有代码的设计 (福勒), 119

正则表达式

库, 153

预编译, 115

移除未使用的代码, 143

需求、问题和分解, 140-141

返回值, 名称, 10

从函数提前返回, 75-76

通过移除嵌套, 78

反向迭代器, 171

Ruby, 或运算符, 86

--run locally 命令行标志, 14-15

S

作用域

全局作用域, 在 JavaScript 中, 100

C++ 中的 if 语句, 98

名称长度和, 18

变量的作用域, 缩小, 97-102

安全漏洞, 名称和, 17

Python 中的集合, 144

ShiftOldEvents() 方法, 173

滥用短路逻辑, 86

信号/中断处理程序, 80

代码轮廓, 36

Smalltalk 最佳实践模式 (Beck), 119

词语的特指性, 名称选择和, 8-10

语句, 分解, 89

静态方法, 98

统计, 递增, 128-130

股票购买, 记录, 134-137

企业门店定位器, 140-141

Stroustrup, Bjarne, 75

子问题代码提取, 110-118

findClosestLocation() 示例, 110-111

通用代码, 112-114

项目特定功能, 115

简化现有接口, 116

做得太过分, 117

实用代码, 111-112

摘要注释, 42, 55

摘要变量, 84-85, 89

“表面”改进, 5

T

任务

从对象中提取值, 124-128

多个与单个, 122-130

大小, 123-124

UpdateCounts() 函数示例, 128-130

临时变量, 94

三元运算符, 73-74

测试代码

创建最小语句, 152

其中的辅助方法名称, 159

定位问题, 150

可读性, 150-153

测试驱动开发 (TDD), 160

测试, 150-163

CheckScoresBeforeAfter() 函数, 153

选择好的输入值, 156-158

代码开发和, 160

走得太远, 162

和好的设计, 161

识别问题, 159-160

大型输入, 157

功能的多次测试, 158

测试函数的名称, 158-159

网站更改, 29

文本编辑器, 单词补全命令, 19

TextMate, 单词补全命令, 19

线程, 80

时间, 理解代码的要求, 3

时间敏感系统, 176

tmp 变量, 替代方案, 11

TODO: 标记, 50

自顶向下编程, 114

TrailingBucketCounter 类, 176-177

true, 27

错别字, 列对齐查找, 39

U

下划线, 带下划线的名称, 20

Unix 工具, 144

UpdateCounts() 函数, 128-130

网页的用户授权, PHP 用于, 132

用户信息, 带有敏感信息的 Python 字典, 117

用户, 匹配期望, 27-28

实用代码, 提取, 111-112

V

值, 从对象中提取, 124-128

var 关键字 (JavaScript), 100

变量

- 类成员, 97
- 消除, 94-96
- 消除中间结果, 95, 101, 105
- 解释, 84
- 全局, 可测试性, 161
- 对可读性的影响, 94-106
- 名称中的计量单位, 16
- 向下移动定义, 101-102
- 名称, 8
- 定义的顺序, 39-40
- 私有, 在 JavaScript 中, 99
- 缩小范围, 97-102
- 总结, 84-85
- 交换, 何时选择名称, 11
- 临时的, 94
- 一次写入, 103-104, 106
- Vi, 单词补全命令, 19
- 虚拟方法, 80

W

- 网页, PHP 用于用户授权, 132网络服务器
- , 跟踪传输的字节数 (参见分钟/小时计数器)

- 网站, 测试更改的实验, 29

- while 循环

- 参数的顺序, 70

- 与 do-while 循环相比, 75

- 单词补全命令、长名称和, 19

- 包装函数, 116

- 一次写入变量, 103-104, 106

- 写作障碍, 评论和, 56

X

- XXX: 标记, 50

关于作者

虽然在马戏团长大大，**Dustin Boswell** 却很早就意识到他更擅长计算机而不是杂技。Dustin 获得了加州理工学院的学士学位，在那里他迷上了计算机科学，然后去了加州大学圣地亚哥分校攻读硕士学位。他曾在 Google 工作五年，参与了包括网络爬行基础设施在内的各种项目。他构建了许多网站，并喜欢研究“大数据”和机器学习。Dustin 现在是一个互联网创业迷，他把空闲时间花在圣莫尼卡山脉徒步旅行和做一个新手爸爸上。

Trevor Foucher 在微软和谷歌从事大规模软件开发工作超过 10 年。他目前是谷歌搜索基础设施的工程师。在业余时间，他参加游戏大会，阅读科幻小说，并担任他妻子时尚创业公司的首席运营官。Trevor 毕业于加州大学伯克利分校，获得电气工程和计算机科学学士学位。

版本说明

封面图片来自 Getty Images。封面字体是 Akzidenz Grotesk 和 Orator。正文字体是 Adobe 的 Meridien；标题字体是 ITC Bailey。